

VECTOR



*The Journal of the
British APL Association*

144 pages of the best in APL ...

- First Impressions of Dyalog 8.2 26
- Eric Iverson at the BAA 49
- John Scholes on Threads 73
- Smith on HTML Imagemaps 85
- Reiter on Fractals 104
- Crossley on Data Migration 138

A Specialist Group of the British Computer Society

ISSN 0955-1433

www.vector.org.uk

Vol.15 No.2 October 1998

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, in workspaces from I-APL, APL+Win, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from Vector Production), and MS Word.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 1998-99

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	10	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or ICB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

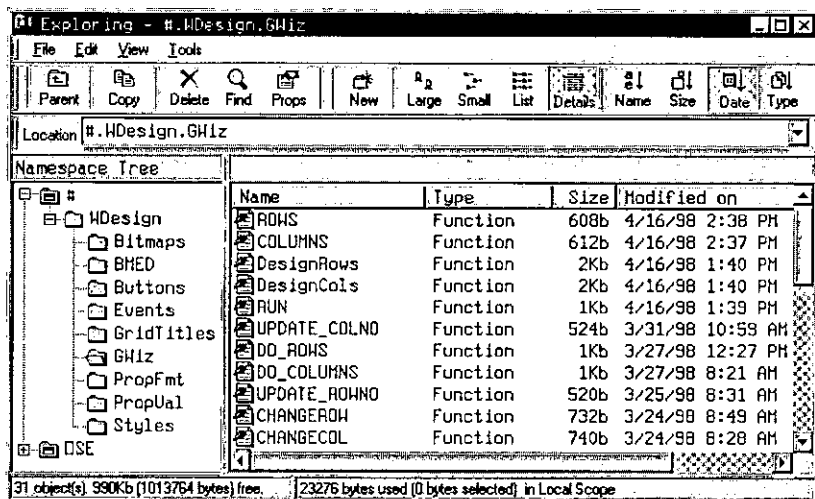
Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385 Email: apl385@compuserve.com

Contents

	Page
Editorial: Dedicated to my Unforgettable Cat	3
APL NEWS	
Quick Reference Diary	5
Correspondence	7
World APL News	9
APL Product Guide	Gill Smith 12
Dyalog APL 8.2 - First Impressions	Adrian Smith 26
The Education Vector	
Zark Newsletter Extracts	edited by Jon Sandles 36
J-ottings 18	Norman Thomson 43
RECENT MEETINGS	
Eric Iverson at the BAA	notes by Adrian Smith 49
BAA Vendors' Forum	notes by Jon Sandles 55
Using Threads - APL98 Workshop	Dyadic Systems 61
GENERAL ARTICLES	
Threads: An Introduction to Multithreading	John Scholes 73
HTML Basics for APLers (VI) — Image Maps	Adrian Smith 85
TECHNICAL SECTION	
Hacker's Corner: Context Help and COM in +Win	Ajay Askoolum 96
At Play with J: Maximum Infix Sums	Gene McDonnell 100
Word of Words from Iterated Function Systems	Reiter <i>et al</i> 104
Data Conversion Between APL Versions	David Crossley 121
Index to Advertisers	143

dyalog APL

The Definitive APL for Windows™



Dyalog APL Namespaces let you ...

- ➔ Organise your workspace into **self-contained** sub-systems
- ➔ **Encapsulate** functions and variables *within* GUI objects
- ➔ **Isolate** utilities from your application code
- ➔ Construct **COM/DCOM** object hierarchies
- ➔ Support **multiple instances** in APL web server applications

That's why Dyalog APL/W remains the **professional** choice. For further information, contact Dyadic or your local distributor today.

Dyadic Systems Limited, Riverside View, Basing Road, Old Basing,
Basingstoke, Hants. RG24 7AL, United Kingdom.
Tel:+44 1256 811125 Fax:+44 1256 811130 Email: sales@dyadic.com

Microsoft is a registered trademark and Windows and the Windows Logo are trademarks of Microsoft Corporation



Editorial: Dedicated to my Unforgettable Cat

by Stefano Lanzavecchia

I love the Internet.

I am sure that people have already written hundreds of books on the phenomena but I don't buy this kind of books, so here is my point of view. First of all I'd like to point out that with the term *Internet* I am referring to anything that is delivered via the TCP-IP protocol and not just the World Wide Web; among the others: real-time and delayed messaging, file archives, chat forums, video and audio streams, online commerce.

You will certainly have noticed that the bibliography accompanying my last editorial was exclusively made of Uniform Resource Locators (URLs in short), whereas a couple of years ago it would have consisted of titles of books and references to chapter or page numbers. Even the editorial itself can now be found on the revamped Vector web site [1]. In my opinion this is an amazing step forward.

How much can be found on the Internet? This is a challenge to me: give me something on which you want some information and one hour to find it on the internet. I hardly ever fail. If I do fail, the reason is that the piece of information required is too recent to have already been put on the net. It is just a matter of time. Hundreds of new sites are being added every hour and one will probably be devoted to your new area of interest, whether it is only curiosity or a business issue.

I will give you an example. Only a couple of weeks ago I was asked the meaning of a Spanish word. You will be surprised to know that Spanish is not as close to Italian as it seems and that I do not want to learn Spanish. For these reasons I was not able to give a translation on the spot. But in less than ten minutes I found an online Spanish-English dictionary on the Web, and found a couple of translations for the word. Since the context from which the word had been extracted was the description of a piece of software, it was easy to choose the right meaning.

Was I lucky? Am I really that good with search engines? Neither, but more about this a bit later.

What can be found on the net? I am very tempted to reply "everything", but since this would be a meaningless answer, I will produce a short list of things that I looked for (or stumbled into) and found on the big Net: multilingual dictionaries including a Japanese to English online dictionary; lessons to learn the Japanese language; an excellent free Japanese Word Processor; a radio with live broadcast from Singapore; the specs of all the internet protocols in use now, with the history of their development; the theoretical explanation behind ray-tracing algorithms and the source code to implement them; discussion groups in Italian about the works of Rumiko Takahashi; the lyrics of the songs from a record by "The Divine Comedy"; new friends; old friends I had lost; the latest patch to Microsoft's Office suite; the beta version of Internet Explorer 5; the English translation (online!) of a novel very difficult to find in Europe; the recipe for Spaghetti alla Carbonara; a detailed map of London; the satellite picture of the beach I used to go to when I was a kid...

How does one go about finding all these wonders? Sure there are a couple of tricks that must be learned before one can find his way in an archive without index, which now contains several terabytes of data [2]. Have patience, be brave (bytes don't bite), keep your mind opened, feel the links. I will return to the qualities of the perfect Web surfer another time, but the main point remains that the Internet is simple.

Why am I so excited about the Internet? There are many reasons, but to me the most attractive reason to spend time on it is that the quality/cost ratio is fantastic. All the examples I have given are of completely free stuff. In general I stay away from sites that ask for money to give me the information I need. I know that there are some intrinsic costs associated with the surfing but apart from the price of the computer (which we need anyway), and of the telephone call (almost irrelevant in "civilised" countries like the U.S.), the only real cost is the time. Can we, busy professionals, afford the time to learn new technologies, new ways of thinking, new ways of searching, collecting and assimilating information? I consider it an investment for the future. And if your answer is "no" why are you reading *Vector*?

References

- [1] Vector's home page: <http://www.vector.org/>
- [2] Alexa in the news: <http://www.alexa.com/company/inthenews/loc.html>

Quick Reference Diary 1998-2000

Date	Venue	Event
August 10-14 1999	Scranton, Penn.	APL99
Late July 2000	Berlin	APL'00

The APL99 Conference is sponsored by SIGAPL and NYSIGAPL and is now in cooperation with ACM/SIGPLAN. The scope of the conference will be extended to include all array-processing languages including Mathematica, MATLAB, Array Based Fortran, SISAL and SAC. SIGPLAN members will be able to attend the conference at member rates. We are grateful to acknowledge the Arthur J. Kania School of Management at the University of Scranton for providing the conference facilities.

The deadlines for paper abstracts and exhibits have been extended to December 31, 1998.

An electronic mailing list for information about APL99 has been set up. Subscription information is available on the web site

<http://www.acm.org/sigapl/apl99.htm>

Corrections: Late/On Site Registration rates should be \$325(US) for ACM/SIGAPL/SIGPLAN members and \$375(US) for non-members. Dorm registration will be handled by APL99; please do not call the university telephone number listed in the brochure.

We are tentatively planning to schedule pre-conference tutorials on Tuesday, August 10, 1999.

Dates for Future Issues of VECTOR

	Vol.15 No.3	Vol.15 No.4	Vol.16 No.1
Copy date	4th Dec	5th March	4th June
Ad booking	11th Dec	12th March	11th June
Ad Copy	18th Dec	19th March	18th June
Distribution	January 99	April 99	July 99

APL Causeway

NewLeaf 1.5c – 23rd November 1998

Some of the recent additions to NewLeaf

- Groups as logical units which will be kept together if possible
- Overlays to allow arbitrarily complex master pages
- Metafiles as master page classes, or in-line with optional wraparound
- Gridlines may be set for each row of data (List and Spread)

Some other interesting changes are:

- Much faster setting of simple tabular data
- Optional HTML and CSV output in parallel with main report

Price to new customers £400; update price (serial before 97042) £50

RainPro 4.3c – 4th September 1998

- Horizontal barcharts
- QuadFMT specification for all numeric labels and data values
- Data hints, data query, drill-down and graphical editing
- Wrapping keys and columnar key layout
- Bitmaps and metafiles as decorations and backdrops

Price to new customers £250; update price (serial before 97042) £50

CausewayPro 1.3d – 28th October 1998

Here are some of the recent additions to Causeway

- Radio-groups may be defined in menus and pop-ups
- In-place editing (trees and listviews); draggable items (listviews)
- "What's This" help available on dialogue boxes
- many small enhancements to the Designer and Class Library.

Price to new customers £400; updates are free for the first year

CORRESPONDENCE

Measures of Programming Language Efficiency

From: Joseph De Kerf

30 September 1998

In 1997, M.H. Halstead of Purdue University published a book on a theory allowing to measure characteristics of software and their relations [1]. One of the parameters is the programming language level λ . Programming time is inversely proportional to the square λ^2 of this language level λ . As such, the square λ^2 is a measure for programming efficiency.

In a paper, published in the ACM SIGPLAN Notices in 1986 [2], programming language level λ and efficiency λ^2 of simple array APL (APL/SA) were compared with those for other the languages, published by Halstead. According to the results, APL/SA would be about 12 times as efficient as Assembler. This result wonderfully agrees with an experimental result reported at the SEAS APL Working Committee's Meeting in 1971 [3].

In a paper, published this year in the ACM SIGPLAN Notices [4], the investigation is extended to nested array APL (APL/NA). According to the results, APL/NA would be about 17 times as efficient as Assembler, giving for APL/NA versus APL/SA an extra increase of programming efficiency of about 39%. An overview of the results is given below.

Language	λ	λ^2	λ^2/ASS
APL/NA	3.62	13.07	16.88
APL/SA	3.07	9.43	12.17
PL/1	1.53	2.34	3.02
Pascal	1.53	2.34	3.02
Algol 58	1.21	1.46	1.89
Fortran	1.14	1.30	1.68
Pilot	0.92	0.85	1.09
Assembler	0.88	0.77	1.00

Both investigations have been reported in the ACM SIGPLAN Notices in order to address not only the APL community, but also all those who are interested in programming languages.

References

- [1] M.H. Halstead: *Elements of Software Science*. Elsevier Computer Science Library; Elsevier North-Holland, New York 1977
- [2] J.L.F. De Kerf: *APL Compared with other Languages according to Halstead's Theory*. ACM SIGPLAN Notices Vol.21 No.1, January 1986, pp. 31-39
- [3] *Abstract of the Proceedings at the SEAS APL Working Committee's Meeting, Grenoble, February 15-16, 1971*. APL Quote-Quad, Vol.3 No.1, June 11 1971, pp. 14-15
- [4] F.H.D. van Batenburg, E. de Ridder & J. De Kerf. *APL Extended compared with other Languages according to Halstead's Theory*. ACM SIGPLAN Notices, Vol.53 No.6, June 1998, pp. 54-60

Vector Back Numbers

Back numbers of Vector are available from:

British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO6 4JJ

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

World APL News

Press Release — Adaytum Software (9th November 98)

Adaytum Software Announces Industry's First Financial Planning and Modelling Solution To Support Microsoft SQL Server 7.0 OLAP Services

The new version of Adaytum Planning will use the OLE DB for OLAP interface to integrate with Microsoft SQL Server 7.0 OLAP Services. It firmly positions Adaytum in the forefront of the rapidly growing financial management applications market providing users with new levels of speed, flexibility, and power. First release of the new product is scheduled for February 1999.

SQL Server OLAP Services and Adaytum Planning

"Adaytum Planning's dynamic modelling flexibility and rich, top-down planning functionality are an ideal complement to Microsoft SQL Server's OLAP Services," said Sue Strother, chief marketing officer for Adaytum. "By tightly integrating Adaytum Planning with SQL Server OLAP Services, we will provide a high-performance, scalable solution for enterprise planning and budgeting."

"We are pleased that Adaytum Software is announcing support for SQL Server OLAP Services to bring cost-effective scalability to enterprise customers," said Andrew Hoover, Microsoft's group industry marketing manager — data warehousing. "The combination of SQL Server OLAP Services and Adaytum Planning's sophisticated budgeting, planning, and analysis tools will provide a new class of financial planning and modelling applications for budgeting and planning managers."

Press Release — APL2000 bought by SS&C (19th October 98)

SS&C Further Invests in PTS 2000 Announces Letter of Intent to Acquire Tool Business of APL2000 Inc.

SS&C Technologies, Inc. announced today it has signed a non-binding letter of intent to acquire the APL2000 tools business of Princeton, NJ-based APL2000 Inc., a wholly owned subsidiary of LEX2000 Inc., of Atlanta, Georgia.

APL2000 is the creator of the programming language APL, which is used in SS&C's PTS 2000 system. As part of the proposed acquisition, Eric Baelen, President of APL2000 Inc., and recognized around the world as one of the leading experts on APL, will join SS&C.

"This is an important strategic acquisition for SS&C," stated William C. Stone, Chairman and Chief Executive Officer at SS&C. "Acquiring APL2000 and its high-powered development team will enable us to enhance PTS 2000. SS&C's ability to leverage the APL expertise and abilities of Eric Baelen and his strong team of developers represents further commitment to our Analytics Business Unit and, in particular, to world-class asset/liability management software."

"This continues LEX2000's strategy of focusing our resources on our Financial Reporting Software. SS&C is an excellent software company which will provide the APL2000 team with the ability to grow," stated Robert F. Reisner, Chief Executive Officer of LEX2000.

APL is a powerful, symbolic programming language. In fact, OLAP technology has its roots in the APL language. The latest version, APL+Win Version 3.0, released in May of this year, delivers a robust development toolset, including Inter/intranet support, a new ODBC driver, and support for Microsoft's COM (Component Object Model), making APL interoperable with other programming languages and applications. APL enjoys a worldwide following of application developers. These developers use APL to build a range of database and computational programs that serve a wide variety of industries, including financial services, and insurance.

"We are excited to join forces with SS&C," stated Eric Baelen, President of APL2000, Inc. "We have been impressed by SS&C's commitment to superior technology and their ability to rapidly bring product to market. We believe we will be able to add strategic value to SS&C's PTS 2000 product, and continue to advance the APL language."

The proposed acquisition will allow SS&C to offer a full complement of consulting and technical services for APL users, including application development and enhancement, optimization, assistance in porting APL applications across platforms, and training.

The transaction is subject to the negotiation and execution of definitive documents, and the receipt of customary approvals. The Company expects the transaction to close by the end of November 1998.

Codework — Press Release (November 6th, 1998)

The Codework company is proud to announce the release of *WinHELM version 1.1 for Win 3.x, Win 95 and Win NT*. Earlier versions of HELM have been around for over ten years. (MS-DOS version is still supported.)

WinHELM is a DSS-OLAP product for the Windows platform. Basically, it is a versatile and powerful handler of large multi-way tables (a.k.a. hypercubes).

Although this version is for the smaller machines, several million records can be loaded and then respond to queries within seconds. Performance is orders of magnitude better than with any record-and-field oriented database.

Manuals and evaluation version can be downloaded from

<http://www.inrete.it/cdwk/eng/homee.html>

The evaluation version entails all services, except database creation. After a (free) registration via Email, such module can also be enabled.

The product is well suited for

- fast prototyping, impromptu requests — students and teachers — developers interested in DSS-OLAP applications for single PCs and LANs
- business analysts that must deliver answers within minutes in unreliable or unfriendly environments (those who must “take the tool with them”)
- those who need a hands-on experience of DSS-OLAP issues with an investment of a day or two (as opposed to months of training, large UNIX machines and the assistance of numerous wizards)
- a natural growth path to more challenging processing, via open-ended user languages.

WinHELM is not a strong candidate for

- company-wide, network-centric Terabyte-range OLAPs
- a workbench for mixing and matching all sorts of last-minute software technology (*WinHELM*'s mission as a “survival kit” makes it very cautious in adopting new technology)
- custom GUI interface development (that is, *WinHELM* has its own GUI interface through which you do ANY processing — *WinHELM* offers only basic support for building your own GUI applications, as you would do with V.B.A. or Delphi, for example).

Send any comments to codework@inrete.it please.

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete Systems (Hardware & Software)
- APL and J Interpreters
- APL-based Packages
- Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

*Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.
Please help us keep it up to date!*

All contributions and updates to the Vector Product Guide should be sent to Gill Smith, at Brook House, Gilling East, York, YO6 4JJ. Tel: 01439-788385, Email: apl385@compuserve.com

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 18 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 18 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
Optima	IBM Compatible	poa	Complete networked or stand-alone solutions including configuration installation, maintenance and commissioning.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Software	APL*Plus/PC Release 10	450	STSC's APL for IBM PCs & compatibles. Upgrades from earlier releases also available.
	Run-time	poa	Closed version of APL*Plus/PC which prevents user exposure to APL.
	APL*Plus II	1,395	All the features of mainframe APL*Plus for your 386PC!
	Run-time Dyalog APL	1000-10,000	2nd generation APL for Unix systems
	APL2/PC	poa	IBM's APL 2 for the PC.
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
The Bloomsbury Software Company	APL+PC Version 11	250	Upgrade to version 11 gives free runtime (£120 from any version)
	APL+Win v3.0	1350	A 32-bit Windows-hosted interpreter that runs under all Windows platforms including Windows 95. Note: Upgrade for £350 from version 1.8 or 2, £920 from version 1.0
	Migration to APL+Win	1060	from APL*PLUS/PC APL*PLUS/DOS any version. from earlier versions of APL*PLUS II
	APL+DOS	1250	APL*PLUS II DOS is renamed to APL+DOS.
	Migration to APL+DOS	760	from APL*PLUS/PC
	APL Link	200	Database Access
	APL Link Pro	500	

	APL*PLUS II for UNIX	poa	APL2000's 2nd generation APL for all major Sparc and Rise Unix workstations.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL.
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
Dyadic	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 8150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle Interface available for IBM, Sun and Xenix versions.
IAC/Human Interfaces			
	I-APL/Mac	13	Macintosh version of I-APL.
I-APL Ltd	I-APL/PC or clones	8	ISO conforming Interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	8	As above
	Strand Software Inc		Strand Software Inc has the sole selling rights to Iverson Software Inc products. I-APL stocks a few of these (mainly APLWIN and the personal J products and books), but is no longer an agent.
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	APL2 PC (US Version)	\$630	Product No. 5799-PGG. PRPQ Number RJ0411. Order from 1-800-IBM-CALL.
	APL2 PC (European Version)	£348	Product No. 5604-260. Part number 38F1753. From all IBM dealers, including MicroAPL.
	APL2 for OS/2 Entry Edition	\$185	Part No 89G1556.
	APL2 for OS/2 Advanced Edition	\$650	Part No 89G1697. Contains all facilities of the Entry Edition plus: DB2 interface; co-operative processing TCP/IP interface; tools for writing APs; TIME facility.
	APL2 for Windows version 1.0	\$1500	Product No. 5639-d46, part number 4229558.
	APL2 for Sun Solaris	\$1500	Product No. 5648-065.
	APL2 for AIX 6000	poa	Product No. 5765-012.
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390.
	APL2 Application Env't Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages.
Insight Systems	APL*PLUS/PC	poa	APL systems marketed and supported ...
	Dyalog APL	poa	from: Dyadic, Manugistics, IBM
	APL2	poa	under: Windows, OS2 and Unix
Iverson Software Inc.	J on the Web online registration ...		
	J Educational Edition	\$50	
	J Standard Edition	\$150	
	J Professional	\$325	
	Books and accessories (discounts for reg users)		
	J Dictionary	\$50	

	J User Manual	\$50	
	J Phrases	\$40	
	J Primer	\$40	
	Set of the above 4 books	\$160	
	Concrete Math	\$40	
	Fractals, Visualization & J	\$50	
	Exploring Math	\$50	
	J User Conference Proceedings	\$35	
	Mugs, T-shirts, Mousepads	\$10 each	
	APLWIN	\$70	For 386/PC under Windows 3.1
J Austria	J	poa	Distributor for Austria and Switzerland
	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor
	Structural Analysis Software	poa	Complete package by IG Zenkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also
	APL+Unix	poa	distributes in Switzerland and Belgium. Call for price quotes.
	APL+DOS	poa	
	APL+Win	poa	
	Dyalog APL/W	poa	French distributor for Dyalog
MasterWork Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APL68000 Level I	2000	First generation APL with numerous enhancements. Multi-user version (Unix, Mirage, MCS).
	APL68000 Level II	2500	Second generation APL. Nested arrays, user defined operators, selective specification etc. Multi-user version (Unix, Mirage, MCS)
	APL68000/X	1500-6000	Second-generation APL. Nested arrays, user defined operators, selective specification, etc. Multi-user AIX version with full OSF/Motif support.
	APL68000 Level I Mac, ST, Amiga	87	First generation APL. Single user, full windowing interface, software floating point support.
	Mac, Amiga	260	First generation APL. Single user, full windowing interface, hardware floating point.
	APL68000 Level II ST	170	Second generation APL. Full windowing interface, software floating point support.
	Amiga	260	Second generation APL. Full windowing interface. Hardware and software floating point support.
	Mac	520	Second generation APL. Full windowing interface. Hardware and software floating point support.
	APL*PLUS Rel 10	450	
	APL*PLUS II V 4.0	1395	
Oasis	Dyalog APL	poa	Dyadic Systems
	APL*PLUS	poa	Manugistics
	APL68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Optima	Dyalog APL/W	995	Fully fledged Windows development environment.

RE Time Tracker Oy	APL+PC (APL*PLUS/PC)	poa	Complete APL+ and Statgraphics product range and links to various 3rd party products.
	APL+DOS (APL*PLUS II)		
	APL+Win (APL*PLUS III), APL+Link		
	APL+UNIX		
	APL*PLUS Sharefile		
Sollton Associates	SHARP APL for MVS	poa	for IBM MVS mainframes
	SHARP APL for Unix	poa	for IBM RS/6000 and Sun SPARC
Strand Software	Canada		
	All APL*PLUS Products	poa	All APL*PLUS products including upgrades and educational.
	Dyadic and ISI products	poa	
	USA		
	Dyadic and ISI products	poa	

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
ADAPTA Software	MPS	poa	Master Production Scheduling
	FBS	poa	Forecasting and Budgeting System
	DRP	poa	Distribution Requirements Planning
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
Adaytum Software	Adaytum Planning	poa	Full-featured Budgeting and Financial Planning system for medium to large enterprises.
The APL Group	Qualedi	\$1500-4000	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
APL Software Ltd (mainframe)	RDS	poa	Relation Data Base System
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APL/W auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APL/W auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
The Bloomsbury Software Company (for VSAPL)	Enhancements & Sharefile	poa	Component files, quad-functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	Compiler	poa	The First APL compiler!

(for APL2)	Sharefile/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage.
Causeway	CausewayPro for Dyalog/W	400/\$600	Causeway application development platform for Dyalog APL/W.
	RainPro Business Graphics	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability, Web publishing and clipboard support to the shareware product. Charts can be included in <i>NewLeaf</i> reports. Functionally compatible across Dyalog/W and APL+Win.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+Win (1.8 and above)
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
CODEWORK	HELM	poa	Decision Support System for top management. Handles large multi-dimensional tables, data analysis, EIS presentations, generates HTML and Latex output. Platforms: MS-DOS, LAN, Windows 3.1/95/NT. Ideal for APL customisation (APL*PLUS II and Dyalog APL); more than 150 installed.
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
HRH Systems	APL Utilities	poa	Software to transfer workspaces between APL*PLUS and Sharp, and between APL*PLUS and I-APL. Software to import IBM .ATF files to APL*PLUS.
	APL*PLUS Utilities		Public domain software, unlock locked fns, a user-friendly alternative to locking, fns of mathematical physics, menus, and others.
IAC/Human Interfaces	SPARKS	poa	Educational simulation of electric circuit (for Apple Mac.)
	EPIDEMIC	poa	Educational simulation of spreading infection (for Apple Mac.)
	COINS	poa	Educational simulation (KSS) of coin-tossing experiment with simple stats (for Apple Mac.)
	FIBONA	poa	Educational simulation of Fibonacci's rabbits (for Apple Mac.)
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson, Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
IBM APL Products	A Graphical Statistical System	\$250	for DOS, Product Number 5764-009
	(AGSS)	\$500	for Workstations (OS/2, Abx, Solaris), Product Number 6764-092
		\$2500	for CMS, Product Number 5764-011
INFOSTROY	APL*PLUS/Xbase Interface (I/386 Version 2)	\$198	Complete package written in C. Comparable with the data, index & memo files of FoxPro, dBASE, & Clipper. Multi-user support. No DBMS license required.
	(DLL Version 1)	\$198	The same in a DLL form! Gives your Windows applications all advantages of DLLs.
Insight Systems	IUTILS/XP	20-95	Cross-platform utility library including simple OS calls (DIR, COPY, DEL, RENAME) and DATE functions. For APL*PLUS II, APL2 and Dyalog APL under Windows, OS/2 and Unix.
	ASI	95	APL Spreadsheet Interface. "Device-Independent" spreadsheet driver supporting Excel, 123 and Quattro-Pro for Dyalog APL/W
	WinCom	95	Asynchronous comms package for Dyalog APL/W
	S2D,22D,X2X	poa	Advanced APL syntax analysis and conversion packages from Sharp and APL2 to Dyalog, and between any two APLs

	SQAPL Client	poa	Interface from APL*PLUS II, APL2 and Dyalog (Windows, OS/2 or Unix) to most SQL databases over most networks.
	SQAPL Server	poa	Makes APL*PLUS II, APL2 or Dyalog APL (Unix) available as Sequelink servers. Can be called from SQAPL clients or other applications such as Excel, C++, Smalltalk, Visual Basic.
JAD Software	JAD SMS	150-500	Software management system for APL*PLUS II based on hierarchical databases; includes full-screen interface and stand-alone functions. Price depends on number of users.
Lescasse Consulting	APL+Win Monthly Training Program	\$600	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
	Advanced Windows Programming ...	\$95	200-page book plus companion disk on interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.
	DLL parser for APL	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and Dyalog/W.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
	APL+Link Pro	poa	ODBC Interface for APL+Win
	SQAPL Pro	poa	ODBC Interface for Dyalog APL/W
	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
Lingo Allegro	FRESCO Business Graphics	poa	Fast and easy business graphics DLL
	AP125/PC	poa	GDDM Interface for Dyalog APL/W
	AP127/PC	poa	ODBC Interface for Dyalog APL/W
	AP119/PC	poa	TCP/IP interface for Dyalog APL/W
	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	ISAP	poa	MS Windows DLL for calling Dyalog APL/W from web pages via MS Internet Information Server. Visit www.lingo.com for a demo.
RE Time Tracker Oy	UIT/W	poa	Comprehensive high-level Windows User Interface library for APL+Win and +W v.5.1. Comprehensive spreadsheets, replicated fields, special field types, etc. 16 and 32 bit versions available.
	AJGRAPH	poa	Graphpak-compatible 2D graphics package for +Win and +DOS. Includes multi-window support, print and metafile support. No DLLs required.
	ECCO PRO with APL	poa	Leading group and personal information management system with comprehensive customising. Supplied with sample +Win workspace to interface to ECCO databases via DDE.
	NEWT TCP/IP SDK with APL	poa	Lead TCP/IP SDK with interfaces to all protocols. Supplied on 3 CD ROMs together with a sample +Win workspace.
	DB+	poa	Database interface for APL+DOS under Windows. Allows combining character-based APL applications with ODBC-compliant databases such as Oracle and SQL-server.
Sollton Associates	LOGOS	poa	Application Development Environment
	MAILBOX	poa	Electronic Mail
	VIEWPOINT	poa	Report generator with interfaces to DB2 and MVS data
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.

Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Ajay Askoolum	Consultancy	poa	APL+Win development and migration of actuarial, financial, mathematical applications.
Andrews	Consultancy	poa	APL programming and analysis, Year-2000 legacy systems, algorithms, tree-processing.
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1969.
AUSCAN Software	Consultancy	poa	APL software development, training
Bloomsbury Software	Consultancy	300-750+VAT	
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
Paul Chapman	Consultancy	250-500	24-hour programmer: APL, Smalltalk, C; Windows front end design a speciality.
CODEWORK	Consultancy	poa	Development, maintenance, migration, documentation of APL applications. Speciality: info systems for top executives, internet applications.
Dinosoft Oy	Consultancy	poa	Specialised in very large databases.
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
Entropy Software	Consultancy	poa	Company reporting, business graphics, Windows applications with Dyalog APL/W
Evestic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
General Software	Consultancy	from 120	
Godin London Inc	Software Development	poa	We have applications in the food manufacturing field, travel agency and airline bookings field and in product lease management.
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Hoekstra Systems Ltd	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
Michael Hughes	Consultancy	poa	Consultant with 10+ years experience with various APL interpreters and C.

IAC/Human Interfaces	Consultancy	poa	APL on Macintosh & PC. HCI design. VDU ergonomics: EC/Health & Safety compliance.
	Documentation	100-200	On-line assistance, product demos & mock-ups, manual writing; foreign language software localization.
	Training	poa	Using I-APL for courseware & distance learning materials; Mac programming in C, APL & HyperCard.
INFOSTROY	Consultancy	poa	Moving applications between platforms. Client/server development. Multilingual user interface.
Insight Systems	Consultancy	poa	Experts in APL conversions between any combination of: APL*PLUS, APL2, Dyalog APL and Sharp APL. We are also experienced right-sizers, comfortable with networks and relational databases (that also means when NOT to use SQL) and client/server development in APL, C and Visual Basic.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
Phil Last	Consultancy	poa	APL consultancy, modelling and programming.
Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APL/W. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Lingo Allegro	Consultancy	poa	General APL consulting, internet website development, migration and downsizing, performance tuning, education and training.
Mackay Kinloch Ltd	Consultancy	200-400	Design, analysis and programming for banking, insurance, financial planning and modelling, corporate performance and legal reporting
MicroAPL	Consultancy	poa	Technical & applications consultancy.
Milinta Inc	Consultancy	poa	Design, development, maintenance, conversion, documentation in all APLs, most APs and some specific Sharp products (LOGOS, ViewPoint, Retrieve). Experience in multi-user, multi-task systems, databases, Y2K, Windows programming.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Oasis	Consultancy	poa	Expertise in APL system design, Project management, conversion, migration, tuning; for all APL versions (10+ years experience)
Object Oriented Ltd	Consultancy	poa	General APL consulting, code recycling — mainframe to PC, performance tuning.
Optima	Consultancy	poa	A range of consultants specialising in all areas of pharmaceutical, industrial and financial systems with 5-15 yrs experience on both PC and mainframe.
RadSys Technologies	Consultancy	poa	Areas of expertise: financial systems, risk analysis systems, healthcare systems.
RE Time Tracker Oy	Consultancy	poa	APL application conversions, APL Windows interfaces, APL to APL-level interfacing to any system under Windows, TCP/IP network and database connectivity.
Rex Swain	Consultancy	poa	Independent consultant, 20 years experience. Custom software development & training, PC and/or mainframe.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Shepp & Associates	Consultancy	poa	APL applications development and consulting, especially in the travel industry, especially on small computers. 25 years experience in APL programming.
Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.

Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.
Stephen Wynn	Consultancy	poa	Most experience of financial planning, and mathematical areas: operational research, quality control, experimental design.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification.
Bloomsbury Software	Training	poa	Contact the company for details.
ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
HMW	Employment	poa	Contractors and permanent employees placed.
HRH Systems	APL lessons		On-screen interactive APL lessons for APL*PLUS, TryAPL2, Sharp and I-APL — in English or French.
	The BBSIAPL	n/a	<i>The BBSIAPL is no longer available. For an email catalogue of its archive files (and how to get them) email dck.holt@juno.com.</i>
I-APL Ltd	An APL Tutorial	3	45pp by Alvord & Thomson
	An Encyclopaedia of APL (2d Ed)	6	228pp by Helzer
	APL in Social Studies	3	36pp by Traberman
	I-APL Instruction Manual (2d Ed)	3	55pp by Camacho & Ziemann
	APL Programs for the Mathematics Classroom (Springer-Verlag)	16	185pp by Thomson
	Programming in J	10	75pp by Ken Iverson
	Arithmetic	12	118pp by Ken Iverson
	Tangible math	8	36pp by Ken Iverson
	Sharp APL Reference Manual	42	349pp by Berry
	APL Press Books	poa	A comprehensive selection of early APL literature
	<i>Please note there is a packing charge of £3 per order</i>		
Oasis	Training	poa	Introductory courses in APL Advanced courses for different APL versions
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Soliton Associates	MVSJINK	poa	Interface from Sharp APL (Unix & MVS) to non-APL data and software in the MVS environment.
	SSQL	poa	High-performance DB2 interface for Sharp APL (Unix and MVS).

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM SigAPL	International	APL QuoteQuad	Conferences; APL white pages; web site	\$30
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$20
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Club Germany	Germany	APL Journal	Semi-annual meetings	DM60
Ass. Francophone pour la promotion d'APL France		Les Nouvelles d'APL		FF350 (private) FF2800 (Company)
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)

Capital PCUG	Washington, D.C.	Monitor	Monthly meetings, occasional classes	free
Danish SIG	Denmark			
Dutch APL Assoc.	Holland	Vector provided	Mini-congress, APL ShareWare Initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL 100FIM(private), 30(student), 1000 (Co)	
Japan APL Assoc	Tokyo	APL Journal	Monthly meetings (4th Sat)	10,000yen to join
Rome/Italy SIG	Roma, Italy			
RusAPL	Moscow, Russia	APL Club	Seminars and Annual Meeting	100,000R (students 20,000)
SE APL Users Grp	Atlanta, Georgia	SEAPL Newsletter	Quarterly meetings	\$10
SovAPL	Obninsk, Russia			
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG)	Bern	Part of Qtly SI-Info		SF60 (SI) + SF20 (SAUG)
Toronto SIG	Toronto, Canada	Glimme Arrays!	Monthly Meetings, APL skills database, J SIG, Toronto Toolkit	\$25

ADDRESSES

ORGANISATION CONTACT

ACM SigAPL	David Siegel
ADAPTA Software GmbH	Michael Baas
Adaptable Systems	Lois & Richard Hill
Adaytum Software	Douglas Rowley
Adfee	Bernard Smoor
Andrews	Dr Anne D Wilson
APL-385	Adrian Smith
APL Bay Area APLBUG	Curtis Jones (Sec)
APL Club Austria	Harald F. Nelson
APL Club Germany	Dieter Lattemann
The APL Group Inc	Stuart Sawabini
APL Solutions Inc	Eric Landau
Ajay Askoolum	Ajay Askoolum
Association Francophone pour la promotion d'APL	Ludmila Lemagnen
AUSCAN Software Ltd	Richard Procter
BACUS	Joseph De Kerf
Beautiful Systems, Inc.	Jim Goff
Bloomsbury Software	Peter Day
Camacho	Anthony Camacho
Ray Cannon	

ADDRESS, TELEPHONE, FAX, EMAIL etc.

ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
Marlenhoehe 88, 25451 Quickborn, Germany Tel: +49 4106 60977 Fax: +49 4106 67869 Email: 101523.1757@compuserve.com
49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: adsys@ibm.net
13 Great George Street, BRISTOL BS1 5RR UK Tel: 0117-921 5555
Dorpsstraat 50, 4128 BZ Loxmond, Netherlands. Tel +31 347 342 337 Fax: +31 347 342 342 Email: adfee@concepts.nl
12 Thomy Hills, Kendal, Cumbria LA9 7AL, UK Tel: 01539-731205
Brook House, Gilling East, York YO62 4JJ UK. Tel: 01439-788385 Fax: 01439-788194 Email: 100331.644@compuserve.com
228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: jonesca@vnet.ibm.com
c/o N-TECH, Siebenbrunnfeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5458063-17
Rheinstraße 23, D-69190 Walldorf, Germany. Tel: +49 6227-53469 Compuserve: 100332.1461
644 Danbury Road, WILTON, CT 06897 USA. Tel: +1 (203) 762-3933 Fax: +1 (203) 762-2108 Email: ssawabini@aol.com, eshaw@aplgroup.com
1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: elandau@cals.com
42 Hanworth Road, Redhill, Surrey RH1 5HT Tel: 01737 771043 Email: 106173.3347@compuserve.com
174 Boulevard de Charonne, F-75020 Paris, FRANCE Email: lemagnen@aol.com
PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: jtp@interlog.com
Rocinberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
308 Old York Road, Suite 5, Jenkintown, PA 19046, USA Tel: +1 (215) 886-2636; Fax: +1 (215) 886-4888
3-6 Alfred Place, Bloomsbury, London WC1E 7EB UK. Tel: 0171-436 9481 Fax: 0171-436 0524 Email: pd@bloomsbury-software.co.uk
11 Auburn Road, Redland, Bristol BS6 6LS UK. Tel: 0117-9730036. email: acamacho@cx.compulink.co.uk Reuternet (Sharp): ACAM
21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS UK Tel: 01252-874697 Email: 100430.740@compuserve.com

Causeway Graphical Systems Ltd	Adrian Smith	The Maitings, Castlegate, MALTON, North Yorks YO17 7DP UK Tel: 01853-698760 Fax: 01853-697719 Email: causeway@compuserve.com
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB UK. Tel: 0171-404 5401. Compuserve: 100343,3210
Cinerea AB	Rolf Kornemark	Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email rolf@cinerea.se
CODEWORK	Mauro Guazzo	Corso Cairoli 32, 10123 Torino, Italy. Tel: +39 11 885168 Fax: +39 11 812 2652 Email: codework@lnrete.it
ComLog Software	Jeff Pedneau	PO Box 5570, Derwood, MD 20855 USA Tel: +1 (301) 990-7063 Email: jeff@softmed.com
CPCUG	Lynne Sturtz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850-2421, USA. Tel: +1 (301) 762-9372 Fax: (301) 762-9375. Email: gjerlov@ibm.net
Danish User Group	Per Gjerlov	Lönnrotinkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 9 70028820 Fax: +358 9 70028824 Email: dinosoft@dinosoft.fi
Dinosoft Oy	Pertti Kalliojärvi	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342
Dutch APL Association	Bernard Smoor (Sec)	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL UK. Tel: 01256-811125 Fax: 01256-811130
Dyadic Systems Ltd.	Peter Donnelly	37 Newhouse Rd, Bovingdon, Herts, HP3 0EJ UK Tel: 01442-834015 Email: 100412.1305@compuserve.com
Entropy Software Ltd	George MacLeod	Bertellusvagen 12A, S-146 38 Tullinge, Sweden Tel&Fax: +46 778 4410 Email: oile.evero@mailbox.swipnet.se
Evestic AB	Oile Evero	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: oile.paavola@pyr.fi
FinnAPL	Olli Paavola	22 Russell Road, Northolt, Middx, UB5 4QS UK. Tel/fax: 0181-864 9537
General Software Ltd	M.E. Martin	12 Gerrard St., London, Ontario, Canada N6C 4C5 Tel: +1 (519) 679-8290 Fax: +1 (519) 438-8381 Email: info@godlin.on.ca
Godlin London Incorporated	Gaëtan Godlin	Hamilton House, 1 Temple Avenue, Victoria Embankment, London EC4Y 0HA UK. Tel: 0171-353 8900; Fax: 0171-353 3325; Email: 100020.2632@compuserve.com
H.M.W.Trading Systems Ltd		Dominique, Salsbury Road, Woking, Surrey, GU22 7UR UK Tel: 01483-771028 Email: bob.hoekstra@khamlin.demon.co.uk
Hoekstra Systems Ltd	Bob Hoekstra	3902 N Richmond St, Suite 271, Arlington, VA 22207 USA Tel: +1 (703) 528-7824; Fax: +1 (703) 528-7617; Email: dick.holt@juno.org
HRH Systems	Dick Holt	28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL UK. Tel: 01536-770999 Email: 101740.1203@compuserve.com
Michael Hughes		IAC Human Interfaces, Unit R21, Auckland New Business Centre, St. Helen Auckland, Bishop Auckland, County Durham, DL14 9TX. Tel: 01388 605544. Email: 100021.3073@compuserve.com
IAC/Human Interfaces	Ian A. Clark	11 Auburn Road, Redland, Bristol BS8 6LS UK. Tel: 0117-9760036 email: acamacho@clx.compulink.co.uk Reutemet (Sharp): ACAM
I-APL Ltd	Anthony Camacho (for queries, order forms)	56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU UK Tel: 01934-625181
J C Business Services (for pre-paid orders only)	Nancy Wheeler	APL Products, IBM Santa Teresa, Dept M46/D12, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 (=2752) Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com Cserve: GO IBMAPL2
IBM APL Products		3 S. Tulenln Lane, St. Petersburg 191186 Russia. Tel: +7 812 312-2673 Fax: +7 812 311-2184 Email: aim@infostroy.spb.su
INFOSTROY	Alexei Miroshnikov	Nordre Strandvej 119C, DK-3150 Hellebæk, Denmark Tel: +45 49 76 20 20 Fax: +45 49 76 20 30 Email: info@insight.dk
Insight Systems ApS	Morten Kromberg	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel: +1 (416) 925-6096; Fax: +1 (416) 488-7559 Email: info@jsoftware.com
Iverson Software Inc.	Eric Iverson	580 Eyer Drive, #81 Pickering, Ontario, Canada L1W 3B7 Tel: +1 (905) 837-1895 Fax: +1 (905) 831-5172
JAD Software	David Crossley	1-8-13 Masujima Bldg.6F Higashi Gotanda Shinagawa-ku, Tokyo Japan 141-0022 Tel: +81 (03) 3280-0411 Fax: +81 (03) 3280-0418 Email: KY00361@niftyserve.org.jp
Japan APL Assoc	Toshio Nishikawa	c/o Causeway Graphical Systems. Email: JoHo@ping.at
J Austria	Joachim Hoffmann	

Mackay Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK Tel/Fax/Answerphone: 0131 228 3580 Pager/Voicemail: 01426 98 3858 Email: Alastair_Kinloch@compuserve.com
Phil Last Ltd	Phil Last	146 Crossbrook Street, Cheshunt, Herts, EN8 8JY UK. Tel: 01992 633807 Fax: 0121 359 0375 Email: phil_last@compuserve.com
Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France Tel: +33.1.46.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescasse.com
Lingo Allegro USA, Inc	Steven J Halasz	1105 Chicago Avenue, Suite 155, Oak Park, IL 60302, USA. Tel: +1 800 546 4621-1 Email: sjhalasz@interaccess.com
Mercla Software Ltd.	Gareth Brentnall	Holt Court North, Heneage Street West, Aston Science Park, Birmingham B7 4AX UK. Tel: 0121-359 5096. Fax: 0121-359 0375
MicroAPL Ltd.	Richard Nabavi	South Bank Technopark, 90 London Road, LONDON SE1 6LN UK Tel: 0171-922 8868 Fax: 0171-928 1006 Email: MicroAPL@microapl.demon.co.uk
Millint Inc.	Dan Baronet	4215 St-Andre, Montreal, CANADA H2J 2Z3. Tel: +1 (514) 529-1434
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE UK. Tel: 01730-263843 Email: Ellis@mrtilm.demon.co.uk
Oasis b.v.	Theo Zwart, Louis Rijkse	Lekstraat 4, 3433 ZB Nieuwegein, Holland Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: info@oasis.nl or rijkse@oasis.nl
Object Oriented Ltd	Walter G. Fil	Am Grendel 2, CH-6004 Luzern, Switzerland. Tel: 41 41 418 70 70 Fax: 41 41 418 70 77 Email: info@object-oriented.com
Optima Systems Ltd	Paul Grosvenor	115 Brighton Road, Purley, Surrey CR8 4HE UK Tel: 0181-763 2490 Fax: 0181-763 2491
RadSys Technologies AB	Randolph Schrab	Lovsångarv. 18, S-756 52 Uppsala, Sweden. Tel: +46 18 32 41 53 Fax: +46 708 1996 11 Email: 100564.2544@compuserve.com
Renaissance Data Systems	Ed Shaw	PO Box 421, Georgetown, CT 06982, USA. Tel: +1 (203) 270-9729
RE Time Tracker Oy	Richard Eller	Mikonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@retime.fi
The Rochester Group Inc.	Robert Miller	500 Park Avenue, Rochester, NY 14607-2926, USA. Tel: +1 (716) 271-1110. Fax: +1 (716) 271-1230
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00100-Roma Trullo, Italy Email: marsac@vnet.ibm.com
RusAPL	Boris Makeev	Box 971 (for Makeev), Dmitrovskoe Sh., 2, 127434, Moscow, Russia Tel/fax: +7 95 210-7763 Email: makeev@atom.al.x-atom.net
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapl@oc.aol.com
Shepp & Associates LLC	Andrew Shepp	1312 Washington Avenue, 6th Floor St. Louis MO 63103, USA Tel: +1 (314) 621-3272 Fax: +1 (314) 621-4267 UK Address: Clardge House, 29 Barnes High St, London SW13 9LW Tel: 0181 8788668 Fax: 0181 8768860 Email: ashepp@compuserve.com
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0654 Fax: +1 (416) 203-8999 Email: bernecky@eeeg.toronto.edu
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Solliton Associates	Laurie Howard	Solliton Associates Ltd, Groot Blankenberg 53, 1082 AC Amsterdam, Netherlands Tel: +31 20 646 4475 Fax: +31 20 644 1206 Email: sales@solliton.com
SovAPL	Alexander Skomorokhov	PO Box 5061, Oblinsk-5, Kaluga Region, Russia Tel: +7(08439)31463 Email: askom2@kaluga.rosmail.com
Strand Software Inc	Anne Faust	19235 Covington Court, Shorewood MN 55331 USA Tel: +1 (612) 470-7345 Email: amfaust@aol.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rhswain@acm.org
SwedAPL	Christer Uthleim	Novator Consulting Group AB, Svärdvägen 11C, S-182 33 Danderyd Sweden Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CService: 100341,404
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: si@ifi.unizh.ch
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250

Toronto SIG	Richard Procter	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada Email: info@torontoapl.org
Stephen Wynn		8 Clarence Gardens, Brighton, Sussex BN1 2EG Tel: 01273-327238 Email: centre@mistral.co.uk
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

WORLD WIDE WEB SITES

ACM SigAPL	www.acm.org/sigapl/
Adaytum Software	www.adaytum.com/
AFAPL	www.ensmp.fr/~scheret/langlet/ (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.demon.co.uk/apl385/
The APL Group Inc	www.aplgroup.com/
AUSCAN	www.interlog.com/~rjp/auscan/
Capital PC User Group	http://cpcug.org/
Causeway	www.causeway.co.uk/
CODEWORK	www.inrete.it/cdwk/eng/homee.html
COSY (Bob Armstrong)	www.cosy.com/
Dinosoft Oy	http://yritys.kolumbus.fi/dinosoft/
Dyadic Systems Ltd	www.dyadic.com/
FinnAPL	www.pyr.fi/apl/
Godin London Inc	www.godin.com/
Hoekstra Systems	www.khamsin.demon.co.uk/about/hst/hoframe.html
IBM APL2	www.torolab.ibm.com/ap/apl/apl2.html
Infostroy	www.insight.dk/infostroy/
Insight Systems ApS	www.insight.dk/
Iverson Software Inc	www.jssoftware.com/
Japan APL Association	www.naska.co.jp/JAPLA/
Lescasse Consulting	www.lescasse.com/
Lingo Allegro USA Inc	www.lingo.com/
Mackay Kinloch Ltd	ourworld.compuserve.com/homepages/Alastair_Kinloch/
MicroAPL Ltd	www.microapl.co.uk/
Oasis b.v.	www.oasis.nl/
RE Time Tracker Oy	www.rett.fi/
The Rochester Group Inc.	www.rochgrp.com/
Shepp & Associates	www.digitravel.com/
SigAPL	www.acm.org/sigapl/
Strand Software Inc.	www.jssoftware.com/
Rex Swain	www.pcnet.com/~rhswain/
Toronto SIG	www.torontoapl.org/
Jim Weigang	www.chilton.com/~jimw/

FTP SITES

IBM APL2	ps.boulder.ibm.com/ps/products/apl2/
Toronto toolkit	see Toronto SIG home page
Waterloo Archive	archive.uwaterloo.ca/ftparch/languages/apl
APL-to-ASCII	archive.uwaterloo.ca/languages/apl/workspaces/aplascli

Dyalog APL/W 8.2: First Impressions

by Adrian Smith (causeway@csi.com)

Introduction

This review has much the same status as the "Dyalog APL for Windows 95" article in Vector 12.2 (October 1995). It is a surface skim over the new features which have not been fully explored elsewhere, and is based on the first beta release of the interpreter from the Dyalog DSS web site. Of course, bugs and major irritations will get fixed before it goes live, but the basic design and feature-list is now fixed.

Language Improvements

APL2 convergence continues with ...

N-wise reduction

Designed originally for moving averages (the replicate, rotate and add method being slow and heavy on workspace):

```
vv←?365p12
p12÷/vv
354
```

... but also very useful for bitmap manipulation. If you want to do edge-detection or smoothing on a bit-pattern (or run a fast 'Life' program):

```
mm
0 0 1 1 0 0
0 0 1 1 0 0
1 1 1 1 1 1
1 1 1 1 1 1
0 0 1 1 0 0
0 0 1 1 0 0
(3÷3+0÷(0,mm,0)÷0)-mm
0 2 3 3 2 0
2 5 6 6 5 2
3 6 7 7 6 3
3 6 7 7 6 3
2 5 6 6 5 2
0 2 3 3 2 0
```

... neighbour-counting just got *really* easy, and a lot faster!

Scalar Functions with Axis

Really, this should have been in APL from the beginning! Row-wise addition of vectors to matrices was just about bearable, but anything else is a real pain, and again soaks workspace unnecessarily. Now we can say:

```
mm+3 4p12
mm+[1] 20 30 40
21 22 23 24
35 36 37 38
49 50 51 52
```

Hard to see how anyone could object to either of these (rather long overdue) extensions.

New Control Structures

:Hold is well covered elsewhere in the articles on threading, so just a brief mention for *:with* which is a nice alternative to $\square CS$, and a lot safer:

```
:with obj
  instancevar++1
:endwith
```

... is so much neater than the current alternatives of:

```
obj* 'instancevar++1'
```

or

```
here+ $\square CS$  obj * instancevar++1 *  $\square CS$  here
```

... which always leaves me feeling nervous if it is spread over several lines, as I may accidentally reuse *here* and lose track of where I came from. I expect that as we encounter more and more OLE servers with object trees yards deep, we will get used to *:with excel1.application.sheet5.range* and our APL code will look more like VB by the day. *C'est la vie*.

Environmental Stuff (mostly session changes)

Session Coolbars

Hmmmm. It took me about 3 sec to hate these and 10 sec to get into the WS Explorer and $\square EX$ the lot of them. Fortunately 8.2 happily loads your existing $\square SE$ so I am back on a black screen with nobbut a menu bar. However *de gustibus non disputandum* as they say in Basingstoke.

Permanent Session Log

Something the mainframe APLs had from the beginning, and APL2/PC showed the way with back in 1990. Very useful, particularly given the increased buffer size mentioned below.

Bigger Input Buffer

This one I really like, for a slightly oddball reason. For years, I have run with a version of *WsLoc* which echoes lines to the session, decorated fore and aft with *v* characters ...

```

[se.WsLoc '38'
vonKeyPress[12]      :CaseList 38 40 a Up/Dn should work weeklyv
vonKeyPress[13]      dte++7x~1*ks=38v

```

This makes for a very efficient global replace operation (with manual verification) as you can simply whip down the session changing any required lines and then hit <Enter> to process the lot. The downside was that it was easy to fill the input buffer, in which case Dyalog just barked and rejected the whole stack. Now I can make the buffer so huge that I will never lose work this way again. Brilliant.

Strong Interrupts (System Tray Icon)

Interested to know why they did it this way! This will get more useful with multithreading, as the cheap way to implement *:hold* is simply to diamond up critical sections into single long lines. It then becomes more necessary to be able to break execution mid-line.

Existing Gui Stuff

The usual pot-pourri of minor updates to existing objects:

- 24-bit bitmaps now report a CBits array which is the encoded RGB value for each pixel.
- graphical objects tell you where the mouse hit them. Also you can use ('Drawmode' 6) on polys and rects to make DIY locators. Essential for stuff like the draggy line-charts in RainPro.
- bitmaps can make uncompressed GIFs and compressed PNGs
- listviews have checkboxes, optional gridlines and (at last) you can select a row by clicking anywhere, not just on the left-hand column, in report view.
- smooth progress bars are back in the frame!

The implementation of checkboxes looks somewhat half-baked – it is not at all clear how you set them and the recommended method for reading back the value:

```
STATE+2 □NQ'Form.ListView' 'GetItemState' 11
13⇒φ(32p2)↑STATE
```

1

... looks like a pretty serious loop if you have a few hundred items on your list. I think a 'checked' property might be in order here!

New Gui Objects

Just as the language is converging slowly in APL2, the Gui is gradually evolving towards the true Microsoft look-and-feel. However the usual caveats apply – use these new toys because you want to, not because you can!

Calendar

Probably the simplest of the new controls to describe, and likely to turn up as a 'drop-down' on many people's date fields very quickly.

```

vCal
[1]  a Simple Calendar
[2]  'ff'□WC'Form' 'Calendar'(400 100)(200 200)('coord' 'pixel')
[3]  :With 'ff'
[4]  □WS'event' 40 '#.onFocus'
[5]  'cal'□WC'calendar'('event' 22 '#.onKeyPress')
[6]  :End
v

```



Note the need for the fully-qualified callbacks if you use `:with` here – I spent a few moments wondering why I was getting `VALUE ERROR` error as soon as I clicked on the form. Of course it was looking for the callback functions in the namespace of the form!

The only serious snag with this one is the total lack of any built-in keyboard behaviour. As a well-known muriphobe, I was relieved to find that it did respond to keystroke events, so here is a first-cut handler for it:

```

vmsg+onKeyPress msg;this;ks;ctrl;dte;ts
[1]  a Do some intelligent things with the Calendar keyboard
[2]  a Extend to taste!
[3]  (this ks ctrl)+msg[1 5 6]
[4]  :With this
[5]    dte+1>[]WG'SelDate'
[6]    ts+2 []NQ'' 263 dte
[7]    :Select ks
[8]      :Case 84  a t=today
[9]        dte+2 []NQ'' 264 []TS
[10]     :CaseList 37 39 a Left/right just roll the days
[11]       dte++-1*ks=37
[12]     :CaseList 38 40 a Up/Dn should work weekly
[13]       dte++7*-1*ks=38
[14]     :CaseList 33 34 a PgUp/Dn
[15]       :Select ctrl
[16]         :Case 0 a ... roll months
[17]           ts[1 2]+1+0 12*(-1*ks=33)+0 12+12+ts-1
[18]         :Case 2 a ... or years if Ctrl
[19]           ts[1]+-1*ks=33
[20]       :End
[21]     dte+2 []NQ'' 264 ts
[22]   :EndSelect
[23]   []WS'SelDate'dte
[24] :End
v

```

You can also queue a 'GotFocus' on it, hence the handler on the parent form:

```

vmsg+onFocus msg
[1]  a Set focus to calendar
[2]  []NQ(=>msg),'.cal')40
v

```

... which makes quite a nice little mini-application out of it. If you click on the month, you get a month menu - click on the year and it magically becomes a spinner! Probably it does other party tricks too - neither of the above were documented anywhere. The calendar runs from 1600, and the last valid date is day 23242571, which is Dec 31st 65535.

The Splitter Object

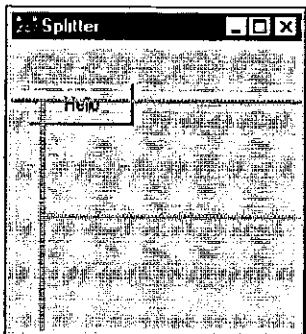
This one leaves me feeling very relieved. Causeway has been faking splitters for some time, and I do this in a very non-Microsoft way by having the splitters at the same level as everything else on the form, and setting a property of the list of other objects the splitter should manage. This has the great advantage that you can add splitters retrospectively to existing designs very easily.

The Dyalog splitter works very much in this way, and provides 'start' and 'end' events so that you can totally ignore the built-in behaviour and use it in almost any way you please. The only thing to watch is that the order of creation is crucial:

```

vSplit
[1]  a Simple Splitter
[2]  'ff'[]WC'Form' 'Splitter'(400 100)(200 200)('coord' 'pixel')
[3]  'ff.s1'[]WC'Splitter'('Posn' 40 0)
      ('Style' 'Horz')('bcol'(3/128))
[4]  'ff.s2'[]WC'Splitter'('Posn' 0 20)('bcol'(3/128))
[5]  'ff.s3'[]WC'Splitter'('Style' 'Horz')('bcol'(3/128))
[6]  'ff.b'[]WC'Button' 'Hello'(30 12)(28 72)
v

```



Normally splitters are invisible, but the colour shows how they organise themselves in a series of smaller and smaller panes, working to the right and downwards. You can set the width of the line, and one element of the position, but a splitter never intersects a previous splitter.

As the example shows, splitters completely ignore objects which are not mentioned in any of their 'splitob' properties. However they always resize each other to prevent crossovers.

Tab Controls and Tab Buttons

Referring back to Vector 12.2, I noticed that I prematurely celebrated the demise of the old TabBar and TabBtn when the property sheet arrived on the scene. Unfortunately, property sheets and property pages are far too specific in what they do, so we have mostly made do with the hand-crafted Dyalog tab-bars until now. The problem with this is that they do not look 'quite right' and that you are very limited in what you can do (e.g. adding and removing tabs) without creating ugly redraw problems.

The new tabs hook straight through to Windows95 common controls, so should look authentic and generally behave better when rebuilt on the fly.

```

vTabs
[1]  a Top-aligned tabbar
[2]  'ff'[]WC'Form' 'Simple Tabs'(400 100)(100 200)('coord' 'pixel')
[3]  'ff.tc'[]WC'TabControl'
[4]

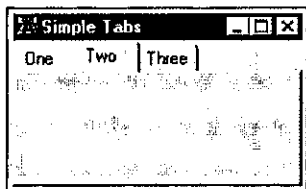
```

```

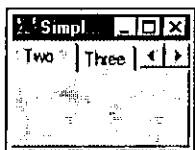
[5] 'ff.tc.t1' WC'TabButton' 'One'
[6] 'ff.tc.t2' WC'TabButton' 'Two'
[7] 'ff.tc.t3' WC'TabButton' 'Three'
[8]
[9] 'ff.tc.s1' WC'SubForm'('TabObj' 'ff.tc.t1')
[10] 'ff.tc.s2' WC'SubForm'('TabObj' 'ff.tc.t2')
[11] 'ff.tc.s3' WC'SubForm'('TabObj' 'ff.tc.t3')
[12]
[13] * Get the second tab to the front ...
[14]   NQ'ff.tc.t2' 30

```

v



My only concern here is the lack of events reported by the tab buttons - only Create and Select. There are applications out there using right-mouse menus on tabs, and fielding drag-drop events to move items between pages of notebooks.



Otherwise these seem to work quite nicely, and the scroller you get when you make the form too small is very much less visually obtrusive than its predecessor:

There are plenty more party tricks here (like adding icons to the tabs), but for the moment I think the important thing is that the basics work well, and that the new-style tabs should be a very painless upgrade for most applications.

Tool Control and Tool Button

Note that both the tool control and 'coolbar' controls need a version of comctl32.dll which is more recent than the one shipped with Windows95 OSR2. This has implications for runtime installation, which you should consider very carefully before using them.

Again, these are a direct Windows-95 pass-through and so they also have the authentic Office-97 appearance and behaviour. Causeway has been faking these since Toronto (cunning use of mouse-enter/mouse-leave and much swopping of bmpix) and it will be a great relief to kick the code well into the long grass, and use a native Dyalog control here. It should build very much faster and be a touch more reliable.

```

vTools cap
[1] * Simple toolbar (cap is 0 or 1)
[2] 'ff' WC'Form' 'Simple Toolbar'(400 100)(100 300)('coord' 'pixel')
[3] 'ff.tb' WC'ToolControl'('Showcaptions'cap)('Style' 'buttons')
[4]

```

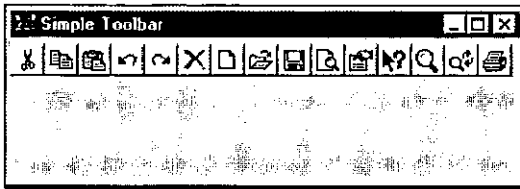
```

[5]  'ff.tb.il'[]WC'ImageList'('Masked' 0)('Mapcols' 1)
[6]  'ff.tb.il'[]WC'Bitmap'('Comctl32' 120)
[7]  'ff.tb'[]WS'ImageList' 'ff.tb.il'
[8]
[9]  'ff.tb.b1'[]WC'ToolButton' 'Cut'('ImageIndex' 1)
[10] 'ff.tb.b2'[]WC'ToolButton' 'Copy'('ImageIndex' 2)
[11] 'ff.tb.b3'[]WC'ToolButton' 'Paste'('ImageIndex' 3)
[12] 'ff.tb.b4'[]WC'ToolButton' 'Undo'('ImageIndex' 4)
[13] 'ff.tb.b5'[]WC'ToolButton' 'Redo'('ImageIndex' 5)
[14] 'ff.tb.b6'[]WC'ToolButton' 'Delete'('ImageIndex' 6)
[15] 'ff.tb.b7'[]WC'ToolButton' 'New'('ImageIndex' 7)
[16] 'ff.tb.b8'[]WC'ToolButton' 'Open'('ImageIndex' 8)
[17] 'ff.tb.b9'[]WC'ToolButton' 'Save'('ImageIndex' 9)
[18] 'ff.tb.b10'[]WC'ToolButton' 'Preview'('ImageIndex' 10)
[19] 'ff.tb.b11'[]WC'ToolButton' 'Point'('ImageIndex' 11)
[20] 'ff.tb.b12'[]WC'ToolButton' 'Help'('ImageIndex' 12)
[21] 'ff.tb.b13'[]WC'ToolButton' 'Find'('ImageIndex' 13)
[22] 'ff.tb.b14'[]WC'ToolButton' 'Again'('ImageIndex' 14)
[23] 'ff.tb.b15'[]WC'ToolButton' 'Print'('ImageIndex' 15)

```

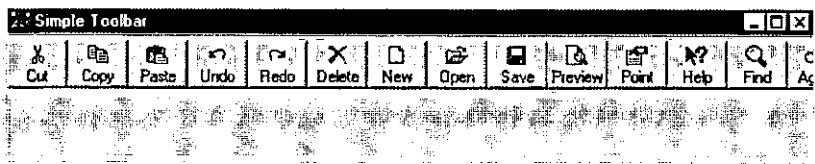
v

Somewhere in the Windows documentation, there must be a complete list of these built-in icons! Here is what this set looks like:



One of the nice touches is that you can turn captions on/off on the fly, which makes for a very easy 'user preferences' dialogue:

```
'ff.tb'[]WS 'Showcaptions' 1
```

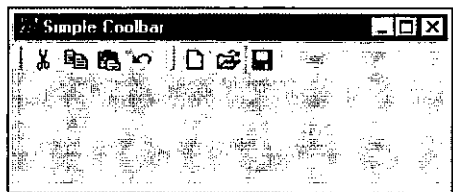


To space the buttons into logical groups you add a button with style 'separator', which I never would have guessed - thanks to Dyadic for pointing out this undocumented facility!.

Again, these do several other party tricks, for example you can do see-through buttons like Explorer (but not the effect of grey shades which light up on mouse-over), and of course the default is the Office-97 style which are flat until they sense the mouse.

Coolbars and Coolbands

These are by far the least convincing of the new controls:



As you can see, it has bad redraw problems (the mouse was over the 'file open' tool which has lost its lower edge, and the grippers cut an unpleasant notch in the upper groove), and the behaviour is unlike any Office tool I have seen, except for Internet Explorer.

```

vCool cap
[1]  a Simple coolbar (cap is 0 or 1)
[2]  'ff'[]WC'Form' 'Simple Coolbar'(400 100)(100 300)('coord' 'pixel')
[3]  'ff.il'[]WC'ImageList'('Masked' 0)('MapCols' 1)
[4]  'ff.il'[]WC'Bitmap'('ComCtl32' 120)
[5]
[6]  'ff.cb'[]WC'CoolBar' o 'ff.cb.c1'[]WC'CoolBand'
[7]
[8]  :With 'ff.cb.c1'
[9]    'tb'[]WC'ToolControl'('ImageList' '#.ff.il')('Showcaptions'cap)
[10]    'tb.b1'[]WC'ToolButton' 'New'('ImageIndex' 7)
[11]    'tb.b2'[]WC'ToolButton' 'Open'('ImageIndex' 8)
[12]    'tb.b3'[]WC'ToolButton' 'Save'('ImageIndex' 9)
[13]  :EndWith
[14]
[15]  'ff.cb.c2'[]WC'CoolBand'
[16]
[17]  :With 'ff.cb.c2'
[18]    'tb'[]WC'ToolControl'('ImageList' '#.ff.il')('Showcaptions'cap)
[19]    'tb.b1'[]WC'ToolButton' 'Cut'('ImageIndex' 1)
[20]    'tb.b2'[]WC'ToolButton' 'Copy'('ImageIndex' 2)
[21]    'tb.b3'[]WC'ToolButton' 'Paste'('ImageIndex' 3)
[22]    'tb.b4'[]WC'ToolButton' 'Undo'('ImageIndex' 4)
[23]    'tb.b5'[]WC'ToolButton' 'Redo'('ImageIndex' 5)
[24]  :EndWith

```

v

You will also see that I have arranged the bars to hide the 'Redo' tool, but there is no visual indication that it is missing. Word-97 shows a tiny '>>' symbol if this happens. My feeling here is that if we can't have proper docking toolbars, which can detach into floating palettes, then better not to bother at all. These guys are just mightily confusing for the user and are best left in the box.

Dyadic rightly point out that the above may well be true, but it is hardly their problem if Microsoft produce a set of 'standard' tools and then proceed to ignore them with the entire Office suite, Visual Studio, etc. If we want to do true 'Office' look-alikes we will have to wait a little longer, which is probably just what Redmond intended.

Other Stuff

TCP 'SendPicture'

Interested to know what this is for! I have been shoving GIFs and PNGs down raw sockets, apparently successfully; clearly there must be some circumstance when the simple-minded approach fails.

SQAPL Version 3

Lots of small improvements, some of which will remove installation problems (the hardwired registry key for SQALnit was a nuisance) and some of which will mean big speedups. Support for array update is very welcome.

Summary

Obviously, the main thrust of work in version 8.2 was towards OCX support, with a substantial investment in multithreading. There must be lots of Dyalog users around who wouldn't know an OCX if it came up and bit them, and who are rightly running scared of multiple concurrent)SI stacks. Hopefully they will have gleaned enough from these notes to judge whether an upgrade to 8.2 is worth the trouble and expense.

My feeling is that it is a bit like Windows-98 - it rolls up a lot of minor improvements which have been filtering out of DSS, consolidates the documentation, and adds a couple of bonus goodies (like the Calendar) to make you feel good about it. It is also remarkably stable - I have not crashed it once while writing these notes, in spite of pushing the Gui around trying to find redraw problems and other minor irritations.

I think it is worth it for the language improvements alone, and I welcome the chance to take a bunch of code out of the Causeway splitters, picture buttons and tabbed dialogues. Forms should load faster, and look a touch more professional, which all helps to keep APL in the frame as a viable Gui development tool.

Zark Newsletter Extracts

introduced by Jon Sandles

This section continues our series of extracts from the Zark APL tutor news. This time we have a classic challenge for which there are many solutions. Please give it a bash and send in your entries!

LIMBERING UP CONTEST

Converting Between Character Vectors and Matrices

(The purpose of this column is to work some flab off your APL midsection. Like muscles, your APL skills can atrophy if not exercised with adequate frequency and variety. This column presents a task for you to perform. Set aside a few minutes from your busy schedule and work the task. Mail in your solution and stay tuned for the results.)

One of the most common types of data encountered in data processing application is a set of names. For example, you might have a set of employee name, or customer names, or product names.

In APL, there are three common methods for storing a set of names:

1. Character matrix. The matrix has one row per name. The name is left-justified, i.e. padded to the right with enough blanks to exactly fit within the width of the matrix. For example:

```
CM←⎕
  Nails
  Screws
  Bolts
  Washers
```

2. Character vector. The vector is the catenation of the names, separated from one another with some strange delimiter character (strange enough to not be found within a name.) Often the delimiter character is also included at the beginning of the vector so the delimiter character can be immediately identified. For example:

```
CV←' /Nails/Screws/Bolts/Washers'
```

3. Character nest. For implementations of APL that support nested arrays (the notable exception being APL*PLUS PC), each name can be "enclosed" or "boxed" into a single item of a nested vector. For example:

```
CN←'Nails' 'Screws' 'Bolts' 'Washers' (APL2, APL*PLUS, →)
```

```
CN←'Nails'∘'Screws'∘'Bolts'∘'Washers' (Sharp APL)
```

Each form has its advantages:

1. The character matrix form is the most readable and is easy to work with when generating reports. It catenates directly to corresponding matrices of formatted values.
2. The character vector form is the most compact (least storage) and the most conducive to transmission. For example, if the delimiter is the "newline" (carriage return) character, you can send the vector directly to a printer, where each name will print on a separate line.
3. The character nest form is the easiest to manipulate. For example, to replace the 3rd 6th and 9th names by the 2nd name:

```
CN[3 6 9]←CN[2]
```

Your task is to write a function that will convert its argument from one of these forms to another. In particular, two functions are sought:

```
CV←DELIM MAT↔VEC CM
```

```
CM←WIDTH VEC↔MAT CV
```

MAT↔VEC converts its character matrix right argument to a delimited character vector result. The left argument of *MAT↔VEC* is the character (e.g. '/') to be used as the delimiter before each name. Assume the names in the matrix are left-justified. Be sure to delete the trailing blanks from each name in the process of converting it to a delimited character vector. For example:

```
'/Nails/Screws/Bolts/Washers'←'/'MAT↔VEC
```

Nails
Screws
Bolts
Washers

VECMAT converts its delimited character vector right argument to a left-justified character matrix, with as many columns as specified in the left argument (e.g. 25), truncating names that are too long. If the left argument is empty (e.g. 10 or ''), the matrix result has as many columns as the longest name. For example:

Nails
Screws
Bolts
Washers

```
+9 VECMAT '/Nails/Screws/Bolts/Washers'
```

In addition to matrix-to-vector and vector-to-matrix conversions, there are four other possible conversions among these forms:

- | | |
|-------------------|-------------------|
| 1. matrix to nest | 3. nest to matrix |
| 2. vector to nest | 4. nest to vector |

The *MATAVEC* and *VECMAT* functions should be as efficient as possible and as small as possible.

The two functions will be judged independently. 400 points will be awarded for the fastest function; 100 points for the smallest. Functions that are not the fastest or the smallest will be awarded points on a pro-rata basis. For example, if your function is 4 times slower than the fastest, you will received 100 points ($400 \div 4$). The highest score possible is 500, but only if the same function is both fastest and smallest.

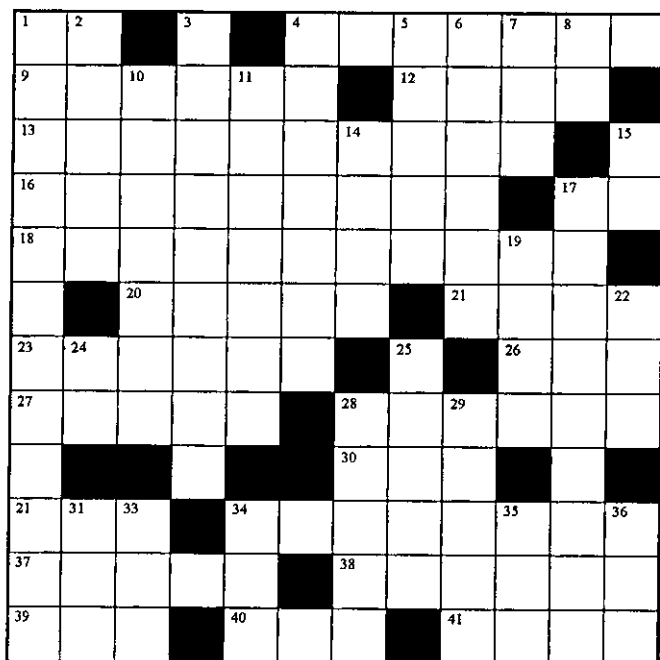
The highest-scoring function will be listed in the next issue of this newsletter. Also listed will be the names and scores of the top five authors.

Reprinted with kind permission from Zark APL Tutor News, a quarterly publication of Zark Incorporated, 23 Ketchbrook Lane, Ellington, CT06029, USA

Solution to Crossword from Vector 15.1

1 2	2 ρ	3 ρ	4 ,	5 A		6 $\times$		7 S	8 ϕ	9 N	10 R
11 B	+	Q	)	ϕ	ρ	B	12)	ρ	V	V	V
13 $\vee$	/	X	$\wedge$	Y		14 -	S	E	L	\	P
15 ~	M		/		16 $\rightarrow$	K	A	$\times$	$\wr$	$\times$	L
2 2		17 $\times$	M	18 	V		19 V	l	D	N	I
20 B	21 V		22)	R	E	S	E	T		24 E	C
	25 $\ominus$	26 B	$\neq$	M		R		27 $\div$	28 [	/	A
29 ~	M	$\vee$	N		30 ∇	$\times$	V	E	C	-	T
N		.		32 A	∇	N	S		33 C	P	I
34 V	R	$\neq$	35 1	$\downarrow$	V	R		36 O		37 D	O
$\in$		38 M	$\in$	L		39 ρ	P	/	40 P		N
41 Z	T	N	V		42 W	S	F	U	L	L	

Crossword



Across

1. Reverse the elements in the vector L
4. Delete from the vector N the even numbers 2 to 100
9. Scale up the numbers in the matrix M by 100000, and return as a vector
12. Fifty-five fives
13. The sum of the absolute value of the vector NV summed with the magnitudes of the first R elements of V
16. The N by N identity matrix (1s on the diagonal)
17. Get outta here!
18. Flag the rows of the 15-column character matrix CMI that match the first fifteen elements of CV
20. A random positive integer no more than 20 bigger than A
21. The amount to subtract from T to make it the next higher integer
23. Given N , a vector of integers, map all values from 1 to 7 to the numbers 0 to 6
26. Find the first four in F fast

27. What's currently localized?
28. Transform the matrix of scores S from students by exams, to exams by students and cap the scores at 100
30. $(N+1) - 1N$, without using ϕ
31. Squeeze all 0s from the vector V
34. $(\rho M) \rho (1 + \rho M) / V$ for the matrix M and the vector V
37. E (bits) if the vector N is empty, otherwise $(\rho E) \rho 1$
38. Divide each row of NM by the vector E , where $S \leftarrow \rho NM$
39. Repeat each column of M , C times
40. $(\rho M) \rho (0)$, for character M
41. $3\phi 2 / 2\rho 'ICE\ CREAM'$

Down

1. $2 - (\rho vec) < VEC \setminus N[\phi \setminus C]$ in origin 1
2. Convert the bit vector LE , having length M , to the designated indices
3. Flag those values of A that are contained in the matrix NT and for which the corresponding values in the non-negative integer array N are nonzero
4. How many 27s are there in each row of the matrix NM ?
5. 1 2 3 4 5, five times
6. From the first N values in T , flag those that aren't multiples of five
7. The indices of the vector V
8. 1 1 1 1 0
10. Generate NI (a multiple of 5) random numbers between 0 and 4, having the same frequency of each value
11. $(V[1] + 1L), (V[2] + 1L), (V[3] + 1L), \dots$
14. Which elements in R are 10s?
15. $+ / 14$
17. Branch to the line labelled V if I is a scalar
19. $-F1 - C$
22. $\&\square \leftarrow 'T', 0\rho T \leftarrow 40$
24. Change the sign of S
25. A random index into the vector S , up to the index at which the shape of M is found
28. Transpose the matrix NM along the MINOR diagonal
29. The number of whole multiples of C in NM
32. None
33. $(\rho M) \rho 0$, for numeric M
34. Transform NM from shape (6 3 4) to shape (4 3 6)
35. Unfinished business
36. If MAT were demoted

OpenGL Workspace & Russian Student Fund

Under the inspiration and guidance of Alexander Skomorokhov, Alexei Zalivine, a Russian Information Processing student in Obninsk, has written about 150 functions to access the OpenGL API functions under Microsoft Windows using Dyalog APL/W version 8 (32 bit). Included in the workspace are a number of utility functions to facilitate the use of the APIs as well as an interactive tool to experiment with the graphic image. For a more detailed description, please see the April, 1998 issue of *Vector*, pages 128-142.

To make it possible for Alexei and some of his colleagues to attend APL99, a Russian Student Fund has been established. For \$99 you will receive a copy of the software and the *Vector* article. \$50 will be contributed to the Fund and the remainder will compensate Alexei for his work (capitalism, you know) and cover the distribution costs. For \$150, the book, *OpenGL Superbible*, by Wright and Sweet, will be included as well. Additional contributions to the Fund are, of course, most welcome.

Please send a check (preferred in US) or money order in \$US made out to Renaissance Data Systems, or credit card information to:
Renaissance Data Systems; P.O. Box 421; Georgetown, CT 06829

Additional information may be posted to our web site, www.aplbooks.cnchost.com. Additional information on OpenGL may be found at www.opengl.org.

J-ottings 18: Transposition

by Norman Thomson

A recent question from Bill McLean about transpose in J provided a natural cue for some notes on what can be at first sight one of J's more daunting corners. The variety of possible options and constraints can make transpose seem quite formidable on first approach — read on to discover that this is not really so!

To begin with, the *purpose* of transposition, in both J and in APL is to *restructure* rectangularly oriented data by either reordering or merging axes. In J, the merge/reorder distinction corresponds to that between boxed and non-boxed left arguments. The reorder (non-boxed) case is straightforward, since the rank of the result does not change, only the order of the axes. The result is therefore a "pure" restructure, that is all the original items appear somewhere in the result, and no items are deselected. Permissible left arguments of |: are *permutations* of m items from $1..n$ where n is the rank of a , that is $\#a$, and $m \in 1..n+1$, that is m is one of the integers $0, 1, \dots, n$. The case $m=0$ is monadic transpose which is equivalent to transposition with the reverse permutation $|.1..n$. The left argument identifies a vector of indices in $\$a$. The rule is to remove these items from $\$a$ and concatenate them onto the end of $\$a$. This is different from the APL rule where the left argument specifies the *destination indices* of the old axes, which means that in general the P-transpose of A in APL, and the P-transpose of A in J where P is a permutation and A is an array, are different. More specifically, the P-transpose in APL is the *inverse* P-transpose in J, so that $(/:P):A$ is equivalent to $P\$A$ and $P|:A$ is equivalent to $(\#P)\$A$.

Suppose for definiteness that a is a four-dimensional array with $\$a = 2\ 3\ 4\ 5$, and think both of the shape of the transposition, and of the image of the item $a[i;j;k;l]$ in a mapping of a onto itself. Here are a few cases of the above rule in J:

left argument	new $\$a$	image of $a[i;j;k;l]$
0 1	4 5 2 3	$a[k;l;1;j]$
0 2 3	3 2 4 5	$a[j;l;k;l]$
3 1 0	4 5 3 2	$a[k;l;j;i]$
2 3 1 0	4 5 3 2	$a[k;l;j;l]$

Each entry in the second column is a permutation of $\$a$. There are 64 possible non-empty left arguments (4 with a single integer, 12 with two integers, and 24

each with three and four integers). Some of the corresponding transpositions must overlap since there are only 24 distinct permutations of \$a. Consider the arguments in the last two rows in the table above. According to the construction rule, they are necessarily identical since pushing three named indices to the right hand end is equivalent to placing the remaining index first. Thus if $n=\#a$ then both the set of all permutations of $i..n$ and also the set of all possible permutations of $n-1$ items from $i..n$ account for all possible restructurings by reordering axes. The opportunities afforded by arguments of length one or two can therefore only introduce more redundancy, which in turn increases the apparent complexity of transpose. Redundancy is further increased by applying monadic transpose at different levels of rank using the rank conjunction. Specifically for a rank 4 object, the following monadic transpositions and dyadic transpositions are equivalent:

$: "1$	$: "2$	$: "3$	$: "4$
0 1 2 3& :	0 1 3 2& :	0 3 2 1& :	3 2 1 0& :

Boxed arguments allow the possibility of restructuring by merging axes. At this point the ways of J and APL diverge even further. In APL, the fact that the left argument of transpose may have repeated indices blurs the distinction between reordering and merging. In J, this difference is sparkingly clear — the argument is either boxed for merge or not-boxed for reordering!

If the argument is boxed, the indices contained in the box are merged, and the corresponding axis-length in the box is the minimum of these axis-lengths. Since this results in general in the non-selection of some items, the overall effect is that of reducing the overall bulk of data. In the shape vector of the result array r , the unmerged axes all appear *before* the merged one. In terms of mappings, transpose effects a many-to-one mapping, for which the image of each point in the result array is given in the final column below. The coordinate names in this column match the values in column two, and can be used to establish systematically from source the value of any item in the result.

argument	new \$a	condition on index for an item of a to be selected	image of $a[i;j;k;l]$
<0 1	4 5 2	$i=j$	$r[k;l;i]$
<0 2	3 5 2	$i=k$	$r[j;l;i]$
<1 2	2 5 3	$j=k$	$r[i;l;j]$
<0 1 3	4 2	$i=j=1$	$r[k;i]$

Two properties of the above table deserve emphasis:

1. unlike the non-boxed case, the order of items in the boxed argument is immaterial, that is the contents of a box is a *combination* of m items from n rather than a permutation;
2. the third column echoes the first column with i, j, k, l replacing $0,1,2,3$, and $=s$ inserted, and the fourth column echoes the second column as explained above.

Also merging axes in the present sense must not be confused with the restructuring associated with e.g. $[1\ 2]a$ in APL where *all* the data is retained but rank is reduced.

There is a parallel between the fourth column above and the SQL command

```
SELECT ... WHERE ...
```

It may help to think of boxed arguments in these terms, although be aware that it is *indices* which are selected and not *values* as would be the case in SQL, so that transpose should *not* be seen as a vehicle for modelling selection from relational databases.

Consider another specific example, say $(<0\ 2) \vdash a$. There are two steps in evaluating this. The first and most straightforward step is to identify the new $\$a$. $(<0\ 2)$ means that it is axes 0 and 2 which are to be merged. The length of the merged axis-length is $2 < 4$, that is, 2. The remaining axis lengths are 3 5, and so $\$a$ is 3 5 2, which means that the result is a list of three lists, each of which is a list of length five, each item within which is a list of two items. Using the SQL analogy $(<0\ 2) \vdash a$ can be read as

```
SELECT a[i;j;k;l] WHERE i=k
```

So much for structure. The second step is to identify which data occupies each cell of the result. This is where the third column comes in. The constraint $i=k$ means that the only items which are to be *selected* are those whose indices are of the form $0\ x\ 0\ y$ and $1\ x\ 1\ y$ where x is chosen from 1..3 and y is chosen *independently* from 1..5. Specifically the indices of the items in the first of the three lists are:

0 0 0 0	1 0 1 0
0 0 0 1	1 0 1 1
0 0 0 2	1 0 1 2
0 0 0 3	1 0 1 3
0 0 0 4	1 0 1 4

and the remaining two are obtained by replacing the 0s in the second column in turn with 1s and 2s. If the result required were a list of five lists, each of which is a list of length three, so that the pattern of indices for the first list is

0 0 0 0	1 0 1 0
0 1 0 0	1 1 1 0
0 2 0 0	1 2 1 0

a further transposition must be applied, viz. `1 0 2 | : (<0 2) | : a .`

Geometrically speaking, the result of a merge transpose is a *diagonal* cross-section of rectangular data whose rank is determined by the number of axes merged. If the length of the boxed left argument is *n* then the rank of the result is less than the rank of the original by *n*-1. Various physical mappings of the non-selected items (that is those whose indices do not appear in the condition in column three in the table above) onto the selected items are possible. The non-selected items may be thought of as being "swept" towards the appropriate diagonal cross-section along either an axial direction or a generalised plane defined by axial directions. For example in the rank two case defined by `<0 1` as left argument, there are two possible *physical* mappings depending on whether items in rows or items in columns are perceived as being swept *horizontally* onto the diagonal.

The reasoning behind one of the most frequently occurring practical applications

```
diag=.(<0 1)&|:      NB. Leading diagonal of matrix
```

should now be readily apparent, as should

```
ndiag=.diag@:({."1)      NB. Non-leading diagonal of matrix
```

(both the parentheses and the colon are necessary in the definition of `ndiag`.) and

```
sym=.-:|:      NB. Returns 1 if argument is symmetrical
(sym 1.3 3),sym 3 3$1
```

```
0 1
```

Transpose can be used as a transformation when either an operational verb is most readily coded for a different arrangement of data axes, or reuse of an existing verb is easier than writing a new one to accommodate a different axis ordering. This principle can be simply illustrated using a rank three object `d=.1.2 3 4` thought of as a structure of planes, rows and columns.

`1 2+d` adds 1 to the first plane and 2 to the second. Corresponding additions to rows are achieved by applying the rank conjunction, `1 2 3 + "2 d`, and to

columns by 1 2 3 4+"1 d. The rank conjunction can be circumvented by promoting say columns to become the major dimension using transpose, adding, and then using the *inverse* transposition to restore the original shape.

```
1 2 0 |: 1 2 3 4 + 2 0 1 |: d
```

An inverse transposition is obtained by using the corresponding inverse permutation. Inverse permutations are obtained directly by grade up. In the above example 1 2 0 and 2 0 1 are inverse permutations, that is:

```
1 2 0 -: /: 2 0 1
```

so that ...

```
a -: 1 2 0 |: 2 0 1 |: a
```

for *any* rank three object a.

The single item permutation cases are sometimes useful. For a rank three object, 1|: exchanges rows and columns within planes. For an object of any rank 0|: promotes the second dimension to become the major dimension, and (n-1)|: swaps the two lowest order dimensions.

Describing the whole range of possibilities may have made this discussion seem complex. There is nevertheless a compelling simplicity in the fundamental underlying rules, and an appropriate conclusion is therefore a summary in a few lines of all there is to know about transpose, viz.:

axes are:	effect	left	contents of	shape vector rule
-----	-----	argument	of left arg.	-----
reordered	unchanged	not boxed	permutation	named axes to tail
merged	reduced	boxed	combination	merged axis to tail

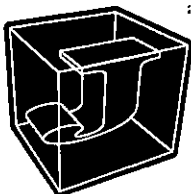
Notes:

1. Reordering only: the full potential is available by using only full permutations — lower order permutations are valid, but give no essentially new transformations.
2. Reordering *and* merging axes requires two applications of |: .



IT'S SURPRISING
HOW JSOFTWARE
MAKES YOUR DATA
JUMP OUT!

J for Windows 95/NT from Iverson Software Inc. provides everything you need to solve problems quickly. You can build complete stand alone Windows applications, or easily integrate **J** with other programs. For example, use **J** as an ActiveX control for a web page, an OLE server for Excel,



J in a Box

or a 32-bit DLL called from a C program. **J** is also available on a variety of other platforms including Windows CE, Mac, Linux, Solaris and AIX. The language is identical in all versions, and programs not making use of platform-specific features will work unchanged on all systems.

Strand Software 19235 Covington Court Shorewood, MN 55331
Tel: (612) 470-7345 Fax: (612) 470-9202
www.jssoftware.com

Eric Iverson at the BAA

reported by Adrian Smith

Introduction

The BAA would like to thank Eric for taking the trouble to break his journey back from APL98 to talk to us in London. In spite of the best efforts of *Computing* magazine, which mistook J for Java and triggered several phone calls from intrigued Java hacks, it was not a well-attended meeting, with only 30 or so able to make the trip.

Those who came enjoyed a splendid talk, with some entertainment – J running on a palmtop under Windows CE – and some solid education in the inner workings of virtual-memory – disguised as a demo of J and memory-mapped files.

The subject of memory-mapped files is an important one for the future of both J and APL, and will be treated in much more depth in the next *Vector*. For now, I will just focus on what Eric showed us on the day.

How J is Marketed and Sold

Eric showed how ISI's marketing strategy is now aimed at the Maths, Physics, Chemistry fraternity, with J being promoted as:

1. an array language (of course!)
2. a maths language, in direct competition with MATLAB etc. They get asked questions like "How does it do derivatives?".
3. a "permissive" language (unlike Java) where people can solve problems in their own way, and 10 programmers are likely to return with 30 different answers when given a problem to solve.
4. a "functional" programming language that's only as pure as you want it to be.
5. a real "object oriented" language, but you don't have to restrict your ideas to this.
6. a very high-performance language (vs MATHEMATICA etc.) when operating on large amounts of data.
7. a possible component in a hybrid application.

ISI are working on selling \$6,000-\$10,000 site agreements for "all the J they can eat" as the basis for their expansion. The CD release has all the web-site content and all back versions of J (an installation only takes 60Mb) which is also a really useful distributed archive for ISI!

J on the Palmtop (under Windows CE)

This is a complete (except for OpenGL which is not available on CE) implementation of J "in your hand". It has met with an overwhelming response at maths and science conferences, where the portability is very attractive. You can program on the train - particularly handy if you need to hide the fact that you are writing programs!

One useful bonus for ISI is that "Pocket Excel" has no graphics capability, so J can be sold quite effectively as a simple graphing add-in for spreadsheets.

Eric was extremely enthusiastic about the machine - all the 16Mb of RAM is for you, as Windows is in ROM - and also about the CE operating system which he noted is extremely stable, very tightly coded and generally feels better engineered than anything else that has recently come out of Microsoft. Personally speaking, I would not be at all surprised to see it sweep the home market over the next few years - the benefits of a 1-sec boot-up time and the impossibility of damaging it will surely give it a massive advantage over NT when Windows 98 comes to the end of the road.

Lapack - for Linear Algebra

We used to think APL was the best at this game, but one consequence of visiting the mathematics circuit was that Eric became aware that there are lots of very competent (and specialised) packages out there which embody some very much better code than we can produce. If you want eigenvalues of a 100×100 matrix on your palm-top, you need *Lapack*, directly interfaced with J.

Memory-mapped Files

This is one of the most significant things which has happened to J in the business and finance community. The surprising thing - which was noticed by A+ and K a long time ago - is how simple it is. As Eric pointed out, it is not new stuff! All the hooks were there in NT 3.5, so why have C++ and VB been so long in catching on to the possibilities?

The important thing to realise is that under NT and Windows95, all memory is actually mapped to disk all the time.

When a process starts it has a 4GB address space. It can address bytes from 0 up to 2^{32} , each process has its own address space and initially all addresses are invalid. An address space byte is ONLY given a value by mapping it to a byte in a file – every byte value in an address space comes from a byte in a file.

Physical memory

On-chip cache memory, off-chip cache memory, and real memory are all simply performance mechanisms to connect an address space byte with its mapped byte in a file. *Real memory is just another level of cache!* It has little to do with the underlying architecture of mapping address space bytes to file bytes.

Mapping a file – 3 steps

1. Create a file object (open a file).
2. Create a mapping object from the file object that describes which bytes in the file are to be used (offset and length) and how they can be accessed (readwrite, readonly, etc.).
3. Create a view from the mapping object. The view connects the bytes in the file described by the mapping object with an address in the address space.

J memory mapped files

The J programmer has complete control over mapping files into the J address space. The J programmer can create a J array header that describes the memory of a mapped file and access the file data directly as a J noun.

Open and load mmf utilities.

```
open 'user\mmf.ijs'
load 'user\mmf.ijs'
load 'files'
```

Set a variable that will be used as the name of a file:

```
fn=: 'c:\t.mmf'
(50$'abcdef') fwrite fn
50
```

Steps to map file bytes to a memory address:

1. `openfile` calls `Win32 Createfile` to create the file object

```
a=: fn;GENERIC_READWRITE;0;OPEN_EXISTING
[ fh=: openfile a
8
```

2. `mapfile` calls `CreateFileMapping` to create a mapping object. By default all bytes in the file are mapped.

```
[ mh=: mapfile fh;PAGE_READWRITE
9
```

3. `viewfile` calls `MapViewOfFile` to map the file bytes represented by the mapping handle into the address space. The result is the address of the file bytes in the address space.

```
[ fad=: viewfile mh;FILE_MAP_WRITE
_2100400128
memr fad,0,50
abcdefabcdefabcdefabcdefa...
```

Release resources:

```
UnmapViewOfFile fad
_2122891228
CloseHandle mh
1
CloseHandle fh
```

Mapping the file to a J noun

`xmap` maps a file, creates a J array header that describes it, and creates a symbol table entry to address the header. The left arg is the name of the noun. The right arg is the file, readonly flag, trailing shape, and type.

```
'abc' xmap fn;0;5;JCHAR
```

abc_base_	c:\t.mmf	_2100400128	26736252	8	9
-----------	----------	-------------	----------	---	---

```
$abc
10 5
abc
abcde
fabcd
efabc
defab
cdefa
```

```

bcdef
abcde
fabcd
efabc
defab

    abc=: 'zxcvb' 1} abc
    abc
abcde
zxcvb
efabc
defab
cdefa
bcdef
abcde
fabcd
efabc
defab

    unmap <'abc'
0
    abc
|value error: abc
|[-1]
    fread fn
abcdezxcvbefabcdefabcdefabcdef...
```

Eric pointed out that this implements 'persistent data' remarkably easily. It also gives you access to really huge datasets which you could never hope to handle in the workspace. He showed an example with a CD of 650Mb of trading data from the New York stock exchange. With these mapped as J nouns (... pause for the CD ROM to get up to speed) you now apparently have 7 x 100Mb numeric arrays to manipulate:

```

NB. map 7 files (600MB) to 7 nouns
nysemapall''
$ctidx
199602 22
$ctbin
11332740 22
*/$ctidx
4391244
*/$ctbin
249320280
```

Of course you need to be careful what you do, but being able to run a grid control on 250Mb of data sure beats Excell!

So - what does it do for you? Well it obviously moves the boundary of the direct array-processing language much further out. You can map parts of files to different nouns, maybe in different processes, which is just like shared variables, except that this really is the *same* array.

The key difference from the Dyalog $\square XT$ implementation is in the separation of the array data - on the CD - from the array header - in the workspace. The data behaves as a real variable, but with certain restrictions, for example assignment is only possible 'in place', although catenation may be possible.

Summary

Seeing this from the point of view of an APL programmer, I would say that the most important lesson was in the need to break out of the workspace and have limited, but convenient, access to very high volumes of data without having to run with enormous workspace sizes.

J already has a clear advantage here, in that it uses the standard Windows mechanisms to allocate and release memory. If APL can hook into some of these ideas on access to external data, it will greatly streamline many existing and future applications.

The potential of the CE machines was also brought home to many of the audience for the first time. Before long we will have access to this kind of computing power on something not much bigger than the £4.95 calculator you can buy at Dixons. The very cryptic nature of APL and J (remember APL was developed on 10cps typewriter terminals) is an important selling point when you are trying to do maths on a small keyboard with limited display area! Watch this space!

[Editor's note: some details in the implementation of the supplied scripts for memory-mapped files are slightly different in the production release, which is now available.]

BAA AGM and Vendor Forum, May 22nd at the Royal Statistical Society

reviewed by Jon Sandles

The BAA AGM passed without incident and we moved swiftly on to the main event which was Peter Donnelly of Dyadic Systems talking to us about the latest features in Dyalog APL. He warned us that the main new features of what will become version 8.2 would not be discussed as he was saving them up for the Rome conference. Instead he was going to bring us up to date with some of the more minor things that Dyadic have been up to in the last year.

So the content of Peter's talk was nothing ground-breaking. He covered the new feature to put Excel-like comments into the Dyadic grid. The file system functions have been revamped to give shy results, this allows them to be strung together to form meaningful expressions. All the functions are fully backward compatible. We saw again the power and beauty of dynamic functions, and a nice example was presented showing a rename dynamic function which used the new shy results of the file system functions.

What was ground-breaking about Peter's talk was the method of delivery. Peter did very little of the talking himself, instead he had programmed his computer to do the presentation for us. What we saw on his screen was a wizard who guided us through the new features and a genie that asked appropriate questions. At one point an APL+WIN user (called Robbie) appeared on the screen and appeared to be amazed by some of the new features. He was then seen rummaging in a toolbox to see if he had any of these features. He appeared disappointed when he could not find them.

Peter was using a new set of tools provided by Microsoft called the Microsoft Agent. (Some of you may have also seen Eric Baelen using this COM object at APL98.) There was more to this than just showing off. The Agent was being driven by Dyalog APL and was showing just how far Dyadic's COM (used to be called OLE Automation) support has come. Others may be interested in how this was done so I looked into it further. First you need to get your hands on the Microsoft stuff. This can be downloaded from the Internet.

Go to <http://www.microsoft.com/workshop/prog/agent>. You need to download quite a lot of files from here to get it all working. First you need to download the core components. You will then need to download the character

data (for example the wizard is called Merlin), and if you want them to speak through your soundcard you must download the Lernout & Hauspie TruVoice text-to-speech engine. There are other more advanced bits and pieces you can use to create your own characters and you can even teach them to recognise speech commands through a PC microphone.

Once you have downloaded and installed the appropriate bits and pieces you are ready to write some APL to make it work. Firstly you must initialise the COM object.

```

␣ Initialises Agent.Server and loads the character
SIZE←256 256 ␣ Size of each character window

␣ Connect to to Microsoft Agent OLE Server

'AGENT'⌵WC'OLEClient' 'Agent.Server.1'

␣ Not only does this object export its events through a specific
␣ event sink named IAgentNotifySink, but it requires that you call its
␣ Register method with this client's event sink ID that is returned by

MSINK←AGENT.Register(2 ⌵NQ'AGENT' 'OLEAddEventSink' 'IAgentNotifySink')0

␣ Load the MERLIN character and set its window size
␣ (uses the "with sound effects" version of MERLIN)
ID_MERLIN←AGENT.Load('c:\agent\MERLINSFX.ACS')-1 0
'MERLIN'⌵WC AGENT.GetCharacter ID_MERLIN''
MERLIN.SetSize SIZE

```

This should be all you need to get you going! Try the following to check it's working...

```
MERLIN.Show 0 0
```

You should see the wizard appear on your screen...



To make him disappear again...

```
MERLIN.Hide 0 0
```

To make him move to the right hand side of the screen, and say something...

```
MERLIN.Show 0 0  
MERLIN.MoveTo 500 400 1000 0  
MERLIN.Speak'Hi Guys' '' 0
```

You should see a speech bubble appear above his head with the text "Hi Guys" and if you installed the text to speech converter you will hear him speak as well. Each character has a number of in-built animations. These can be invoked using the Play method. For example,

```
MERLIN.Play'Wave' 0
```

You should remember to shutdown the control correctly when you are finished with it...

```
AGENT.Unregister MSINK  
[]ex + 'AGENT' 'MERLIN'
```

This should be plenty of information to get you all going. Back to Peter's presentation.

Dyadic Systems (Peter Donnelly)

The new Dyadic Support Service (DSS) was introduced, which is an extended support service that you can sign up to if you use Dyalog APL. It's a bit like joining Microsoft's MSDN, as it gives you access to all the latest updates and extra development utilities online via the Internet. People may object to paying more for support, but if it's done well it is worth every penny.

As a current user of Dyadic's DSS service, I can assure you it's done very well. In fact Dyadic think they are doing it so well that they have plans to make the tools they are using to manage the process commercially available. In fact writing this review was made much easier by DSS. The workspace (MSAGENT.dws) that contained the scripts to drive Peter's, talk was available in the COM Enhancements section of the DSS service. All I needed to do to work out how he did it was to go to the Dyadic website, go to the DSS section and type in my DSS account and password, and then download the relevant workspace. The code you see above (and comments) is lifted almost directly from this workspace. So, if you want to see the talk in full, just download the workspace and play it back. (You still need to get hold of all the Microsoft Agent downloads as well.) Peter uses nearly all the features of the Microsoft Agent, so it is well worth looking at this workspace in detail if you are really interested.

Causeway (Adrian Smith)

Peter's talk was followed by Adrian Smith of Causeway Graphical Systems who presented some of the latest features he is implementing in his rain graphics package. Adrian demonstrated a feature he calls 'Active' Rain where the chart on the screen is able to provide feedback to the user depending on where the mouse is positioned or where the user double-clicks.

This can be used to provide "drill-down" on pie and bar charts. Developers will have full control of what callbacks can be attached, and hot-spots are embedded into the PostScript that Adrian uses to produce the charts, but with special tags so that they are ignored by printers, but interpreted by his chart engine.

The way this is done enforces a strict separation between the chart engine and the Gui objects on which the charts have been rendered. This is important, as the engine can render charts on to almost any Dyalog object (subforms, buttons, metafiles) but must not attempt to 'steal' the event handlers which the application may have defined for its own use. Instead, it provides a set of utilities which let the existing event handlers perform 'hit tests' on the object to see if there is anything interesting at a particular (x,y) location.

A simple example would be to call *PostScrp.FindSeries* from a mouse-down handler:

```
xy+φ2+2+msg  a Mouse position  
info+guiobj #.PostScrp.FindSeries xy
```

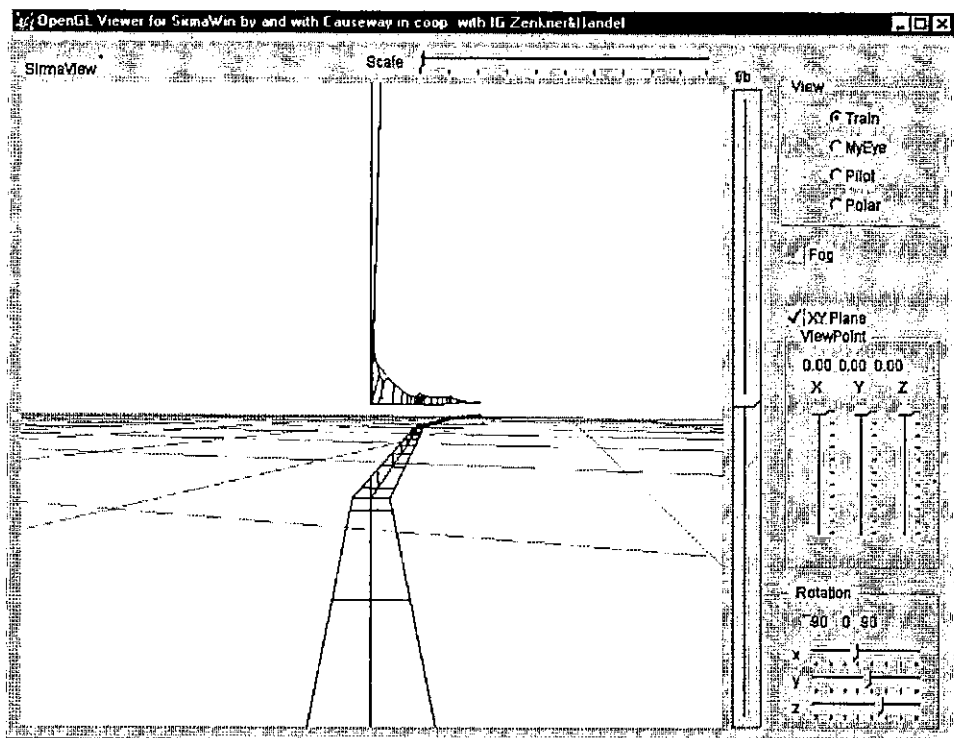
If this returns an empty array, there are no qualifying data points. Otherwise it returns a 4-element vector giving (chart name)(series name)(row)(column), where (row,column) are the indices into the original data series. Piecharts are treated as 1-column matrices, so report the segment in the row-number.

Similar utilities are provided to test for standard items such as the chart header and axis captions, or to report all the data points which fall inside a bounding rectangle. Adrian showed how these utilities can be combined to offer graphical drill-down, and he also was able to demonstrate a very recent feature of the Dyalog interpreter (the 'Drawmode' property of polylines) with a simple graphical editor which allowed the user to move the points of a timeseries plot vertically, while continuously updating a dashed line to show the connections to the previous and next points in the series.

This is a powerful extension to the existing Rain package and will enable users to start to use it not just as a reporting tool, but a genuine alternative front-end for querying and analysing their data.

For a much more convenient interface, *CausewayPro* users have the option of using either the 'Chart' or 'PostScript Viewer' classes which do the hit-test automatically and queue events back to the chart object, where they can be conveniently handled in the behaviour table. For example a 'CDATA' event is queued whenever the user clicks the mouse over a bar, pie-wedge or marker. The application might use this to eliminate points from a trend analysis, put up a dialogue box showing details about that data item, and so on.

Adrian also demonstrated some of the collaborative work Joachim Hoffmann has been doing to provide full OpenGL support from within the Causeway programming environment. Adrian showed a very impressive "virtual walkthrough" of the cabling design for a new section of electrified railway.



The simple case is a fixed camera in the cab, which just 'drives' up and down the track with the slider bar. Adrian also attempted to demo the 'free-flying' version, but soon found himself lost in space, unable even to locate the railway.

Bloomsbury Software (APL2000)

Unfortunately Bloomsbury Software had other commitments on the day of the meeting but they were able to provide some slides of the highlights of APL2000's new version of APL+WIN (v3)...

- TCP/IP support via [J]NI
- APL applications as an ODBC server
- APL as a COM server
- APL as a COM client
- Additional Mouse Events for Flat Toolbars
- Media Player
- Virtual Function Calls
- Utilities to print Trees and Lists with images

We also happened to have just received the new version in the post, so I was able to do a very simple presentation of the new COM features that allow other applications (like Excel or Word) to call the APL engine directly. An example of some Visual Basic to load a workspace would be...

```
Dim ws As Object  'Declare an Object Variable
Dim v As Variant  'Declare a Variant Variable

'Create APLW Workspace Engine

    Set ws = CreateObject("APLW.WSEngine")

'Load APL Workspace

    Call ws.SysCommand("load MyWs")
```

So all in all a fascinating meeting, the most disappointing feature of which was that not many people turned up! It is getting increasingly difficult to persuade people to put in time and effort to produce quality presentations when the audiences are so small. So next time, please try and come along, you will be surprised how much you learn and how entertaining the afternoon can be!

APL98 Workshop – Threads in Dyalog APL

by Dyadic Systems Ltd.

Editor's Note

This set of notes correctly illustrates the syntax and capabilities of the Dyalog interpreter as used in the Rome workshops. There has subsequently been one small change in the usage of the `⍷` functions:

`⍷TNUMS tnums` is now

`⍷TCNUMS tnums` – numbers of child threads (direct descendants only)

`⍷TNUMS` is now niladic, and returns the numbers of all running threads, regardless of parentage.

As this does not significantly affect the Rome examples, it was thought better to leave the notes unmodified.

Things to know about threads:

- Start a new thread: `⍺`
- My thread number: `⍷TID`
- The thread numbers of children: `⍷TNUMS tnums`
- Kill threads: `⍷TKILL tnums`
- Wait for threads to terminate: `⍷TSYNC tnums`
- Hold a semaphore token: `:Hold`
- Display all semaphore tokens: `)HOLDS`
- Extended State Indicator Display.
- Localisation issues.
- GUI considerations.

Example 1 – Grab: a game for any number of players.

```
)load grab
```

During development of this workspace, it will occasionally be necessary to stop run-away threads. Function key: F3 has been programmed with:

```
␣tkill 0      a Kill background threads.
```

So, if you get into trouble:

- **Ctrl+Pause** to interrupt foreground thread if there is one;
- **F3** to kill background threads;
- **↵)reset** for good measure.

Examine function *grab*:

```
)ed grab
▽ grab numb      a Grab: ...
[1]
[2] table+numbp0  a Prepare the table.
[3]
[4] player&"\numb a Play!
▽
```

```
grab[2]  Initialises the table to zeros.
grab[4]  Starts asynchronous players.
```

Copy mk1 player:

```
)copy grab mk1.player
)ed player
```

Mk1.player

```

v player me;adj;vic          a Play grab.
[1]
[2]   table[me]++1           a Buy into game.
[3]
[4]   adj+1 0 1/3+(me-2)*table a Adjacent friends.
[5]
[6]   :Repeat                a Endlessly ...
[7]
[8]       vic+adj[?padj]      a Choose a victim.
[9]
[10]      :If *table[vic]     a Worth the hit?
[11]          wait
[12]          table[vic]--1    a Take vic's candy.
[13]          wait
[14]          table[me]++1     a Keep it for self.
[15]      :End
[16]
[17]      show table           a State of play.
[18]
[19] :End
v

```

Function *show* displays the state of play in the session log.

```

v show table;tag;tab;sum      a Display table.
[1]   tag+4 0*?TID           a Thread id:,
[2]   tab+3 0*table          a Table values,
[3]   sum+?+/table           a Sum,
[4]   [+tag,' ':',tab,' ':',sum a ... output to ses
v

```

Now run the simulation:

```

grab 4
1: 1 0 0 0 : 1
1: 1 0 0 0 : 1
2: 1 1 0 0 : 2
1: 1 1 1 1 : 4
2: -1 0 1 1 : 1
3: -1 0 2 1 : 2
3: -1 0 2 1 : 2
4: -1 0 2 2 : 3
4: DOMAIN ERROR
player[10] :If *table[vic]    a Worth the hit?
^

```

Branching State Indicator

Dyalog's state indicator looks like a branching tree:

```
      )si
.    player[11]
&3
.    player[11]
&2
.    player[13]
&1
.    player[10]*
&4
```

State indicator with name list:

```
      )sinl
.    player[11]  vic      adj      me
&3
.    player[11]  vic      adj      me
&2
.    player[13]  vic      adj      me
&1
.    player[10]* me      adj      vic
&4
```

Reset state indicator:

```
      )reset
```

Holds - Mk2.player

```
      )copy grab mk2.player
      )ed player
```

```

v player me;adj;vic          A Play grab.
[1]
[2]   table[me]++1           A Buy into game.
[3]
[4]   adj+1 0 1/3+(me-2)phi;table A Adjacent friends.
[5]
[6]   :Repeat                A Endlessly ...
[7]
[8]       vic+adj[?padj]      A Choose a victim.
[9]
[10]      :Hold *vic          A Hold him,
[11]
[12]          :If *table[vic]  A Worth the hit?
[13]              wait
[14]              table[vic]--1 A Take vic's candy.
[15]              wait
[16]              table[me]++1 A Keep it for self.
[17]          :End
[18]
[19]      :EndHold            A Release victim.
[20]
[21]      show table          A Show state of play.
[22]
[23]  :End
v

```

grab 4

...

<F3>

□tkill 0

Edit player to comment out line[21]:

```
[21]  A player[21] show table      A Show state of play.
```

Now restart game:

grab 4

look at the holds:

```

)holds
)holds
)holds
)holds

```

Thread Wars

We can change the spirit of the game by altering the rules slightly.

The following is a 'Wild West Saloon' version:

```
)copy grab mk3.player
)ed player
```

```

v player me;vic;rand      A Play rough.
[1]
[2]   table[me]++1        A Buy into game.
[3]
[4]   rand+{w[?pw]}       A v Random choice.
[5]
[6]   :While 1<p[]TNUMS 0  A Still someone there.
[7]
[8]       vic+rand;table   A Choose random victim.
[9]
[10]      :Hold vvic        A Hold him,
[11]
[12]          :If *table[vic]  A Worth the hit?
[13]              wait
[14]              table[vic]--1 A Take vic's candy.
[15]              wait
[16]              table[me]++1 A Keep it for self.
[17]          :End
[18]
[19]      :EndHold           A Release victim.
[20]
[21]      []TKILL rand []TNUMS 0 A Shoot wildly.
[22]
[23]      show table         A Show state of play.
[24]
[25]      :End
[26]
[27]      'Thread',[]TID,'wins.' A Gloat.
v
```

```
grab 20
```

Watch the number of threads decreasing in the session status bar.

Example 2 - Fibonacci Numbers

```
)load fibs
```

```
)ed fib1
```

```

v n+fib1 n          a Fibonacci number
[1]  :If n>1
[2]    n+//fib1"n-1 2
[3]  :End
v

```

```

      fib1 8          a 8'th Fibonacci number.
21
      fib1"1:10       a First 10 Fibonacci numbers.
1 1 2 3 5 8 13 21 34 55

```

Fib2 traces the stages of evaluation:

```
)ed fib2
```

```

v n+fib2 n          a Fibonacci number
[1]  trace n
[2]  :If n>1
[3]    n+//fib2"n-1 2
[4]  :End
[5]  trace n
v

```

Trace shows the thread ID and stack depth:

```
)ed trace
```

```

v trace n;tag;dent          a Trace evaluation
[1]  tag+4+'&',w[]TID       a Tag with thread Id.
[2]  dent+,(^2+p[]SI)p<'    ' a Indentation.
[3]  []+tag,' ',dent,w n    a Display.
v

```

```

      fib2 5
&0 5
&0 - 4
&0 - . 3
&0 - . . 2
&0 - . . . 1
&0 - . . . 1
&0 - . . . 0
&0 - . . . 0
&0 - . . . 1
&0 - . . . 1
&0 - . . . 1
&0 - . . 2
&0 - . . 2
&0 - . . . 1
&0 - . . . 1
&0 - . . . 0
&0 - . . . 0
&0 - . . 1
&0 - . 3
&0 - . 3
&0 - . . 2
&0 - . . . 1
&0 - . . . 1
&0 - . . . 0
&0 - . . . 0
&0 - . . 1
&0 - . . 1
&0 - . . 1
&0 - . 2
&0 5
5

```

```

)ed fib3

```

```

v n+fib3 n      a Fibonacci number
[1] trace n
[2] :If n>1
[3]   n++/[]TSYNC fib3&"n-1 2
[4] :End
[5] trace n
v

```

Fib[3] spawns sub-threads and *TSYNC* synchronises the results.

```

      fib3: 5
&0  5
&2  .  3
&1  .  4
&4  .  .  1
&4  .  .  1
&5  .  .  3
&6  .  .  2
&3  .  .  2
&7  .  .  .  1
&8  .  .  .  0
&8  .  .  .  0
&9  .  .  .  2
&7  .  .  .  1
&6  .  .  1
&12 .  .  .  .  0
&12 .  .  .  .  0
&13 .  .  .  1
&13 .  .  .  1
&11 .  .  .  .  1
&11 .  .  .  .  1
&10 .  .  .  1
&10 .  .  .  1
&9  .  .  .  1
&5  .  .  2
&1  .  3
&14 .  .  .  0
&14 .  .  .  0
&3  .  .  1
&2  .  2
&0  5
5

```

We can generalise this technique with a 'parallel each' operator: *peach*.

```
)ed fib4
```

```

v n+fib4 n          a Fibonacci number
[1] trace n
[2] :If n>1
[3]   n+:/fib4 peach n-1 2
[4] :End
[5] trace n
v

```

)ed peach

```

v rslt+(func peach)argt
[1] rslt+[]TSYNC func&"argt
v

```

... or the *dynamic* operator equivalent:

```
peach+({[]TSYNC aa&"w})
```

Fib5 contains an error if *n* is 0:

```

v n+fib5 n          a Fibonacci number
[1] :If n>1
[2]   n++/fib5 peach n-1 2
[3] :Else
[4]   °
[5] :End
v

```

```

      fib5
4:SYNTAX ERROR
fib5[4] °
      ^
      )si
. . peach[1]
. . fib5[2]
. &3
. . fib5[4]*
. &4
. peach[1]
. fib5[2]
&1
. peach[1]
. fib5[2]
&2
peach[1]
fib5[2]

```

```

)sin1
. . peach[1]      rslt  func  argt
. . fib5[2] n      n
. &3
. . fib5[4]*      n      n
. &4
. peach[1]      rslt  func  argt
. fib5[2]      n      n
&1
. peach[1]      argt  func  rslt
. fib5[2]      n      n
&2
peach[1]      argt  func  rslt
fib5[2] n      n

```

Example 3 - GUI applications

```

)load gui
form 5 5

```

Click on several of the buttons.

Notice that each button starts counting only when the previous one has finished.

```

)ed form callback

```

```

▽ form rc;fm;□IO      A Counting buttons
[1] □IO←0
[2] 'fm'□WC'form'
[3] {
[4]     size←100÷rc
[5]     posn←w×size
[6]     name←'fm.btn',▽rclw
[7]     evnt←'Event' 'Select' 'count'
[8]     name □WC'button' '0'posn size evnt
[9] }''rc
[10] □DQ'fm'
▽

```

```

v count msg;i;obj      a counting callback.
[1]  obj+=msg
[2]  :For i :In 0,(1999),0
[3]  :If *NC obj o obj WS'Caption'(*i)
[4]  :Else
[5]  :Leave
[6]  :End
[7]  :End
v

```

We could make the buttons count in separate threads, by having callback function *count* spawn an asynchronous sub-function.

There is a shorthand way of doing this by appending & to the name of the callback function when setting the *Event*.

Change line *form[7]* to the following and try again:

```

[7]      evnt+'Event' 'Select' 'count&'

```

Now try:

```

form&5 5
)sinl

```

Threads – an Introduction to Multithreading

by John Scholes (john@dyadic.com)

Introduction

A *thread* is a strand of execution in the APL workspace. Prior to version 8.2, Dyalog APL supported only a single thread. Version 8.2 introduces multiple threads running in parallel.

A thread is created by calling a function *asynchronously*, using the new primitive operator 'spawn': &.

With a traditional APL *synchronous* function call, execution of the calling environment is paused, *pendent* on the return of the called function. With an *asynchronous* call, both calling environment and called function proceed to execute concurrently.

An asynchronous function call is said to start a new *thread* of execution. Each thread has a unique *thread number*, with which, for example, its presence can be monitored or its execution terminated.

Any thread can spawn any number of sub-threads, subject only to workspace availability. This implies a hierarchy in which a thread is said to be a *child thread* of its *parent thread*. The *base thread* at the root of this hierarchy has thread number 0.

With multi-threading, APL's stack or state indicator can be viewed as a branching tree in which the path from the base to each leaf is a thread.

When a parent thread terminates, any of its children which are still running, become the children of (are 'adopted' by) the parent's parent.

Thread numbers are allocated sequentially from 0 to 2147483647. At this point, the sequence 'wraps around' and numbers are allocated from 0 again avoiding any still in use. The sequence is re-initialised when the active workspace is cleared or a new workspace loaded. A workspace may not be saved with threads other than the base thread: 0, running.

Threads introduce new language elements:

- Primitive operator: *spawn*: 8.
- System functions: `□TID`, `□TNUMS`, `□TKILL`, `□TSYNC`.
- An extension to the GUI Event syntax to allow asynchronous call-backs.
- Control structure: `:Hold`.
- System command: `)HOLDS`.

Note that features marked '!' were not implemented in the version used in Rome for the APL98 workshops. There have also been some minor syntax changes since these notes were issued - see the help file which accompanies the recent Dyalog beta release.

Thread switching

Programming with threads requires care.

The interpreter may switch between running threads at the following points:

- Between any two lines of a defined (or dynamic) function or operator.
- While waiting for a `□DL` to complete. !
- While waiting for a `□FHOLD` to complete. !
- While awaiting input from:
 - `□DQ`
 - `□SR` !
 - `□ED` !
 - The session prompt or `□:` or `□`.
- While awaiting the completion of an external operation:
 - A call on an external (AP) function. !
 - A call on a `□NA` (DLL) function. !
 - A call on an OLE function. !

At any of these points, the interpreter might execute code in other threads. If such threads change the global environment; for example by changing the value of, or expunging a name; then the changes will appear to have happened while the thread in question passes through the switch point. It is the task of the application programmer to organise and contain such behaviour!

You can prevent threads from interacting in critical sections of code by using the `:Hold` control structure.

Name Scope

APL's name scope rules apply whether a function call is synchronous or asynchronous. For example when a defined function is called, names in the calling environment are visible, unless explicitly shadowed in the function header.

Just as with a synchronous call, a function called asynchronously has its own local environment, but can communicate with its parent and 'sibling' functions via local names in the parent.

This point is important. It means that siblings can run in parallel without danger of local name clashes. For example, a GUI application can accommodate multiple concurrent instances of its call-back functions.

However, with an asynchronous call, as the calling function continues to execute, both child *and* parent functions may modify values in the calling environment. Both functions see such changes immediately they occur.

If a parent function terminates while any of its children are still running, those children will thenceforward 'see' local names in the environment that called the parent function. In cases where a child function relies on its parent's environment (the setting of a local value of `⌈IO` for example), this would be undesirable, and the parent function would normally execute a `⌈TSYNC` in order to wait for its children to complete before itself exiting.

If, on the other hand, after launching an asynchronous child, the parent function calls a *new* function (either synchronously or asynchronously), names in the new function are beyond the purview of the original child. In other words, a function can only ever see its calling stack decrease in size - never increase. This is in order that the parent may call new defined functions without affecting the environment of its asynchronous children.

Debugging

If a thread sustains an untrapped error, its execution is *suspended* in the normal way. At the same time, all other thread activity in the workspace is *paused* until the suspension is cleared or restarted. A consequence of this is that, at any one time, only a single thread can be suspended.

The session is attached to the suspended thread, so that you can:

- Examine and modify local variables.

- Trace and edit functions.
- Clear suspensions.
- Restart execution.

When the last suspension in a thread is cleared or restarted, the session is re-attached to the base thread, and any paused threads are resumed. From the session in a suspended thread, you can spawn a new thread, but its execution is immediately paused until the parent's suspension is removed.

The error message from a thread other than the base is prefixed with its thread number:

```
260:DOMAIN ERROR
Div[2] rslt+num+div
^
```

State indicator displays: `)SI` and `)SINL` have been extended to show threads' tree-like calling structure.

```
      )SI
.    Calc[1]
&5
.    .    DivSub[1]
.    &7
.    .    DivSub[1]
.    &6
.    Div[2]*
&4
Sub[3]
Main[4]
```

Here, *Main* has called *Sub*, which has spawned threads 4 and 5 with functions: *Div* and *Calc*. *Div*, after spawning *DivSub* in each of threads 6 and 7, has been suspended at line [2].

Removing stack frames using 'Quit' from the tracer or '←' from the session affects only the current thread. When the final stack frame in a thread (other than the base thread) is removed, the thread is expunged. `)RESET` removes all but the base thread.

GUI Considerations

To launch a call-back function asynchronously, you can append the character: '&' to its name in the *Event* specification:

```
□WS'Event' 'Select' 'Press&'
```

Spawn Thread: $\{R\} \leftarrow \{X\} f \& Y$

$\&$ is a monadic operator with an ambivalent derived function. $\&$ spawns a new thread in which f is applied to its argument Y (monadic case) or between its arguments X and Y (dyadic case). The shy result of this application is the number of the newly created thread.

When function f terminates, its result (if any), the **thread result**, is returned. If the thread number is the subject of an active $\square TSYNC$, the thread result appears as the result of $\square TSYNC$. If no $\square TSYNC$ is in effect, the thread result is displayed in the session in the normal fashion.

Note that $\&$ can be used in conjunction with the **each** operator: " to launch many threads in parallel.

Examples:

```

0.25      ÷&4          a Reciprocal in background
          □÷÷&4       a Show thread number
1
0.25      FOO&88      a Spawn monadic function.

          2 FOO&3      a          dyadic

          {NIL}&0       a          niladic
          *&'NIL'      a          ..

          X.GOO&99     a thread in remote space.

          *&'□dl 2'    a Execute async expression.

          'NS'*&'FOO'  a .. remote .. .. ..

          PRT&''+□nl 9 a PRT spaces in parallel.

```

Current Thread Number: $R \leftarrow \square TID$

R is a simple integer scalar: the number of the current thread.

Examples:

```

     $\square TID$           a Base thread number
0
     $\&\square TID'$       a Thread number of async. 1.
1

```

Child Thread Numbers: $R \leftarrow \square TNUMS\ Y$

Y must be a simple array of integers representing thread numbers. R is a vector of the child threads of Y .

Examples:

```

     $\square TNUMS\ 0$ 
2 3
     $\square TNUMS\ 2\ 3$ 
4 5 6 7 8 9

```

Kill Threads: $\{R\} \leftarrow \{X\} \square TKILL\ Y$

Y must be a simple array of integers representing thread numbers to be terminated. X is a boolean single, defaulting to 1, which indicates that all descendant threads should also be terminated. The shy result: R is a vector of the numbers of all threads that have been terminated.

The base thread: 0 is always excluded from the cull.

Examples:

```

     $\square TKILL\ 0$           a Kill background threads.

     $\square TKILL\ \square TID$       a Kill self and descendants.

     $0\ \square TKILL\ \square TID$   a Kill self only.

     $\square TKILL\ \square TNUMS\ \square TID$  a Kill descendants.

```

Wait for threads to terminate: $\{R\} \leftarrow \square TSYNC Y$

Y must be a simple array of thread numbers.

If Y is a simple scalar, R is an array; the result (if any) of the thread.

If Y is a simple non-scalar, R has the same shape as Y , and result is an array of enclosed thread results.

The interpreter detects a potential deadlock if a number of threads wait for each other in a cyclic dependency. In this case, the thread that attempts to cause the deadlock issues error number 1008: DEADLOCK.

Examples:

```

dup+{w w}                                R Duplicate

□+dup&88                                  R Show thread number
11
88 88

□+□TSYNC dup&88                          R Wait for result
88 88

□TSYNC,dup&88
88 88

□TSYNC dup&1 2 3
1 2 3 1 2 3

□TSYNC dup&"1 2 3
1 1 2 2 3 3

□TSYNC □TID                              R Wait for self
DEADLOCK
□TSYNC □TID
^
*EN
1008

```

Hold: $:Hold Y$

$:Hold$ provides a mechanism to control thread entry into a critical section of code. Y must be a simple character vector or scalar, or a vector of character vectors. Y represents a set of 'tokens', all of which must be acquired before the

thread can continue into the control structure. `:Hold` is analogous to the component file system `□FHOLD`.

Within the whole active workspace, a token with a particular value may be held only once. If the hold succeeds, the current thread *acquires* the tokens and execution continues with the first phrase in the control structure. On exit from the structure, the tokens are released for use by other threads. If the hold fails, because one or more of the tokens is already in use:

1. If there is no `:Else` clause in the control structure, execution of the thread is blocked until the requested tokens become available.
2. Otherwise, acquisition of the tokens is abandoned and execution resumed immediately at the first phrase in the `:Else` clause.

`T` can be either a single token:

```
'a'
'Red'
'#.Util'
''
'Program Files'
```

... Or a number of tokens:

```
'red' 'green' 'blue'
'doe' 'a' 'deer'
,'abc'
+□nl 9
```

Pre-processing removes trailing blanks from each token before comparison, so that, for example, the following two statements are equivalent:

```
:Hold 'Red' 'Green'
```

```
:Hold +2 5p'Red Green'
```

Unlike `□FHOLD`, a thread does not release all existing tokens before attempting to acquire new ones. This enables the nesting of holds, which can be useful when multiple threads are concurrently updating parts of a complex data structure.

In the following example, a thread updates a critical structure in a child namespace, and then updates a structure in its parent space. The holds will allow all 'sibling' namespaces to update concurrently, but will constrain updates to the parent structure to be executed one at a time.

```

:Hold [cs]'          A Hold child space
...                 A Update child space
:Hold ##.[cs]'      A Hold parent space
...                 A Update Parent space
:EndHold
...
:EndHold

```

However, with the nesting of holds comes the possibility of a 'deadlock'. For example, consider the two threads:

Thread 1	Thread 2
:Hold 'red'	:Hold 'green'
...	...
:Hold 'green'	:Hold 'red'
...	...
:EndHold	:EndHold
:EndHold	:EndHold

In this case if both threads succeed in acquiring their first hold, they will both block waiting for the other to release its token. Fortunately, the interpreter detects such cases and issues an error (1008) DEADLOCK. You can avoid deadlock by ensuring that threads always attempt to acquire tokens in the same chronological order, and that threads never attempt to acquire tokens that they already own.

Note that token acquisition for any particular `:Hold` is atomic, that is, either *all* of the tokens or *none* of them are acquired. The following example *cannot* deadlock:

Thread 1	Thread 2
:Hold 'red'	
...	:Hold 'green' 'red'
:Hold 'green'	...
...	:EndHold
:EndHold	
:EndHold	

Examples:

`:Hold` could be used, for example, during the update of a complex data structure that might take several lines of code. In this case, an appropriate value for the token would be the name of the data structure variable itself, although this is just a programming convention: the interpreter does not associate the token value with the data variable.

```

:Hold 'Struct'
  ...           * Update Struct
  Struct ← ...
:EndHold

```

The next example guarantees exclusive use of the current namespace:

```

:Hold 'CS'      * Hold current space
  ...
:EndHold

```

The following example shows code that holds two positions in a vector while the contents are exchanged.

```

:Hold 'to fm
  :If >/vec[fm to]
    vec[fm to]←vec[to fm]
  :End
:End

```

Between obtaining the next available file tie number and using it:

```

:Hold 'FNUMS'
  tie←1+[/0,FNUMS
  fname FSTIE tie
:End

```

The above hold is not necessary if the code is combined into a single line:

```

fname FSTIE tie←1+[/0,FNUMS
or,
tie←fname FSTIE 0

```

Note that *:Hold*, like its component file system counterpart: *□FHOLD*, is a device to enable *co-operating* threads to synchronise their operation.

:Hold does not *prevent* threads from updating the same data structures concurrently, it prevents threads only from *:Hold*-ing the same tokens.

Display held tokens:**)HOLDS**

System command **)HOLDS** displays a list of tokens which have been acquired or requested by the **:Hold** control structure. Each line of the display is of the form:

```
token:  acq  req  req ...
```

Where *acq* is the number of *the* thread that has acquired the token, and *req* is the number of *a* thread which is requesting it. For a token to appear in the display, a thread (and only one thread) must have acquired it, whereas any number of threads can be requesting it.

Example:

Thread 300's attempt to acquire token '*blue*' results in a deadlock:

```
300:DEADLOCK
Sema4[1] :Hold 'blue'
      ^
      )HOLDS
blue:   100
green:  200      100
red:    300      200      100
```

Blue has been acquired by thread 100.

Green has been acquired by 200 and requested by 100.

Red has been acquired by 300 and requested by 200 and 100.

The following cycle of dependencies has caused the deadlock:

Thread 300 attempts to acquire <i>blue</i> ,	300 → <i>blue</i>
which is owned by 100,	↑ ↓
which is waiting for <i>red</i> ,	<i>red</i> ← 100
which is owned by 300.	

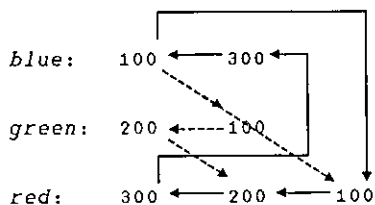
A useful technique for debugging complex deadlocks is to take the output from **)HOLDS**; and add the hold that caused the error as if the deadlock had not been detected, in this case, *blue* requested by thread 300:

```
blue:   100      300
green:  200      100
red:    300      200      100
```

Next, draw arrows:

- From each thread number in a request (second and subsequent) column, to the thread number on its left.
- From each thread number in the first (acquired) column, to each of its counterparts in any of the request columns.

The deadlock results from a cycle in this 'graph'.



HTML Basics for APLers (VI): Pictures and ImageMaps

by Adrian Smith (causeway@csi.com)

Introduction

This is the sixth article in our series on HTML, and introduces the basics of image generation and graphical 'drill-down' from pictures shown in the browser. The examples are coded in Dyalog APL 8.2 (pre-release) which should be generally available by the end of the year. The only new facility used is the ability to generate .GIF or .PNG files directly from bitmap objects in the workspace. If you want to try the techniques before you have this, you can easily fake the process by copying the bitmap to the clipboard, pasting it into PaintshopPro, saving the image as a GIF, and reading it back into the workspace as a native file.

Why Bother?

The internet currently has almost no active content. There are plenty of pointless animations and buttons that light up as you cross them with the mouse, but it is hard to find a web site that can actually take some input from you and produce a custom picture in response. With an APL server we could do something quite interesting, for example it would be fun to take the old Paul Berry "Starmap" workspace (circa 1979), allow the user to enter date, time, latitude and longitude, and display a night-sky picture on their browser. It would be even more fun if they could click on the major objects (like the moon) and be told some details about sky position, phase etc.

Another obvious example would be to offer graphical drill-down from a series of charts, perhaps starting with a summary Piechart, and drilling to detail by brand or by time period. This is probably an easier example to follow, and it has been running on Dyadic's web site since July, so this is the one I will work through in detail. As in the previous articles, threading issues are ignored to keep it slightly more simple.

Images in Web Pages

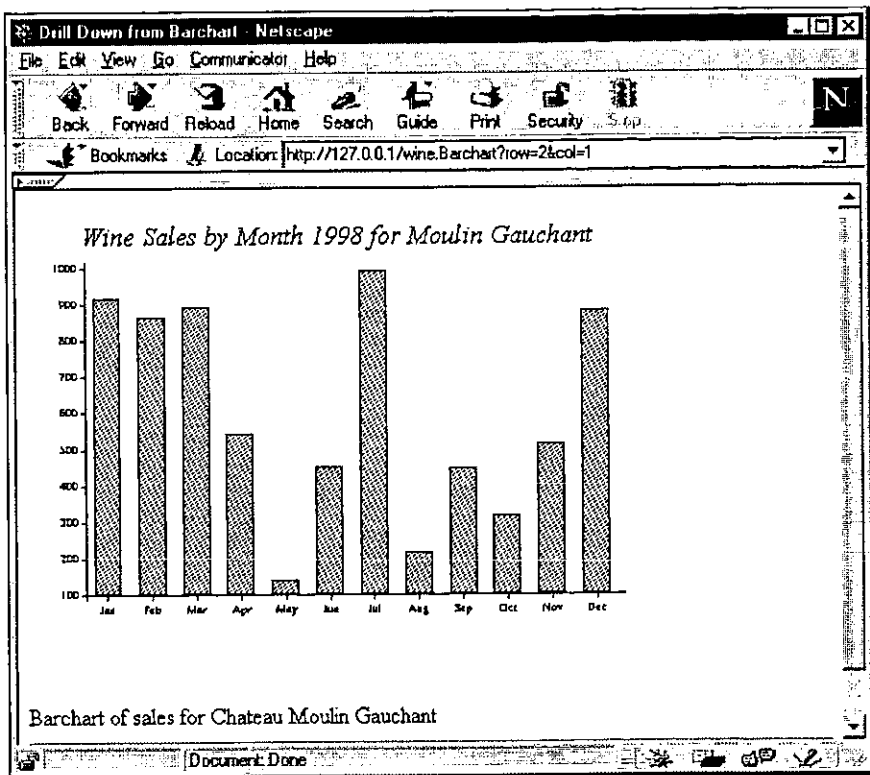
Because the internet was originally a text-based system, the mechanism for including pictures is quite strange and can be extremely inefficient in the way it

uses the server. If you want your photo on your home page, you probably know that you code the HTML as something like:

```
<p>Here is my picture</p>

<p> ... and here is my dog ...</p>
```

... and so on. What you may not realise is that each time you do this, your page generates a completely fresh HTTP connection to the server to get the image. If you embed 10 pictures, that makes 11 totally independent connections, each of which goes through the rigmarole of socket generation, receive, send and close. Of course the server is not just serving you, so your requests may get interleaved with other transactions. Also you may get bored and hit the <STOP> button long before all the pictures have been requested. Consider the implications for something which is machine-generated such as:



Both the HTML and the chart have been constructed on the fly, but we first send the HTML to the browser:

```
<HTML><HEAD>
<Title>Drill Down from Barchart</title>
</HEAD>

<body>

<p>Barchart of sales for Chateau Moulin Gauchant
<HR>
  a Causeway Graphical Systems Ltd 1998
</body></HTML>
```

... it may then return with a new request to get the chart. Clearly it makes sense to have a single function generate both the enclosing text and the chart (when we include an *imagemap* in the text it is really unavoidable), but we cannot be sure that the chart will be the next thing requested, or even if it will be requested at all. This problem explains much of the complexity in the functions which follow – they must generate the HTML for the page and hide the chart under a (uniquely named) stone. If a request comes in for the chart under that stone, it can be served up and discarded (the *Readcache* function in the example). If it has not been requested after a decent lapse of time, we can throw it away and reuse the stone for another chart. If we are really busy, we could run out of stones (or workspace), in which case we should throw away the oldest charts, risking the possibility of broken pictures appearing in our users' browsers.

After all that, the mechanics of making the page and mapping the image is very straightforward! I shall start with a very brief introduction to the *imagemap* syntax, and then work through the APL code I use to make mapped charts which can appear on the fly in generated web pages.

How Imagemaps Work

In the early days (pre Netscape 3.0 and IE3) you had to put the map on the server and use a CGI script to get it interpreted. This still works, and has the benefit of keeping the transmitted page much smaller, but it is a devil to debug and gives almost no feedback to the end user. Consequently this introduction talks only about 'client-side' maps which you can develop and test in the comfort of your own laptop.

The idea is very simple ...



Here is a completely pointless artwork (mapped.gif) which I want to use as a graphical menu. See the Thornton hatchment on www.demon.co.uk/apl385/gilling/stonegrv.htm for a more useful example. When the user waves his or her mouse over the 'hot' areas on the picture, the browser shows a 'pointing hand' cursor and indicates the destination page in the status line. Netscape-4 can even use the 'Alt' text to provide a pop-up tip. The HTML to achieve this is simply:

```
<HTML><HEAD>
<TITLE>Page Title</TITLE></HEAD>

<BODY bgcolor="#FFFFFF">

<map name="simple">
<area shape=circle coords="12,16,12"
href="top.htm" alt="Pretty Circle">

<area shape=rect coords="35,12,91,39"
href="detail1.htm" alt="Rectangle">

<area shape=poly coords="12,84,34,55,62,75,79,55,94,77"
href="detail2.htm" alt="Polygonal">
</map>

<H2>First Heading</H2>

Add body text here.
</BODY></HTML>
```

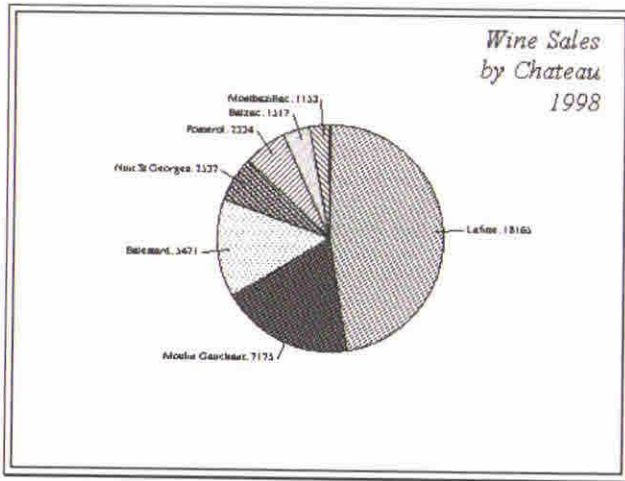
The map section(s) - one per mapped image - conventionally go at the beginning of the file, or may be written to a separate file entirely. The only shapes you can have are circles (give the centre and radius); rectangles (give top left and lower right corners) and polygons. All co-ordinates are (x,y) pairs, from the origin in the top left corner. Then you add the rest of your page, remembering to associate the map with the appropriate picture by naming it and referring to the name in the 'usemap' parameter. Note the name is given as "#simple" as it is a reference to a map in the same file. The general form is more like "mymap.map#one" to refer to a named map held in a separate file (N.B. this does not work in Netscape-3).

You can make your maps by hand quite easily (just read off the co-ordinates from PaintshopPro), but these days most web-authoring tools come with a competent map-editor to save you the trouble. Adobe PageMill is one of the better examples.

However what we want to do is to build the graphic and the map from data in an APL workspace, so knowledge of the above mechanics is essential.

Mapping a Piechart

I started out with the idea of writing a server application that would not just serve up canned charts, but that would be a genuine graphical browser of data that could change in real time. For example:



It would be very nice to click on a pie sector to get details of the monthly sales for a particular chateau, and so on. Clearly a pre-requisite for this is for the original chart to record the elements in the source data which correspond to each segment in the pie (or bar in a barchart, marker in a line-plot and so on). The way *RainPro* does this is to add some dummy functions to the intermediate PostScript code (*setseries* in the fragment shown below):

```
[(chart1297043) () 3 7 1] setseries % cht,series,type,shape
0 8 2 setttext
185 113 79.1 -80.5 90 4 2 0.5 wedge
[255.9 118.9 275.7 120.5] 0 1 0.6 pL # stroke
278.1 118.5 M # (Lafitte: 18165) 0 0 0 pshow

185 113 79.1 -147.9 -80.5 1 6 0.5 wedge
[155.8 48.1 147.7 30] 0 1 0.6 pL # stroke
146.1 28 M # (Moulin Gauchant: 7175) 1 0 0 pshow
```

...

In this example the (generated name) *chart1297043* has an unnamed series of type-3 (a piechart) with 7 rows and one column. When it is rendered on to the screen or any other Gui object - in this case a bitmap - each call to *wedge* adds an entry to a table in the Gui object's namespace to record where it was placed. With a bit of low cunning (needed to represent the pie sectors as reasonable polygons - see the appendix for the code) this can be reformatted as:

```
<MAP name="chart1297043">
<area shape="poly"
  coords="225,159,238,237,263,227,285,210,299,186,304,159,
          299,132,286,108,265,90,239,81,225,79"
  href="http://127.0.0.1/wine.Barchart?row=1&col=1">
<area shape="poly"
  coords="225,165,157,207,176,227,200,240,227,244,238,243"
  href="http://127.0.0.1/wine.Barchart?row=2&col=1">
<area shape="poly" coords="225,159,150,132,145,160,151,187,157,201"
  href="http://127.0.0.1/wine.Barchart?row=3&col=1">
<area shape="poly" coords="225,165,167,111,152,134,150,138"
  href="http://127.0.0.1/wine.Barchart?row=4&col=1">
<area shape="poly" coords="225,159,191,87,168,103,167,105"
  href="http://127.0.0.1/wine.Barchart?row=5&col=1">
<area shape="poly" coords="225,165,210,87,191,93"
  href="http://127.0.0.1/wine.Barchart?row=6&col=1">
<area shape="poly" coords="225,159,225,79,210,81"
  href="http://127.0.0.1/wine.Barchart?row=7&col=1">
</MAP>
```

Each pie sector now has a map area which will cause it to call our server, requesting the *Barchart* function, and passing the sector index in the http 'GET' request string. This can be returned to the calling function, along with the integer vector representing the bytes of the GIF or PNG file, and the id of the chart:

```
v{htm}+{sock}Piechart tokens;pc;map;id;gif;cbk
[1]  a Return piechart of annual totals as a GIF (or PNG)
[2]  a Ignores all parameters!
[3]  #.ch.Set'boxed,values'
[4]  #.ch.Set('hstyle' 'right')('head' 'Wine ... 1998')
[5]  #.ch.Set'xl'achat
[6]  #.ch.Pie+/Asales
[7]  pc+#.ch.Close
[8]  a Call back to the barchart for that chateau
[9]  cbk+'http://',home,'/wine.Barchart'
[10] (gif map id)+cbk #.PostScrp.MakeGIF pc
[11] a Bury the GIF and point to it in the HTML
[12] htm+'<HTML><HEAD>' '<Title>Sample Piechart</title>' '</HEAD>'
[13] htm,+map,c'<body bgcolor="#FFFFFF">'
[14] htm,+cBury(gif id)
[15] htm,+c'</body></HTML>'
v
```

This is where things get messy, but I can't think of a better way of ensuring that the correct picture is retrieved when the browser calls back for it.

```

vinit home
[1]  a Clear cache (and initialise freelist)
[2]  a Cache has space for 12 pictures
[3]  a Columns are: [;id,data,[]AI[2]]
[4]  Δcache+12 3p'' '' 1000000000
[5]  Δfreecache+12
[6]  Δhome+home
v

vimg+Bury arg;gif;map;id;inx;ai;exp
[1]  a Hide the gif in a transient cache (freelisted)
[2]  a Return the <IMG> section of the finished page keyed on the id
[3]  (gif id)+arg o ai+[]AI[2]
[4]  a Free and clear any time-expired entries ...
[5]  exp+(60000<ai-Δcache[;3])/1+Δcache
[6]  Δcache[exp;1 2]+<' ' o Δcache[exp;3]+1000000000
[7]  Δfreecache+exp
[8]  inx+Δfreecache o Δfreecache+1
[9]  Δcache[inx;]+(id gif ai)
[10] a Make a signpost back to the cache entry we just filled
[11] img+'
v

vgif+{sock}Readcache tokens;inx;id;ai
[1]  a Extract cached image, free slot and return
[2]  inx+Δcache[;1]+tokens[1;2]
[3]  (id gif ai)+Δcache[inx;]
[4]  'Gif extracted from cache for ',id
[5]  Δfreecache,+inx
[6]  'Cache entry ',(vinx),' is now available for reuse'
v

```

If you want to make a more professional version of this, I would suggest checking for errors in *Readcache* (line [3] could fail with an index error if someone had pictures turned off, went for lunch, then pressed 'Show Picture'), and perhaps extending the cache if the freelist was still empty on line [8] of *Bury*. In practice you have to push this quite hard to use more than one cache entry at a time.

Note that *Readcache* adds the freed entry to the *back* of the free-list, and does not actually clear the entry in the table. This allows for the possibility that someone might hit 'Show Picture' or 'Save Picture as ...' in the browser. As long as the site is not too busy, and their time has not expired, they will still be able to retrieve it.

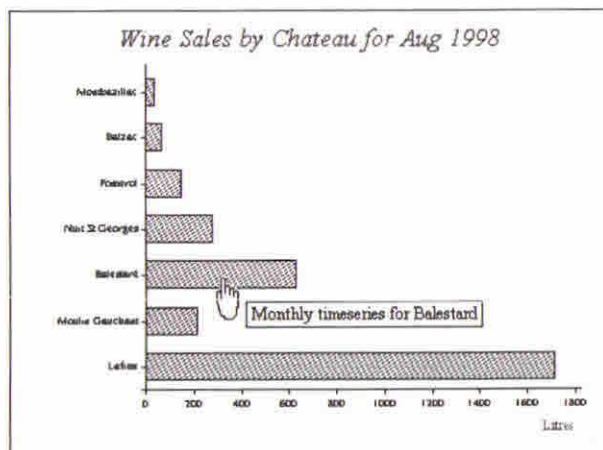
If we click on the third chateau, we generate a callback to the *Barchart* function:

```

v(htm)+=(sock)Barchart data;chat;map:gif;id;img;bc;cbk
[1]  a Cache barchart of annual sales as a GIF (or PNG)
[2]  a Passed the required chateau in the argument
[3]  a Returns the HTML to reference it (with image map)
[4]  chat+>data[1;2]
[5]  #.ch.Set'head'('Wine Sales by Month 1998 for ',chat>Δchat)
[6]  #.ch.Set'xlab'Δmonths
[7]  #.ch.Bar Δsales[chat;]
[8]  #.ch.Hint(12),1,[1.5](='Brand split for '),Δmonths
[9]  bc+#.ch.Close
[10] a Pass the function to be run from the mapped image
[11] a This will be returned, qualified with row= etc
[12] cbk+'http://',Δhome,'/wine.Monthchart'
[13] (gif map id)+cbk #.PostScrp.MakeGIF bc
[14] a Bury the GIF and point to it in the HTML
[15] htm+'<HTML><HEAD>' ' <Title>Barchart</title>' '</HEAD>'
[16] htm,+map,c'<body>'
[17] htm,+cBury(gif id)
[18] htm,+c'<p>Barchart of sales for Chateau ',chat>Δchat
[19] htm,+c'<HR>' 'a Causeway Graphical Systems Ltd 1998'
'</body></HTML>'

```

... this in turn maps the bars to give a monthly summary showing all the chateaux for that month, and so on:



In this case, all the hot areas are rectangles, and I have exploited Netscape-4's ability to show the Alt text as popups.

The generated code is quite lengthy, but much simpler to build:

```
<HTML><HEAD>
<Title>Sample HTML</title>
</HEAD>
<MAP name="chart400179">
<area shape="rect" coords="99,245,389,265" alt="Monthly timeseries
for Lafitte" href="http://127.0.0.1/wine.Barchart?row=1&col=1">
<area shape="rect" coords="99,213,136,233" alt="Monthly timeseries
for Moulin Gauchant"
href="http://127.0.0.1/wine.Barchart?row=2&col=1">

<area shape="rect" coords="99,181,206,201" alt="Monthly timeseries
for Balestard" href="http://127.0.0.1/wine.Barchart?row=3&col=1">
<area shape="rect" coords="99,149,147,169" alt="Monthly timeseries
for Nuit St Georges"
href="http://127.0.0.1/wine.Barchart?row=4&col=1">

...

</MAP>
<body bgcolor="#FFFFFF">
<h3>This shows the data split by month</h3>

</body></HTML>
```

Once you get the hang of this, you can give the impression of a marvellously complex web site, populated with hundreds of pre-cooked charts, which is actually all generated from a few simple data tables!

References

- [1] *The HTML Sourcebook*, Ian S. Graham, John Wiley 1995
- [2] *HTML Publishing for Netscape*, Harris & Kidder, Netscape Press 1996.

Appendix

The code in the *PostScrp* namespace relies on several internal variables which are designed to support graphical drill-down and editing from APL applications. These may change in future releases, so please read this as an example of how it might be done, not a definitive version. The relevant variables are:

```
locshdr+0 8p'' 0 0 1 1 '' 1 1 a Name, series, type, shape[2] ...
locadet+0 8p0 a hptr,where[x1 y1 x2 y2 0],series[r,c]
locshint+0 3p'' ''(0 3p0 0 '') a Chart, series, hints
```

```

v r+{callback}MakeGIF page;bb;clip;sz;hdr;bmp; ...
[1]  A Create bitmap (to match bounding box) and copy chart to it
[2]  A Return (GIF, scaled 1:1) and matching (<MAP>imagemap</MAP>) and (id)
[3]  scl=1
[4]  hdr+("1+{'newpath'gpage}11)+page
[5]  bb+0 0 432 324 psahdr'%PreviewBox:' o sz+bb[4 3]
[6]  'bmp'WC'bitmap' '({[scl*sz]p15)Δmap('Coord' 'Pixel')
[7]  'bmp'WS('Coord' 'User')('Yrange'(2+sz[1]))('Xrange'('~2+sz[2]))
[8]  PS page'bmp'scl Δmap
[9]  A ===== New 8.2 options - 260 for PNG =====
[10] bytes+2 ΔNQ'bmp' 'MakePNG'
[11] A Now scrag it for the imagemap (globals left by PS in the object)
[12] (hdr detail hint)+'bmp':locΔhdr locΔdet locΔhint'
[13] id+>hdr A Unique id for the chart
[14] hint+>hint[;3] A Just one chart here
[15] :If 2=ΔNC'callback'
[16]   map+c'<MAP name="',id,'">' o r2d+180+201
[17]   :For ct :In 11+pdetail
[18]     (r c)+>'detail[ct;7 8]
[19]     A Various series types (1=line, 2=bar, 3=pie)
[20]     type+detail[ct;1]>hdr[;3]
[21]     :Select type
[22]       :Case 2 A Barcharts are really easy (but swop corners here)
[23]       detail[ct;5 3]+sz[1]-detail[ct;5 5]
[24]       area+<area shape="rect" coords="
[25]       area,+(1+ε','',v*["detail[ct;2 3 4 5]],',''
[26]       A Would be nice to show data hints here ...
[27]       A Only seem to work reliably on bars?
[28]       :If (1+phint)>pos+(+hint[;1 2])<detail[ct;7 8]
[29]         area,+>' alt="',(pos>hint[;3]),',''
[30]       :Else
[31]         area,+>' alt="(r,c)=",r,',',c,'"
[32]       :End
[33]       :Case 1 A Linecharts get a 4-pixel hotzone
[34]       detail[ct;3]+sz[1]-detail[ct;3]
[35]       area+<area shape="circle" coords="
[36]       area,+(1+ε','',v*["detail[ct;2 3],4),',''
[37]       :Case 3 A Piecharts
[38]       detail[;3]+sz[1]-detail[;3]
[39]       area+<area shape="poly" coords="
[40]       area,+(1+ε','',v*["detail[ct;2 3] A Centre
[41]       A 20-degree intervals should be very adequate
[42]       circ+{1φ(φw),w[1]+20*1(20+21-/-w)detail[ct;5 6]
[43]       circ+detail[ct;4]*42 1*.ocirc*r2d
[44]       circ,((pcirc)pdetail[ct;2 3])+((pcirc)p1 ~1)*circ
[45]       area,+(ε','',v*["circ),',''
[46]     :EndSelect
[47]
[48]   A Append arguments to generic callback
[49]   area,+>' href="',callback,'?row=',r,'&col=',c,'">'
[50]   map,+>area
[51]   :End
[52]   map,+>c'</MAP>'
[53]   :Else
[54]     map+0p<'
[55]   :End
[56] r+(bytes map id)
v

```

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hacker's Corner: Context Help and COM in +Win	Ajay Askoolum	96
At Play with J: Maximum Infix Sums	Gene McDonnell	100
Word of Words from Iterated Function Systems	Reiter <i>et al</i>	104
Data Conversion Between APL Versions	David Crossley	125

More Utilities for the Dyalog Session by Ray Cannon	(held for Vector 15.3)
---	------------------------

Hackers' Corner: Context Sensitive Help and COM in APL+Win

by Ajay Askoolum (106173.3347@compuserve.com)

Context sensitive help driven by Windows is a powerful refinement to any application. Indeed this is a hallmark of any 'polished' or 'professional' Windows application and should be considered integral to the design of applications. There are several Microsoft Word add-on packages which facilitate the production of the *.hlp file. A help facility based on a *.hlp file offers the following advantages:

1. the *.hlp file is independent of the application itself. Provided that the topics are known, the *.hlp file compilation can be delegated to help authors: the application development and help file authoring can proceed concurrently.
2. the *.hlp file can be written in such a way that it can also be printed as the user manual.
3. the 'annotate' facility enables users to elaborate the content of the help topics within the *.hlp file itself.

By convention, context sensitive help is available on pressing F1: help on the context indicated by the position of the cursor is shown by WINHELP which then takes over, enabling selective browsing/printing/copying of the topics in the help file.

WINHELP "Displays Help for a particular topic identified by a *context number* that has been defined in the [MAP] section of the .HPI file." (Windows 3.1 SDK)

Now, the provision of WINHELP driven help does not seem straightforward. The APL+Win manuals do not cover this *at all*. However, APL+Win provides the building blocks.

If an edit object has focus, the name of the object can be found as follows:

```
('#' ⍵wi 'focus') ⍵wi 'name'
```

Note that the above expression yields the name of the edit object [works while the form on which the object appears in the Wait loop], not its context number. In fact APL+Win does not cater for a property which can contain the *context number*.

The APL+Win header file APLWADF.INI does not include the WinHelp call: that is, the `Wmcall` function **cannot** make sense of an application programming interface call as documented:

```

BOOL WinHelp(hwnd, lpszHelpFile, fuCommand, dwData)

HWND hwnd;           /* handle of window requesting help */

LPCSTR lpszHelpFile; /* address of directory-path string */

UINT fuCommand;      /* type of help */

DWORD dwData;        /* additional data */

```

APL+Win will recognise the call to WinHelp if the following line is added to the APLW.INI file:

```

[Call]
WinHelpIdx=B(HW,*C,U,C) ALIAS WinHelpA

```

Now, `Wmcall 'WinHelpIdx' {parameters}` will be recognised. This still relies on the context number.

The following alternative line (courtesy of Dave Phillips, many thanks Dave) overcomes this problem:

```

[Call]
WinHelpIdx=B(HW,*C,U,*C) ALIAS WinHelpA

```

Now, a line such as:

```
Wmcall 'WinHelpIdx' 0 'C:\APLWIN\APLGUI.HELP' 'HELP_KEY' 'Form'
```

will display help on topic Form from the Windows Interface help file provided with APL+Win. Note that the topic itself rather than the context number is used.

The F1 facility can be added to an application as follows:

```

Wmself+ 'form.Help' Wm 'New' 'Menu'
Wm 'caption' '&Help'

Wmself+ 'form.Help.Context' Wm 'New' 'Menu'
Wm 'caption' ('&Context', TCHT, 'F1')
Wm 'shortcut' 112 0
Wm 'prompt' 'Context Sensitive Help'
Wm 'onClick' 'CSHELP'

```

where the function CSHELP identifies the context (focus) and calls WINHELP.

Alternatively, the help facility can be provided from a floating menu (available on right click) as follows:

```

[]wself+ 'ROBOT' []wi 'New' 'Menu'

[]wself+ 'ROBOT.OPT1' []wi 'New' 'Menu' ('caption' 'Context Help')
[]wi 'onClick' 'CSHELP'

```

In fact both mechanisms can be deployed and since the help call is managed by the function CSHELP both will behave identically.

The function CSHELP is:

```

▽ CSHELP
[1] 0 0p[]wcall 'WinHelpIdx' 0 'C:\SAMPLE\APPLIC.HLP'
'HELP_KEY' (('#' []wi 'focus') []wi 'name')
▽

```

where C:\SAMPLE\APPLIC.HLP is the help file to use.

The help facility is now in place. What of the procedure for creating the help file itself? I can recommend *HelpBreeze* from SolutionSoft. It is time to search the WEB for further information on help compilers!

APL and the Component Object Model

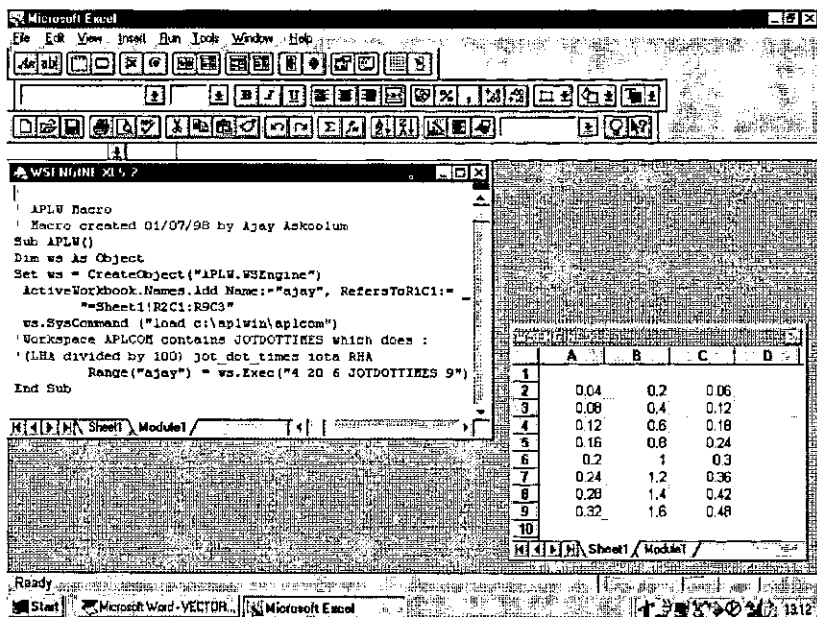
by Ajay Askoolum (106173.3347@compuserve.com)

APL+Win version 3.0 can act as an ActiveX server to Excel. As usual, the APL+Win Reference Manual provides the bare bones of this capability, on pages 4.43 to 4.46. I did not find this comprehensive enough to produce a quick working example.

However, in case you are interested in using this, the screen print below provides an example which illustrates this capability.

In this *simple* example, a range in an Excel Sheet is updated by running a macro, APLW, which calls APL+Win to load a workspace, run the function

JOTDOTTIMES with the arguments supplied. By default, the taskbar does not show an APL+Win session.



The function JOTDOTTIMES is :

```

▽ Z←L JOTDOTTIMES R
[1]  a Example of APL+WIN acting as an ActiveX server to EXCEL 5.0
[2]  Z←(L×0.01)×.×1[R
▽
  
```

Note that the appropriate registrations detailed in the manual must be carried out for this example to be reproduced. Also, note the implicit $\&$ of the result when it gets to Excel.

At Play with J: Maximum Infix Sums

by Gene McDonnell

In his book *Programming Pearls* (Addison-Wesley 1986), Jon Bentley collected a baker's dozen of his columns of the same name that had appeared in *Communications of ACM*. Column 7 therein is called "Algorithm Design Techniques", and the problem discussed in there is the subject of this column. Here's the problem: Given a vector x of numbers (positive, negative, or zero), define a function f that yields the maximum of the sums of all the infixes. For example, if

```

      x
31  _41 59 26 _53 58 97 _93 _23 84

```

then $f\ x$ is 187, the sum of 59 26 _53 58 97. If all the numbers are positive the maximum infix is x , with sum $+x$. When all inputs are negative the maximum infix is the empty vector, with sum 0.

How many infixes are there in a vector? An infix can start at any point and end at any point. The number of infixes starting at the very beginning is $\#x$, one for each possible ending point. The number of infixes beginning at the next position is $(\#x)-1$; at the next is $(\#x)-2$, and so on; the total is thus the triangular number $-\colon (\#x) * (1 + \#x)$, or, expressed functionally, $-\colon @(*>:)$, one of the series 1 3 6 10 15... The number of infixes for $\#x$ where x is a power of 10 are:

$\#x$	$-\colon @(*>:)x$
1	1
10	55
100	5050
1000	500500
10000	50005000
100000	5000050000
1000000	500000500000

So even for a vector of length 1000, the number of infixes to consider is greater than half a million. Bentley gives as a bad example an algorithm (I think written in Pascal) that tests the sum of each infix, and shows that a computer that takes an hour for $\#x\ 1000$ will take 39 days for $\#x\ 10000$.

```

MaxSoFar := 0.0
for L := 1 to N do
  for U := L to N do
    Sum := 0.0
    for I := L to U do
      Sum := Sum + X[I]
    /* Sum now contains the sum of X[L..U] */
    MaxSoFar := max(MaxSoFar, Sum)

```

He goes on to discuss faster algorithms, but which still take quadratic time. He describes then a subquadratic algorithm which uses $O(N \log N)$ time. He tells how he and a colleague thought this was probably the best possible. They described this at a meeting at Carnegie Mellon University, and someone in the audience designed a much improved linear time algorithm in less than a minute. Several APL colleagues of mine have voiced their suspicions that the designer knew APL.

The algorithm (again I think written in Pascal) for the linear-time algorithm is:

```

MaxSoFar := 0.0
MaxEndingHere := 0.0
for I := 1 to N do
  /* Invariant: MaxEndingHere and MaxSoFar
     are accurate for X[1..I-1] */
  MaxEndingHere := max(MaxEndingHere+X[I], 0.0)
  MaxSoFar := max(MaxSoFar, MaxEndingHere)

```

This can be translated into a J verb:

```

mis =: monad def '>./(0:>.+)/\..y.,0' NB. maximum infix sum

```

We append 0 to the end of the list argument ($y., 0$), because the last value on the list may be negative, then do a suffix scan ($/\..$) using the verb 'the maximum of zero and the sum so far' ($0:>.+$). This produces a list of sums formed according to the rule that if a negative sum is encountered it is replaced by zero. Finally, the maximum of all the sums is yielded ($>./$). To see how this works we use `mis0`, which is like `mis` but without the maximum over, on a short list:

```

y =: _100 2 3 4 _100 5 6 _7 8 9 _100
mis0 y
0 9 7 4 0 21 16 10 17 9 0 0
mis y
21

```

The maximum infix is 5 6 _7 8 9, with sum 21.

The need for appending a zero may be demonstrated by using `mis1`, which is like `mis0` but without the appended zero:

```
j1=:13 : '(0:>.)/\.y.'
j1 y
0 9 7 4 0 12 7 1 8 0 _100
```

Jon Bentley showed the advantage of linear over quadratic algorithms by writing his slow algorithm in fine-tuned Fortran and running it on a Cray-1 supercomputer, and the linear algorithm on a Basic interpreter running on a Radio Shack TRS-80 Model III microcomputer. The runtime for an argument of length N was $3.0N^3$ nanoseconds for the slow algorithm on the Cray, and $19.5N$ milliseconds ($19,500,000N$ nanoseconds) for the fast algorithm on the TRS-80. The linear algorithm on the slow machine caught up with the slow algorithm on the fast machine at $N = 2500$, and for $N = 1,000,000$, the slow algorithm/fast machine took 95 years (estimated) and the fast algorithm/slow machine took just 5.4 hours.

The moral of the story is something like, "It's foolish to improve performance by getting a faster machine; one's money is better spent on finding a faster algorithm." This isn't always possible, of course. Machine designers are always finding ways to build faster machines, but it takes a very rare skill to devise faster algorithms.

This brings me to a confession. The J version of the algorithm wasn't translated from Pascal (which is a language I don't understand), but from Arthur Whitney's K. Many of you will know of the existence of two different APL successors, J and K; some of you will know of their progenitors: Ken Iverson and Roger Hui for J, and Arthur Whitney for K. There are connections among these three. All are from Alberta, in Canada (Roger Hui by way of Hong Kong); all worked for I. P. Sharp Associates in Toronto (Whitney also worked for IPSA in Australia, Hong Kong, and Singapore), and all are good friends. I might also say that all have first-rate brains. That they have taken divergent paths in pushing the APL idea into the next millenium might lead you to think that their relationship might be strained, and they do indeed have a rivalry in that regard; but it is a friendly, though determined one. I use both J and K, and believe that each language is excellent, but also that each has features that I wish the other had.

The K algorithm, which I learned from Arthur Whitney (we live just a mile or so apart in Palo Alto, California) is:

```
f : |/(0|+)\
```

In K the vertical bar (|) signifies the dyad 'larger'. So | / is 'the largest of all, or maximum'. This is applied to a scan using the hook 0|+, where 0| is the monad 'larger of 0 and the argument', and + is the dyad 'plus', of course. In K the scan (like over) begins at the left. The 0 preceding (0|+) is the initial value of the scan, and serves the same purpose as the appended 0 at the end of the J function j. The verb f also illustrates that K can be used as a functional language, but unlike J, there is no stress laid on that fact. The verb could also have been defined by:

```
f : { | / 0 (0|+) \ x }
```

Since I have both J and K on my machine (K can be downloaded from the website kx.com) I was motivated to time m1s and f on similar length arguments. The results are interesting.

The arguments are named xn, where 10^n is the length of the list.

	J	K
x1	0.00017	0.000386
x2	0.0005	0.0024
x3	0.0077	0.0212
x4	0.082	0.089
x5	0.87	0.532
x6	--	5.091

J is faster for arguments up to 1000 items. and there is a tie for arguments 10000 items long, but K is faster for larger arguments, and, indeed, can handle much larger arguments than J can. I've reported this difficulty with J and perhaps by the time you read this it will have been remedied.

Word of Words from Iterated Function Systems

by Benjamin J. Reiter, Clifford A. Reiter, Zachary X. Reiter



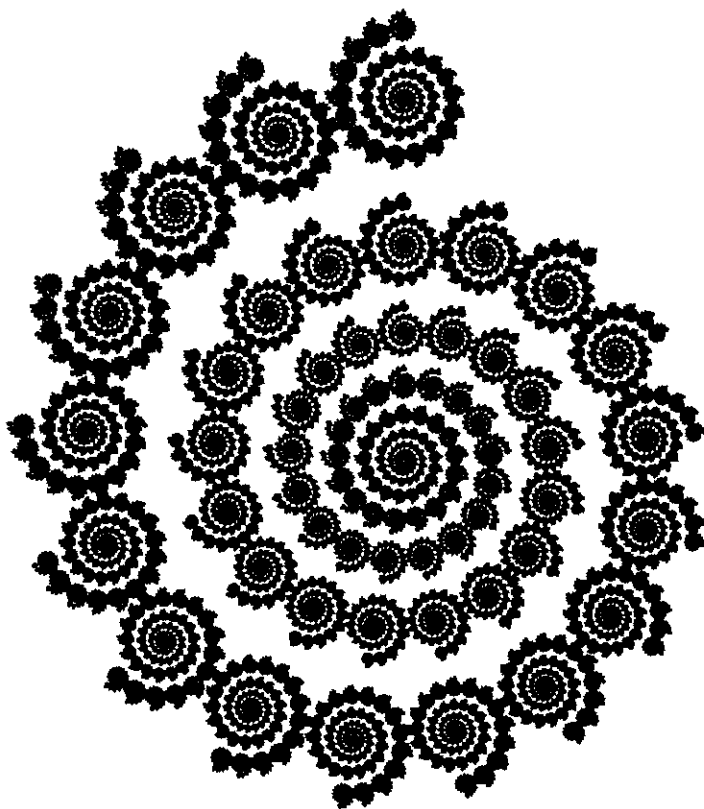
Introduction

Barnsley [1] and others including [9,10,12] have described techniques for constructing wonderful and amazing images from simple formulas known as *iterated function systems*. The purpose of this note is to give an experimental introduction to these ideas based on Cliff's experience in teaching this topic in his linear algebra and visualization classes. The most dramatic illustration we give is of a mini J application that two of Cliff's sons, Ben and Zach, wrote as their final project in a version of his visualization class that they attended.

Before we get to the good stuff (the J and making the pictures), we need to say a bit about the mathematics and the history. The images we will create are created using "iterated function systems" which are simply collections of functions. Here we think of a function as taking points in the plane as argument and resulting in another point in the plane; hence we also call these functions transformations or maps.

Usually we want these functions to be "nice" in the sense that they move points closer together. Namely, they are contractions – we will make this precise in the next section.

Figure 1 shows a slight variation on the Barnsley fern which is a classic, beautiful example of an object that can be created using four especially simple functions. While [1] is highly recommended for study by advanced undergraduate mathematics students, the mathematics there could be daunting for the less seasoned; yet iterated function systems provide many opportunities for motivating familiarity with the behaviour of transformations of the plane and hence encourage creativity – all the while giving valuable experience with fundamental ideas of computer graphics: geometrically useful transformations of the plane.



The basic ideas are delightfully simple and trying to create an image with a certain pattern using iterated function systems can be addictive. For example, the spiral in Figure 2 is produced with just two simple transformations. Can you guess what they are? Barnsley's text also contains a classic "COKE" of "COKE"s

created with 14 transformations similar to Figure 3. This has some resemblance to the "Reductionism/Holism" sketch on p.310 in Hofstadter's *Gödel, Escher, Bach* [7]. The letters of the "Coke" are somewhat crude; we will see that we can use functions that transform the plane with bending (really just conversion to polar coordinates) to improve the fractal font; but in any case, the patterns are fascinating.



Figure 3: Affine COKE

Readers can have fun trying to duplicate these pictures, or better yet, creating a fractal version of their name. You can also think about how to automate the process of creating iterated function systems that have specified words as their result. Alternately, you may view this paper as provoking ideas for your own explorations in another language or simply as an introduction to a playful application available over the web that creates fractal "words of words" with curved letters.

Iterated Function Systems

Iterated function systems are collections of functions. There is a beautiful mathematical theory that explains when iteration of these systems converges to a "nice" set. Roughly speaking, one wants the functions to be contractions. A function f defined on a subset of the plane is a *contraction* if there is a constant $0 \leq c < 1$ so that $|f(y) - f(x)| \leq c |y - x|$ for all x and y in the domain of f . All this means is that points move, under the application of f , toward each other by at least a factor of c . We will illustrate what we mean by iteration of such a system in a moment.

Our examples will use J which is available from <http://www.jsoftware.com>. First consider a function that halves each of its coordinates. We define `f` in J using a functional programming style. The symbol `&` bonds (or curries) the multiplicative factor `0.5` to the times function yielding the function that is multiplication by half. Then this is composed with other arithmetic operations using `@` to denote composition.

```
f=:0.5&*           Function that multiplies by half
f 3 4             Evaluate f at 3 4
1.5 2
```

```
g=:0.5&+@f        Function that adds half after multiplying by half
g 3 4
2 2.5
```

```
h=:0 0.5&+@f"1    This function adds 0.5 only to the second coordinate
h 3 4
1.5 2.5
```

At the end of the definition of `h` the use of `"1` designates the rank of application of `h` to be one. That is, `h` operates on the rows of its input (1-dimensional cells), not the individual entries. These three functions can be made into a formal collection by tying them together as a composite object. In J this is done using tie, ```, and the result is called a gerund since it is made up of functions (verbs) acting as objects (nouns). There are two ways in which iterated function systems are typically iterated. The first way is to apply all the functions to all the points creating a collage of results. The other way we "iterate" is to apply a randomly selected transformation; then each point in the input corresponds to a single point in the output.

```
]sq=:4 2 $ 0 0 1 0 1 1 0 1  The vertices of the standard unit square;
0 0                          $ reshapes data into a matrix;
1 0                          the identity function ] displays the
1 1                          result
0 1
```

```
collage=:f`g`h`0           Function giving the collage of the
                             transformations
collage sq                 The collage of the square results
0 0                         in three transformed squares
0.5 0
0.5 0.5                     f on sq
0 0.5
```

0.5 0.5
1 0.5
1 1
0.5 1

g on sq

0 0.5
0.5 0.5
0.5 1
0 1

h on sq

The top left of Figure 4 shows the square and the top middle shows the three images of the square under the collage of the three given transformations. We can look at the collage of the collage of the squares that result from another application of the collage function. These squares are given by $\text{collage}^2 \text{ sq}$ where $\wedge$ designates iteration.

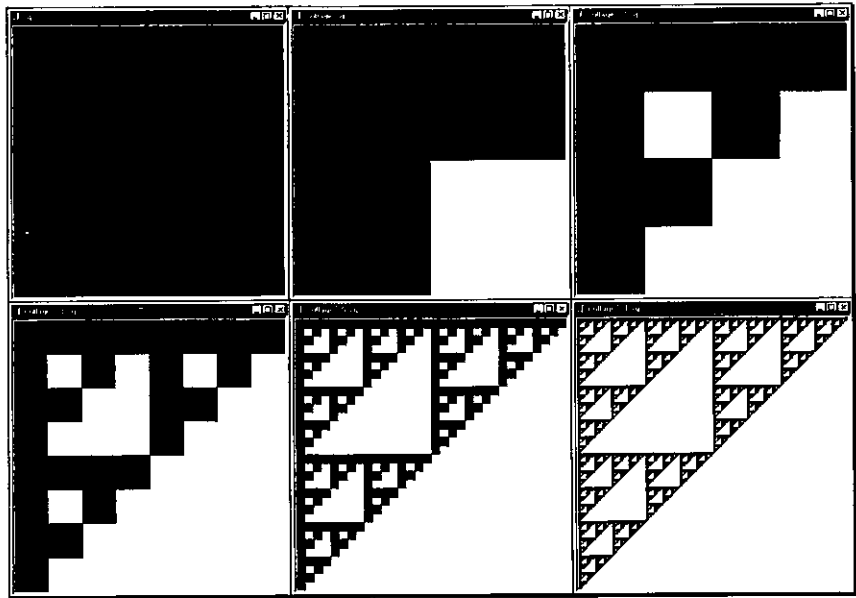


Figure 4: Iteration of collages

Figure 4 shows that second iterate in the upper right and also shows some further iterates of this process of creating collages. Notice that if the original square was replaced by another polygon, or even a single point, after several iterations of collage the image would be indistinguishable from our last image. Indeed, if the iterated function system is made up of contractions there is a unique image

resulting from the iteration of collages and that image forms a pattern such that taking the collage of the pattern results in the same pattern; precise statements and proofs appear in [1].

We now consider iteration of our function systems in the second way. Here we want to select one of the transformations at random and apply it. We use agenda, denoted @. (the dot distinguishes this from composition which we saw was denoted $\circ$). Agenda selects which function among $f \circ g \circ h$ to apply according to the function on its right. In the definition of rt below the selection is determined according to the function $?@3$: . That function randomly selects among the first three indices: 0, 1 or 2.

<pre> ?@3: 1 1 2 ?@3: 1 1 0 rt=:f`g`h@.(?@3:) rt 1 1 0.5 0.5 rt 1 1 0.5 1 rt 1 1 1 1 rt 1 1 0.5 0.5 </pre>	Random index Another random index Random Transformation Randomly apply one transformation
--	---

We can apply random transformations one after another by listing more than one iteration number. It is convenient to list successive indices for the desired powers.

<pre> i.8 0 1 2 3 4 5 6 7 rt^:(i.8) 1 1 1 1 0.5 0.5 0.75 0.75 0.375 0.875 0.6875 0.9375 0.84375 0.96875 0.421875 0.484375 0.210938 0.242188 </pre>	Several successive indices Several successive random transformations
--	--

Of course, if we randomly iterate enough times we will end up near the attractor for the iterated function system. We are using non-zero probabilities for each transformation; thus, if we randomly iterate enough we will visit near every point on the attractor. Images of the points visited can be created by marking the entries

in a matrix corresponding to pixels. In procedural languages this is often done a pixel at a time via interaction with the operating system. In J this can be accomplished by working on a matrix representing an array of pixels and saving the result to a graphics file which can then be viewed. The J script `wwifs.ijs` contains utility functions that accomplish this in a way that displays intermediate stages. You can obtain the script from a link to "Word of Words" from the fractal page <http://math.pardee.lafayette.edu/Cliff/J>. Once you have J running, you can run the script `wwifs.ijs` from the "run" menu in J. Here are the steps you should take to create a raster image of the iterated function system demonstrated above once you have run the script and have defined `f`, `g`, `h` and `rt` as above.

```
makevra 'Fractal'
```

```
10 rtiter 1 1
```

Create a graphics window with given title;
`vra` stands for virtual raster array.

This computes and displays 10 thousand iterations of `rt` with 1 1 as a starting point; but the first hundred pixels are discarded.

You can make the image more complete by running more iterations. Before working with more complicated examples, we will consider homogeneous coordinates as a way to organize our transformations.

Homogeneous Coordinates

Homogeneous coordinates for points in the plane can be thought of as the usual two coordinates with a third coordinate that is always 1 attached. Thus 0.75 0.5 in ordinary Cartesian coordinates is 0.75 0.5 1 in homogeneous coordinates. At first glance this seems silly and wasteful, but it turns out that it allows us to think of the usual linear transformations of the plane along with translations of the plane in a uniform manner. Namely, using matrix multiplication. In most mathematics texts, matrices are used to transform points by left multiplication. In computer science the convention is reversed, and since this keeps the J experiments slightly simpler, we will use the computer science convention.

It is convenient to think about three simple geometric transformations of the plane: scaling, rotation and translation. We can accomplish the scaling by putting the scale factors on the diagonal, rotations about the origin involve multiplication by a matrix of sines and cosines of the angle (which is a standard application of linear algebra [4]), and translations can be accomplished by putting the amount to be translated in the last row.

In particular, the form of these matrices is as follows.

scaling	rotation	translation
$r \ 0 \ 0$	$\cos(t) \ -\sin(t) \ 0$	$1 \ 0 \ 0$
$0 \ s \ 0$	$\sin(t) \ \cos(t) \ 0$	$0 \ 1 \ 0$
$0 \ 0 \ 1$	$0 \ 0 \ 1$	$u \ v \ 1$

Now we multiply out the scaling transformation in homogeneous coordinates; we see

$$(x_{new}, y_{new}, 1) = (x, y, 1) \begin{pmatrix} r & 0 & 0 \\ 0 & s & 0 \\ 0 & 0 & 1 \end{pmatrix} = (rx, sy, 1).$$

Thus, the x -coordinate is multiplied by r and the y -coordinate is multiplied by s . The rotation matrix applies a clockwise rotation through an angle of t and the translation adds u to the x -coordinate and v to the y -coordinate. Some examples are as follows.

```

]A=:3 3$0.1 0 0 0 0.5 0 0 0 1          Scaling
0.1  0 0
0 0.5 0
0 0 1

```

Recall that `$` denotes "reshape" so the definition of `A` above can be read "A assigned the 3 by 3 reshape of the data".

```

2 3 1 +/ . * A          Matrix multiplication is +/
. *
0.2 1.5 1

```

```

]C=:3 3$1 0 0 0 1 0 0.4 0.6 1          Translation by 0.4 0.6
1 0 0
0 1 0
0.4 0.6 1

```

```

2 3 1 +/ . * C
2.4 3.6 1

```

```

t=: +/ . * & C          Translation as a function
t 2 3 1
2.4 3.6 1

```

We bonded the matrix to matrix multiplication in order to obtain a function that applies directly to the homogeneous coordinate points. Notice that when doing matrix arithmetic directly, the input point appears on the left while when applying the transformation t , the input point appears on the right (as is usual for functions).

A function that builds rotation matrices is given below. This uses the trigonometric functions which can be loaded with the `trig.js` script that comes with J or we can use the built in circular functions denoted `o.` along with the constant functions `0:` and `1:` and the juxtaposition functions denoted with comma and comma-colon.

```
sin=: 16o.          sine is the first circular function
cos=: 26o.          cosine is the second

rotm=: (cos , -@sin , 0:),(sin , cos , 0:),: 0: , 0: , 1:

]pi=: 1p1           1p1 designates 1 times pi to the first power
3.14159

rotm pi % 4          45° clockwise rotation matrix;
0.707107 _0.707107 0 % denotes division
0.707107 0.707107 0
0          0 1
```

As an illustration we will think about the two transformations that are required to produce the spiral of spirals in Figure 2. First, there is a transformation that scales the entire image by a factor of 0.2 and translates the image by 0.4 0.8 producing the top spiral. We can produce that transformation by multiplying suitable matrices together.

```
]a=: 3 3$0.2 0 0 0 0.2 0 0 0 1
0.2 0 0
0 0.2 0
0 0 1

]b=: 3 3$1 0 0 0 1 0 0.4 0.8 1
1 0 0
0 1 0
0.4 0.8 1
```

```

x =: +/ . *           Name matrix multiplication x for brevity

a x b
0.2   0 0
0 0.2 0
0.4 0.8 1

s=: x&(a x b)         The transformation applying a and b

s 1 1 1
0.6 1 1

```

The second transformation we want maps the image to the entire remainder of the image after the one subspiral is removed. This is a composite of rescaling the image by a factor 0.97 in both coordinates, rotating by a 19th of a turn, and translating slightly to centre the image.

```

]d=:3 3$0.97 0 0 0 0.97 0 0 0 1
0.97   0 0
0 0.97 0
0      0 1

]e=:rotm _2r19 * pi    Note _2r19 denotes the rational number  $-\frac{2}{19}$ 

0.945817 0.324699 0
_0.324699 0.945817 0
0          0 1

]f=:3 3$1 0 0 0 1 0 0.189 _0.14 1
1      0 0
0      1 0
0.189 _0.14 1

t=:x&(d x e x f)

rt=:s`t@.(?@2:)       Random transformation

```

Again, if you have run the `wwifs.ijs` script you can create a graphics window and iterate `rt` using `makevra` and `rtiter` as described in the previous section. *However, be sure to remember to make your initial guess a point in homogeneous coordinates since that is what these transformations act upon.* Your image will not be as balanced as Figure 2 unless you run an extremely large number of iterations. The reason for this is because we really should select the transformation that is to produce greater area with greater likelihood. This can be done relatively easily by replicating `t` several times. Here is a way to replicate `t` several times without much typing. The function `#` copies its right argument the specified number of times.

```

2 4 # 5.4 7
5.4 5.4 7 7 7 7

```

Copies of numbers

```
1 3 # s` t
```

Copies of functions

s	t	t	t
---	---	---	---

In a similar way we can create `rt` with `s` appearing once and `t` appearing 20 times. To complete that we also need the constant function `21` which is written `21" _` in the definition.

```
rt=(1 20#s`t)@.(?@(21"_))
```

Iterating this `rt` ought to allow you to replicate Figure 2 with a modest number of iterations.

A nice first exercise is to implement the transformations associated with the fern in Figure 1. The parameters are given in the Appendix. Which transformations should be repeated and how much in order to get a balanced image? Then trying to duplicate other interesting fractals can be fun, valuable experience with transformations. Can you create variants of the spiral? Can you duplicate the Coke of Cokes? Can you create a fractal version of your name?

All of our examples so far could be written in terms of matrix multiplication in homogeneous coordinates. These are known as *affine transformations*. In the next section we take a deeper look at fractal words that includes nonaffine transformations based on conversion to polar coordinates.

Word of Words

The letter E can be thought of as being formed by 4 straight strokes; that is, we want to find four affine transformations that when applied to the standard unit square (its coordinates take values between 0 and 1) gives rise to those strokes. Then, the attractor of those affine transformations ought to be an E made up of Es. The three horizontal strokes can be thought of as being created by a scaling followed by a translation. The vertical stroke can be thought of in the same way. However, we have in mind extending these transformations in a way that leads to words, hence the horizontal strokes will fill nicely with words while the vertical stroke would be crunched horizontally. While crunching can be interesting, we can avoid crunching with a rotation. Notice that we already saw a transformation of the type: scale, rotate, and then translate in the previous section when we

created `t`. Since transformations of these types are quite convenient for building letters, we introduce some utilities for constructing the transformations.

If you have run `wwifs.ijs`, these utilities are already defined and you can immediately experiment with them but we list them below since they are concise. Unlike our previous J functions, these are defined explicitly in terms of left and right arguments; the left argument is denoted `x.` while `y.` designates the right argument. Several new J primitive functions appear without further comment there, but they can be looked up in the online help in a J session if desired.

Before looking at the utilities themselves, we return to our first example, the middle stroke of the E. It can be formed by taking the standard unit square and contracting by a factor of three quarters (`3r4`) in the *x*-coordinate and by one eighth in the *y*-coordinate. Translating by `1r8 7r16` centres it vertically and moves it to the right so it will abut the vertical stroke which will have width one eighth. See Figure 5 for an illustration of these steps applied to an E.

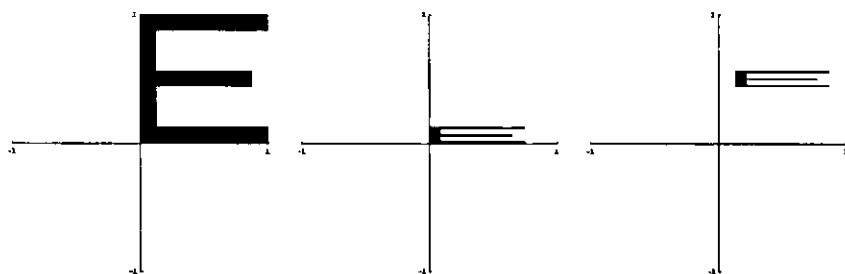


Figure 5: Middle Stroke of an E. Scale and Translate

```
satm=: 3 : '3 $1j3 1j1 1 1 1#y.,1'      Scale and translate matrix
                                          :
satm 3r4 1r8 1r8 7r16                    3r4 is 0.75
0.75      0 0
0      0.125 0
0.125 0.4375 1

sat=: 1 : 'x & (satm x.)'                Gives the function resulting
                                          from satm

e0=: 3r4 1r8 1r8 7r16 sat
```

```

e0
x & 3r4 0 0
    0 1r8 0
    1r8 7r16 1
e0 0 1 1
1r8 9r16 1

```

The vertical stroke can be thought of as being formed by scaling vertically by one eighth and not scaling horizontally (this transformation isn't technically a contraction, but since it is "on the edge", we get away with it!). Rotating this $\pi/2$ clockwise results in transforming the standard unit square to a vertical strip below the x-axis. Translating 0 1 moves it to the left edge of the unit square as desired. See Figure 6. The function `ratm` creates a matrix that does this scaling, rotation and translation. The mnemonic for this ought to be `sratm` but `ratm` seemed more winsome. The right argument gives the angle for the rotation and the left gives scaling and translation. The result of `rat` with the same arguments is the function that applies the transformation given by that matrix.

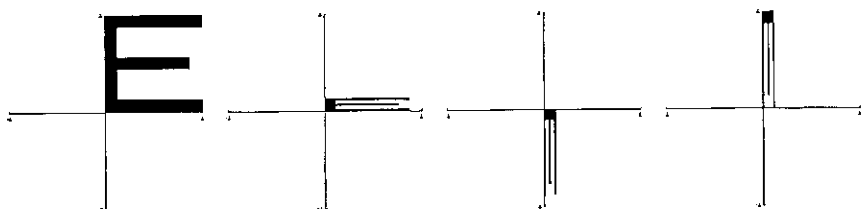


Figure 6: Vertical Stroke of an E. Scale, Rotate and Translate

```
ratm=: 4 : '(satm (2{.x.),0 0)x (rotm y.)x satm 1 1,-2{.x.'
```

```
rat=: 2 : 'x & (x. ratm y.)'
```

Transformation that applies the
ratm matrix

```
e1=:1 1r8 0 1 rat (1r2 * pi)
e1
```

Some roundoff error appears

```

x & 6.12303e-17 -1 0
    0.125 7.65379e-18 0
    0 1 1
e1 1 1 1
0.125 0 1

```

Similar transformations can be applied to other strokes, `e2` and `e3` of `E` and so we can combine these to get a list of transformations associated with the letter `E`.

```
e2=:7r8 1r8 1r8 0 sat
e3=:7r8 1r8 1r8 7r8 sat
E=:e0`e1`e2`e3
```

We are ready to discuss curved strokes. A simple way to deal with these strokes is to map the standard unit square to the intersection of a sector with an annulus. That is, the Cartesian coordinates are scaled, translated and interpreted as polar coordinates. The function created by `(a,b) pol (c,d)` maps the unit square to the polar region $a \leq r \leq b, c \leq \theta \leq d$. The letter G can be thought of as being formed by 3 quarter circle curved strokes along with two short straight horizontal strokes, one at the top and the other in the middle. If we map the standard unit square to $\frac{3}{8} \leq r \leq \frac{1}{2}, \frac{\pi}{2} \leq \theta \leq \pi$ we get a quarter circular stroke of the size and orientation appropriate for the upper left stroke of the G. We can translate by 0.5 0.5 in order to position the stroke. See Figure 7. We use the utility `pol` to map to the region described. The definition below gives the details and, as before, if you have run `wwifs.ijs`, this utility is already defined. While the utility does a good bit of arithmetic, the main function is `r.` which converts from polar to Cartesian coordinates.

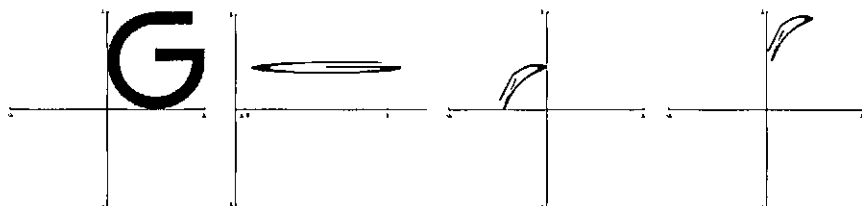


Figure 7: The upper-left curve of a G. Scaled and translated, interpreted as polar co-ordinates and translated.

Note, here we combine J notation for rational and pi related constants for the first time: `1r2p1` designates one half pi to the first power.

```
pol=: 2 : '({(.x.)&-@((-/x.)&*)@{1&{}} ,&1@+.,@r.
              ({(.y.)&-@((-/y.)&*)@{0&{}})"1'

g0=:1 1 1r2 1r2 sat@(3r8 1r2 pol 1r2p1 1p1)

g0 1 1 1
0 0.5 1

g0 0 0 1
0.5 0.875 1
```

Below we create the other four transformations for the letter G.

```
g1=:1 1 1r2 1r2 sat@(3r8 1r2 pol _1p1 _1r2p1)
g2=:1 1 1 1r2 1r2 sat@(3r8 1r2 pol _1r2p1 0)
g3=:3r8 1r8 1r2 7r8 sat
g4=:1r2 1r8 1r2 4r8 sat
```

If we want to combine the letters E and G into a word this can be done by composing the transformations given for each letter with a simple affine transformation in the x-coordinate in order to squeeze all the letters into the interval from 0 to 1. Then plotting the result on a virtual raster array suitably expanded in the horizontal direction gives a balanced image. For example, if we wanted to spell the word EGG, we can create three transformations scaled by one third in the x-coordinate and shifted according to letter position.

```
s0=:1r3 1 0 0 sat
s1=:1r3 1 1r3 0 sat
s2=:1r3 1 2r3 0 sat
```

Now we can create a random transformation composing the strokes for each letter with the appropriate shift.

```
rt=:s0@e0`(s0@e1)`(s0@e2)`(s0@e3)
    `(s1@g0)`(s1@g1)`(s1@g2)`(s1@g3)`(s1@g4)
    `(s2@g0)`(s2@g1)`(s2@g2)`(s2@g3)`(s2@g4)@.(?@{14"_))

150 450 makevra 'egg'                                Make a long window
```

Figure 8 shows the result of running `rtiter 1 1 1` for that `rt` (at higher resolution). Can you figure out how to make the curved words in the G change orientation so they are directly readable?

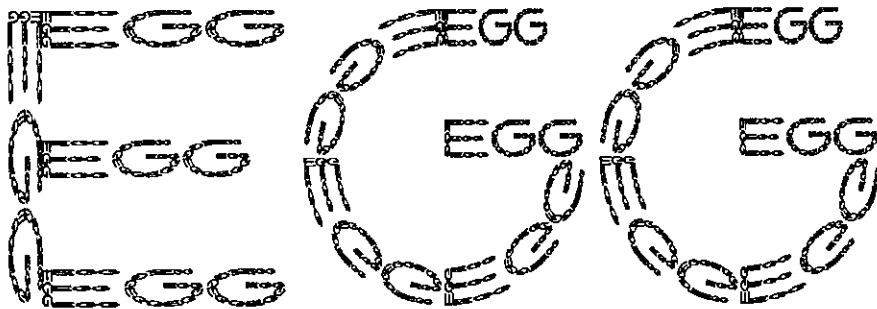
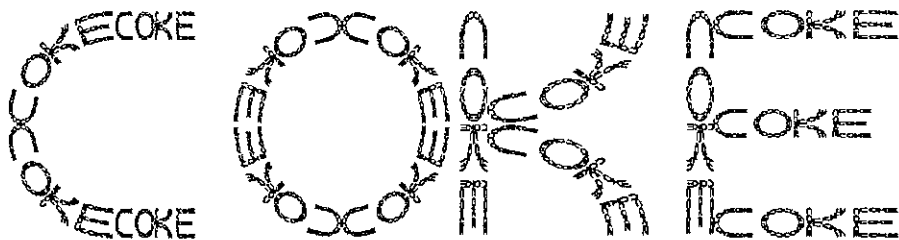


Figure 8: Egg of Eggs

The above idea allows you in principle to create a fractal word of words based on any word given the strokes required for the letter. It is quite inconvenient to do the final x-coordinate scaling by hand as above, but that process can be easily automated. The script `wwifs.js` contains a program for automatically creating the random transformation based simply on the input word. You can run the program in Windows versions of J by running the following.

```
wwifs ''
```

You can type in any word, an iteration count and the number of pixels to use for each letter. The program will produce a word of words based on the appropriate iterated function system and display the result. Figure 9 shows an example of a fractal "COKE" produced that way. While this version is much more readable than the version in Figure 3 it is doubtful it will win any font design prizes. How many levels of COKE within COKE can you see?



Further Reading

Other sources of ideas for fun projects with iterated function systems include Gröller [6] who looks at nonlinear iterated function systems on two and three dimensional grid arrays. Frame and Angers [5] look at nonlinear iterated function systems including functions such as $az^n + b$ in the complex plane. Sprott [13] automatically finds affine iterated function systems that have visual appeal. These fractals are abstract but quite interesting. Reiter [12] contains several playful images such as a fractal smile and spiral of smiles of spiral of smiles that his students enjoy trying to replicate. While we identified our transformations by eye, automating the process of finding transformations that result in digitized photographic images has important implications for image compression [2] so these topics are more than merely playful.

Cliff Reiter

Department of Mathematics, Lafayette College, Easton PA 18042 USA

Email: reiterc@lafayette.edu Web: <http://math.pardee.lafayette.edu/Cliff/J>

Appendix: Parameters for the Fern

The matrices in normalized homogeneous coordinate form that were used to construct the fern in Figure 1 are as follows.

0	0	0	0.85	-0.04	0	0.2	0.23	0	-0.15	0.26	0
0	0.16	0	0.04	0.85	0	-0.26	0.22	0	0.28	0.24	0
0.25	0	1	0.0375	0.17	1	0.2	0.1025	1	0.2875	-0.021	1

References

- [1] M. F. Barnsley, *Fractals Everywhere*, 2nd edition, Academic Press, Cambridge MA, 1993.
- [2] M. F. Barnsley and L. P. Hurd, *Fractal Image Compression*, AK Peters, Wellesley, MA, 1993.
- [3] O. M. Carducci, Four elementary linear algebra projects, *PRIMUS*, 3 4 (1993) 337-344.
- [4] J. B. Fraleigh and R. A. Beauregard, *Linear Algebra*, Third edition, Addison-Wesley Publishing, Reading, MA, 1995.
- [5] M. Frame and M. Angers, Some nonlinear iterated function systems, *Computers & Graphics*, 18 1 (1994) 119-125.
- [6] E. Gröller, Modeling and rendering of nonlinear iterated function systems, *Computers & Graphics*, 18 5 (1994) 739-748.
- [7] D. Hofstadter, *Gödel, Escher, Bach*, Vintage Books, New York, 1980.
- [8] J. Hutchinson, Fractals and self-similarity, *Indiana University Journal of Mathematics*, 30 (1981) 713-747.
- [9] H-O Peitgen, H. Jürgens and D. Saupe, *Chaos and Fractals*, Springer-Verlag, New York, 1992.
- [10] H-O Peitgen and D. Saupe, *The Science of Fractal Images*, Springer-Verlag, New York, 1988.
- [11] C. A. Reiter, Linear Algebra and Laboratories, *Mathematica in Education*, 2 2 (1993) 7-9.
- [12] C. A. Reiter, *Fractals, Visualization and J*, Iverson Software Inc., Toronto, 1995.
- [13] J. C. Sprott, Automatic generation of iterated function systems, *Computers & Graphics*, 18 3 (1994) 417-425.

Data Conversion Between APL Versions

by David Crossley (adcrossley@pacwan.fr)

Many APLers will have had some experience of transferring objects between different versions of APL. The usual technique for user-defined functions (and operators if these are supported) is to transfer the canonical representations (`⎕CR` forms). For variables, I have seen various approaches used. One method is to include a header with each variable that identifies its name, data type, rank and shape, followed by the ravelled data as though typed in. At the receiving end, the data part may then be executed (with quotes placed around character variables), re-shaped, and assigned to its name. A similar method re-constitutes each variable from an executable string including the name assignment and re-shape expressions. A nested array using this approach can look very complex! In addition, the transferred characters must be converted from one character set to the other if the internal representations of the atomic vectors (`⎕AVs`) of the two versions of APL differ, which they usually do.

These methods are not very efficient. The processing at each end is quite significant, but more importantly, the amount of data to be transferred for a variable is considerably larger than the space the variable actually consumes in the workspace. For infrequent transfers, this may not matter. However, for regular transfers through a network with constant interaction between remote systems, the priorities change somewhat. Both elapsed time and network transfer volumes must be kept to a balanced minimum.

Table 1 indicates the maximum size overhead per element for each data type if a simplistic transfer technique is employed. For booleans and numerics to be re-assembled by execution, a space character is required between elements, and the requisite number of digits and other paraphernalia must be represented in order to retain accuracy.

Data type	Workspace size per element	Maximum transfer size per element
Character	1 byte	1 byte
Boolean	1 bit	2 bytes
Integer	4 bytes †	12 bytes
Real	8 bytes	23 bytes
† Some APLs also support 1 and 2 byte Integers		

Table 1: Maximum element sizes using string execution

Of course, the maximum sizes will often not be required. Integers will realistically require far less than the theoretical 10 digits plus sign and separating space. Trailing zeros can be omitted from floating point numbers. But in most cases, and always for booleans, there is a big overhead compared with workspace storage size. This is especially the case for floating point numbers, which comprise most business data.

Another Approach

The idea occurred to me some time ago that a good way of transferring data between APLs might be to convert to one or other of the internal data structures used by the different APL versions. After all, the data storage methods of modern APLs are a reasonably efficient compromise between storage space versus processing costs when functions are applied.

The 8-bit byte is the usual base unit for transport protocols, whether it be TCP/IP, or a file-to-file protocol such as IBM's IND\$FILE facility, or whatever. For a binary transfer, all 256 bit combinations of a byte may be used. Thus, a Real (Floating Point) number that is represented in the workspace by 8 bytes may also be transferred using 8 consecutive bytes. The same applies for other data types, but for Boolean arrays, padding may be required to extend the single bits defining the data up to a byte boundary. We can think of the byte representation of data as a text string, although the text if printed would appear as gibberish. How to convert to and from text string form, and between the representations of those data structures in the different versions of APL, is the principal subject of this article.

A further factor that induced me to experiment with this approach is the availability of a function in some APLs to convert a variable from one data type to another. I believe this concept was first introduced on mainframe APL*PLUS as a "fast function" called CHTYPE (for Change Type). This subsequently became available for the PC version of APL*PLUS (now known as APL+Win) as the `⌈DR` system function (for Data Representation), and later in Dyalog APL version 8. The dyadic form of `⌈DR` converts the array given as its right argument to the form indicated by a numeric code (or codes) specified by its left argument, providing that the conversion is feasible (otherwise *DOMAIN ERROR* is reported). In fact, this simply involves altering the data type and shape of the array. For example,

```
82 ⌈DR REALNRS
```

converts each item of Real array *REALNRS* to 8 characters defined by the bit patterns of the consecutive sets of 8 bytes that comprise a Real number; 82 is the code that says convert to a Character array. If *REALNRS* were a 1-column matrix,

the result would be an 8-column Character array. To convert from Character to Real, the length of the last axis must be a multiple of 8, unless the array is an empty vector. The combined bit pattern must also be a valid Real representation (not all bit patterns are). The monadic usage of `⌈DR` returns the data type code for a given array, e.g. 82 for a Character array.

This system function or the `CHTYPE` equivalent is extremely quick, providing the basis for fast algorithms to perform the conversions for the transport process and re-constitution in the receiving workspace. Although the conversions could easily be achieved using native APL functions with quite acceptable performance, the important advantage of rapid conversion at one end of the process is lost.

Development Platforms

For the purpose of concrete example, the article will describe the approach based on the actual platforms for which it was developed, although the same approach could be applied between any APL versions using whatever transport protocol is available.

Specifically, transfer of data is required in both directions between applications running under Dyalog APL/W Version 8 on PCs (the clients), and an IBM mainframe running APL*PLUS under CP/CMS (the server). The transport protocol uses IBM's `IND$FILE` facility which sends data via a CMS file and a DOS file, to be picked up by the client/server as appropriate. These processes are controlled interactively by APL programs at each end without direct user intervention. However, the transport procedures are not of concern for this article; therein lies another tale. The applications involve frequent interactions between client and server, with sometimes quite large data transfers (up to perhaps 2 Megabytes of workspace storage), mainly from server to client. The critical criterion for a transfer process is the elapsed time that a transfer takes. This is affected by the processing times at each end to perform the conversions, the actual transmission time of the transfer, and the messaging and synchronisation protocols that are required. The described approach impacts on the first two of these.

Data Representations

The way in which arrays are stored in the workspace is dependent on the APL vendor, although they will be influenced by the hardware platform for which the version is designed and by convention amongst implementers.

For Intel-based PCs, covering 80386, 80486 and Pentium processors, the storage of numeric data is likely to conform to the IEEE standards, since these are fully supported at the micro-code level. Another standard is that for IBM 360/370 machines. These standards are reviewed below.

By convention, all simple APL arrays are defined by a header followed by a data part. The word "tuple" has been coined (tuppence!) to describe this two-part string. It is not a word in the dictionary that I have found, but if Lewis Carroll can invent words, why not?

Taking Dyalog APL as an example, the header contains the following:

- Length of header and data parts in words 4 bytes
- Array type (Simple or Nested) 1 byte
- Rank, i.e. number of dimensions 1 byte (0-15 limit)
- Element type, e.g. CHAR,BOOL etc. 1 byte
- Pad to make up 4 byte boundary 1 byte
- Shape, number of words = rank 4 bytes per shape item

The data part contains elements in ravelled order, padded at the end if necessary to align to a 4-byte boundary. If the array is nested (and perhaps if it is heterogeneous, though Apl+Win handles this in a single tuple), each nesting structure is identified as a "pointer" type, and its data part is a set of pointers to other tuples. A nested array is therefore a linked set of tuples. At the end of each path through the structure, there may be several pointer tuples, but ultimately it must end with a tuple for a simple array.

For transfer purposes, exactly how the header part is identified is not important, although the information is required to reconstruct the array correctly. However, APL will look after the header during this process, e.g. when `⎕DR` is used to convert the data, and the shape information is used to re-shape the array. I use a similar construct to the above as follows:

- Length of the header and data parts in bytes 4 bytes
- Element type, e.g. CHAR,BOOL etc. 1 byte
- Compression methods used 1 byte
- Rank, i.e. number of dimensions 1 byte

- Shape, number of words = rank 4 bytes per shape item

Note that for transfer, the base unit size is a byte, as opposed to a 4-byte word in the workspace, so there is no need to pad the header. The Compression Method indicates how the transferred array has been compressed, if at all, to be discussed later.

The conversion process is applied to the data part of the array, which is always a ravelled vector. There are, in fact, two aspects to the conversion, although the actual processes will sensibly be consolidated:

- conversion, if necessary, because the data representation in the workspace differs between the APL versions;
- conversion of this representation to text string form for the transfer.

If there are data representation differences, the conversions both to and from the form required for the other should be undertaken on the same platform, normally where it can be done most quickly. Also, since different code must be developed for the two versions, to, say, convert Real numbers, it is much more reliable and easier for testing to have pack/unpack functions that are reversible. Applying the packing function to an array and then applying the unpack function to its result, i.e. the transfer form, should provide the original array within precision limits. It also means that on the other, presumably slower, platform, the data conversion process is much simpler, and almost trivial if a facility like `⎕DR` is available.

For the discussion case, the mainframe APL*PLUS server takes the brunt of the processing task, leaving the Dyalog APL client with a rapid, re-assembly task. The remaining sections discuss conversion issues relating to these specific platforms, although the underlying logical processes may be extrapolated to other platform combinations.

Translation of Binary Data

The strategy described here relies on generating the exact bit patterns, represented as bytes for transfer, that will reproduce the data part exactly in one of the APLs (in this case, Dyalog APL). It is therefore very important that the transport protocol passes the data without translation. There are usually options that allow for automatic translation when writing to or reading from a transfer file. The option must be disabled, set to "No Translation".

Specifically for Dyalog APL, writing to and reading from native files, i.e. DOS files, assumes a default translation. The file must be set to "No Translation" by the statement:

$((:256)-\square IO)\square NXLATE \textit{tn}$ where $\textit{tn}$ is the negative file tie number.

Equally important, a similar setting must be made for $\square DR$, otherwise a conversion will be made in converting to and from the byte strings, thus:

$((:256)-\square IO)\square NXLATE 0$

Conversion of Character Arrays

A re-mapping of characters is required if the $\square AV$ s of the respective APLs do not match either in the characters that they represent, or in the bit patterns that describe each character. Neither condition is met in the discussion case.

The re-mapping is achieved by defining a 256-element Character vector in the APL*PLUS workspace in which each element identifies an APL*PLUS character whose position in the vector has the bit pattern that matches the equivalent character in Dyalog APL. The equivalent bit pattern for a given position number is $(8\rho 2)\textit{tnumber}$. Beware that this position may not be the *subscript* of the character in Dyalog APL's $\square AV$. For example, the letter 'a' has origin-0 position 38 in APL*PLUS's $\square AV$; in Dyalog APL, its bit pattern translates to 97 whereas its subscript in $\square AV$ is 17. 'a' needs to be mapped to the position in the translation vector that has the bit pattern equivalent to 97. As it happens, this is also position 97, since the bit patterns in APL*PLUS's $\square AV$ progress consecutively from 0 to 255, but it must be verified. Characters with no equivalent are mapped arbitrarily to bit pattern 255. In most business uses, this should not occur in practice, but for function transfers, it needs to be handled in some way.

Thus, at the APL*PLUS end, Character data is translated for transfer as follows:

Outgoing: $\square AV[(256-\square IO)\lfloor \underline{\Delta\Delta AV}\textit{vector}]$ where $\underline{\Delta\Delta AV}$ is the translation vector

Incoming: $\underline{\Delta\Delta AV}[(256-\square IO)\lfloor \square AV\textit{vector}]$

At the Dyalog APL end, no translation is required, either outgoing or incoming.

This is the sole case in which translation is required for $\square AV$ differences, in contrast to the string execution approach when all transfers require this translation.

Conversion of Boolean Arrays

Both APL*PLUS and Dyalog APL store Boolean data in the same way. For outgoing transfer, the vector must first be extended up to a byte boundary:

```
shape←pbool ∘ bool←,bool
bool←8×⌈(pbool)÷8⌉+bool
```

Recording the shape and raveling of the object are assumed for all further examples.

Both versions support data type conversion functions, which may therefore be used for rapid conversion to/from text string form:

Outgoing	Incoming
APL*PLUS: <i>str</i> ←'CB'CTYPE <i>bool</i>	<i>bool</i> ←shapep'BC'CTYPE <i>str</i>
Dyalog APL: <i>str</i> ←82 ⌈DR <i>bool</i>	<i>bool</i> ←shapep11 ⌈DR <i>str</i>

Conversion of Integer Arrays

Both APL*PLUS and Dyalog APL store 4-byte Integer data in a similar, though not identical, way. Dyalog APL also supports 1-byte and 2-byte Integers if each element *n* of the array is respectively in the range $-128 \leq n \leq 127$ or $32767 \leq n \leq 32768$. It is worthwhile taking advantage of this space optimisation.

In principle, the first bit of the internal storage form of an Integer indicates the sign of the number and the remaining bits identify the value. In Dyalog APL, there may be 7, 15 or 31 value bits, whereas in APL*PLUS there are 31.

For a positive number, with a sign bit of 0, the expression *2×value_bits* gives the numerical value.

For a negative number, with a sign bit of 1, a form known as "twos-complement" is used. Here, the value bits are reversed, and the expression $-1+2^{1-\text{value_bits}}$ gives the numerical value.

Intel-based machines, on which the Windows version of Dyalog APL is installed, have a further twist. All numeric values that take more than one byte are stored in reverse byte order. For example, the number 1234 is stored as a 2-byte Integer which conceptually may be thought of as:

```

0 0 0 0 0 1 0 0   1 1 0 1 0 0 1 0
↑    1st byte      2nd byte
sign bit

```

but is actually stored as:

```

1 1 0 1 0 0 1 0   0 0 0 0 0 1 0 0
      2nd byte      ↑    1st byte
                    sign bit

```

This is sometimes known as a "low-endian", as opposed to a "high-endian", method of storage. The same principle also applies to the storage of Real numbers.

At the APL*PLUS end, for fast conversion:

Outgoing: `S+S+3=S++/1, (127 32767<[/int)~128 ~32768>[/int
str+,φ(~2 F 3 2 0[1 2 4: S])+ 'CI'CHTYPE((pint),1)pint`

Incoming: `T+ 'BC'CHTYPEφ(φS, [(pstr)+S)pstr
int+ 'IB'CHTYPE ((1+8×3 2 0[1 2 4: S])/T[;, [IO]), 0 1+T
int+shapepint`

At the Dyalog APL end:

Outgoing: `str+82 [DR int`

Incoming: `int+83 163 323[1 2 4: S][DR str`

Conversion of Real (or Floating Point) Arrays

A Real number is an abstract mathematical concept. Its value may lie in the range between $+\infty$ (plus infinity) and $-\infty$, and between any two Real numbers there lie an infinite number of Real numbers. Any digital representation of a Real number is therefore usually an approximation. Strictly speaking, a Floating Point number is an approximation of a Real number, though the terms are often used as though synonymous. We normally work using decimal values. A binary representation of a decimal value again is usually not exact, so rounding is required when a conversion is made from decimal to binary and vice versa. Also, even if numbers are represented in binary form, there are varying methods of storage, and again rounding may be required when numbers are converted from one platform to another.

A Floating Point number is stored in three parts, being its *sign*, an *exponent* and a *mantissa*. The mantissa identifies a number of significant digits up to the limit of

the storage method. The exponent is a power to which a base value will be raised. The represented number is conceptually calculated from:

$$(-1^{\text{sign}}) \times \text{mantissa} \times \text{base}^{\text{exponent}}$$

For computer storage, the first (logical) bit represents the sign (0 for positive, 1 for negative), followed by a number of bits that represent the exponent, and the remaining bits in the allocated storage size represents the mantissa. Normally, the base is 2, but the IBM 360/370 convention has a base of 16.

In order to give the exponent a range from negative to positive without using an explicit sign bit, a *bias* is added to the actual exponent for storage, normally at the centre of the range.

To make comparison and arithmetic simpler, the mantissa is *normalised*. With a base 2 exponent, this means that the bits in the mantissa are shifted such that the left-most bit (i.e. the most significant bit) is a 1.

Consider an example with decimal numbers. There are many ways of representing the value 10.34 using scientific E notation, such as 1034E-2, 0.01034E3, 10.34E0 etc. The normalised form is 1.034E1. That is, the decimal is shifted so that just one non-zero figure occurs to its left. The number of places to shift is $\lfloor \log_{10} \text{number} \rfloor$ and this becomes the exponent value.

In storage form, the adjustment is slightly more complicated as we have to consider the precision, i.e. the maximum number of digits that can be stored. Let us consider this number 10.34 represented in base-10, a 2-digit exponent with a bias of 50, and 6 digits of precision in the mantissa:

- The mantissa takes the number 103400 (i.e. 6 digits)
- Its normalised value is 1.03400, i.e. $103400 \div 10^5$ (1 digit before the decimal)
- The mantissa calculation is $10.5 + 10.34 \times 10^{\lfloor \log_{10} 10.34 \rfloor}$
- The corresponding exponent, before applying bias, must then be $\lfloor \log_{10} 10.34 \rfloor$, i.e. 1
- The biased exponent is $50 + 1$, i.e. 51
- The sign is 0 for positive (it would be 1 for negative)
- The number would be represented in sign-exponent-mantissa notation (spaces shown for clarity) as 0 51 103400

The same principle is applied to the digital representation. Table 2 identifies pertinent characteristics for IEEE and IBM 360/370 Standards for 8 byte Real numbers.

Characteristic	Intel/IEEE Standard 754	IBM 36/370 Standard
Sign bit (s)	1 bit: 0 = +ve / 1 = -ve	1 bit: 0 = +ve / 1 = -ve
Exponent (e)	11 bits: Base 2, Bias 1023	7 bits: Base 16, Bias 64
Mantissa (m)	52 bits + 1 implicit bit	56 bits
Storage order	Low byte to high byte	High byte to low byte
Normal numbers	$0 < e < 2047$	$0 \leq e \leq 127$
Sub-normal numbers	$e = 0$; at least one bit in m	Not supported
Zero	$e = 0$; $m = 0$	$e = 0$; $m = 0$
Max number	1.797693134862316E+308 $e=2046, m=\text{all } 1\text{s}$	7.237005577332262E+75 $e=127, m=\text{all } 1\text{s}$
Min positive normal number	2.225073858507201E-308 $e=1, m=0\text{s}$ (implicit 1st bit)	5.397605346934028E-79 $e=0, m=1$ in 4th bit
Min positive sub-normal number	4.940656458412466E-324 $e=0, m=1$ in last bit	Not supported

Table 2: Standards for 8-byte Real Numbers

For IEEE representation, a little trick is applied to the mantissa. Although 52 bits are stored, since the normalised form adjusts the mantissa such that there is a 1 in its leftmost bit, it can be assumed. This is known as the *implicit* bit. It effectively gives 53 bits of precision. This trick cannot be pulled off for the IBM 360/370 standard, because the exponent base is 16, i.e. 2^4 . The condition for a normalised number is therefore that there must be a 1 in any of the first 4 bits, as shift adjustments may only be made in powers of 16.

As previously mentioned, Intel storage is low-endian whereas IBM 360/370 storage is high-endian. The 8-bytes of an Intel Real number must be rotated to place the bits in notional order for the sign-exponent-mantissa convention.

Zero is considered to be a special case for the IEEE standard. This is because of the implicit bit, so an exponent of 0 is reserved and outside the range for normalised numbers. Actually, the standard supports positive and negative zero, and so does Dyalog APL from a translated bit pattern. The conversion algorithms must treat zeros separately, mainly to avoid *DOMAIN ERROR* when taking the logarithm.

The standard also uses a 0 exponent to handle sub-normal numbers. These are numbers between 0 and the smallest absolute number that can be represented as a normal number. The calculation for a sub-normal number is:

$$(-1^s) \times (2^{-1022}) \times (21m) + 2 \times 52$$

This extension provides for "gradual underflow", important in high-precision calculations when, for example, two small numbers are subtracted. Sub-normal numbers are supported by Dyalog APL, but not by APL*PLUS on the mainframe (where the result from underflow is 0).

The IEEE standard also makes use of the highest exponent value 2047 for special purposes, but these are not supported by Dyalog APL. Out of interest, if the mantissa is 0, this signifies ∞ , either + or - depending on the sign bit. Any other mantissa value is termed NaN (for Not a Number). Dyalog APL reports a *DOMAIN ERROR* for these cases, so it is not a good idea to translate arbitrary bit strings.

Another point to notice when converting is that the largest absolute number that can be represented is very much greater in Dyalog APL than in mainframe APL*PLUS. It is therefore necessary to prevent a calculation when converting from Dyalog APL to APL*PLUS that would exceed this number otherwise *DOMAIN ERROR* raises its head again.

Because Real number conversion is the most complex of the conversions, and most critical in terms of processing costs, three different variants are presented in order to discuss the problems that occurred during its evolution to the final version. Each function handles the packing and unpacking tasks in the mainframe APL*PLUS workspace, converting from IBM360 form to IEEE form. I have excluded full header comments from the first two but not the last to reduce space.

The task at the Dyalog APL end is trivial, amounting to:

Incoming: *realnos*+645 $\square$ DR *string*

Outgoing: *string*+82 $\square$ DR *realnos*

This is the first mainframe APL*PLUS variant. I have marked the lines that differ by the bar character against the comments.

```

      V R←A PKREAL1 B;⊂CT;E;I;M;S;Z
[1]  A←Pack/Unpack Real numbers...Mainframe APL*PLUS version
[2]  A Variant 1: uses 1 and 7
[3]
[4]  ⊂CT←0 ⊙ +A+L1
[5]  A Pack object from IBM360 form to Intel IEEE form
[6]  R←(Z+R≠0)/R+,B A compress zeros
[7]  S←R<0 ⊙ R+|R A extract sign, make +ve
[8]  R←|0.5+R×2×52-E+|2⊙R A | normalised mantissa
[9]  E←E+1023 A | add bias

```

```

[10]  R+Q1=(52P2)T R A      | mantissa bits
[11]  E+Q1=(11P2)T E A      | exponent bits
[12]  R+S,E,R O S+E+O A      put together with sign bit
[13]  R+Q'CB'CHTYPE R A      character equivs, 8 per Real
[14]  R+Z\A O R[(-Z)/1PZ];+OAV O R+,R A expand to restore zeros
[15]  +EXIT
[16]
[17]  A Unpack object from Intel IEEE form to IBM360 form
[18]  L1:R+Q((1(+/OPR)÷8),8)PR+,B A group by Real nr
[19]  R+'BC'CHTYPE R A      convert to bits
[20]  I+V/O 1+R A          note zeros
[21]  R+Q R A              | transpose now
[22]  S+R[1;] O E+1 0+-52 0+R O R+12 0+R A | sign, expont,mantissa
[23]  E+-1023+21E A          | base 2 exponent
[24]  E+E*-M+E>251 A        | adjust if maximum number exceeded
[25]  R+1+(21R)*2*-52 A      | value represented by mantissa
[26]  R+I*R*2*E O E+I+O A    | non-zero * mantissa * 2* exponent
[27]  +(1E M)+L2 O R[M/1P M]+[/10 A        maximum nr adjustments
[28]  L2:+(1E S)+EXIT O R[S/1PS]+-S/R A      apply sign
[29]
[30]  EXIT:
      V

```

This variant works on the Real values themselves, performing the arithmetic to work out the mantissa and exponent values for the IEEE representation. It does not explicitly use any of the parameters for IBM 360/370 format, although certain relationships are implied. Firstly, in line [8], the normalised mantissa is generated. This requires an exact integral representation to work, but as just 53 bits are required for this calculation, whereas an IBM360 Real number allows 56, there is no problem.

Also, since the largest absolute number and the smallest absolute numbers are well within the range supported by IEEE, there is no need to check for these conditions. This is not the case when unpacking IEEE numbers. A check must be made to ensure that the largest number is not exceeded before re-calculating from the mantissa and exponent (otherwise a *DOMAIN ERROR* will occur); in this case, the algorithm sets the unsigned value to $1/10$, i.e. the largest representable number. However, there is no need to check for underflow, as APL*PLUS simply sets the number to 0.

Some precision may be lost in the conversion as the IBM360 standard has up to 3 extra bits representing the mantissa. (Recall IEEE's implicit bit, and the fact that there may be up to three leading 0 bits in the IBM 360/370 mantissa).

Notice that the CHTYPE fast function is used only to convert from Boolean to Character form, and vice versa.

Although the most straightforward of the variants, it suffers from a considerable drawback. This is caused by the way the *Encode* function works (the one with the bar on top τ - hands up all those who have to try it out at the desktop), and it may well be common to all versions of APL; certainly it is to those I am using. Lines [10-11] perform an encode to convert the integral representations, but actually Real numbers, to Boolean with an encoding base of 2's. The result of this function has the same type as its right argument. Thus, until forced Boolean, but too late, the temporary result to obtain the mantissa bits amounts to 416 bytes per item. With 10,000 Real numbers, that would require 4,160,000 bytes. The algorithm needs to handle 100,000 Real numbers in about 4 Megabytes of free workspace. Amen to that idea.

The second variant follows another approach:

```

v R←A PKREAL2 B;CT;E;I;L;M;S;U;Z
[1]  A V Pack/Unpack Real numbers...Mainframe APL+PLUS version
[2]  A Variant 2: Conversion by bit manipulation
[3]
[4]  CT←0 ⋄ A←L1
[5]  A Pack object from IBM360 form to Intel IEEE form
[6]  R←(Z←R÷0)/R, B A compress zeros
[7]  S←R<0 ⋄ R←|R A extract sign, make +ve
[8]  R←'BR'CHTYPE((pR),1)pR A | convert to real to boolean
[9]  S←R[;1] ⋄ E←0 1+0 ~56+R ⋄ R←0 8+R A | sign,exponent,mantissa
[10] L←+/Λ0=((pR),3)+R A | rotate to place 1 in leftmost posn
[11] E←4×~64+21qE A | IBM360 exponent
[12] E←q1=(11p2)τE+1023-L A | IEEE exponent
[13] R←0 ~4+(L+1)φR A | rotate inc 1st bit/keep 52 bits
[14] R←S,E,R ⋄ S←E+L+0 A put together with sign bit
[15] R←'CB'CHTYPE R A character equivs, 8 per Real
[16] R←Z\R ⋄ R[(-Z)/1pZ;]+[AV[1] ⋄ R+,R A expand to restore zeros
[17] →EXIT
[18]
[19] A Unpack object from Intel IEEE form to IBM360 form
[20] L1:R←φ((l(+/pR)÷8),8)pR+, B A group by Real nr
[21] R←'BC'CHTYPE R A convert to bits
[22] I←v/0 1+R A note zeros
[23] S←R[;1] ⋄ E←0 1+0 ~52+R ⋄ R←0 12+R A | sign, expont,mantissa
[24] E←~1023+21qE A | IEEE exponent
[25] E←64+|E+(E←E-L+~4|E)÷4 A | IBM360 exponent to base 16
[26] M←E>127 ⋄ U←E<0 A | >maximum number/Underflow ?
[27] E←q1=(7p2)τE~MVU A | exponent bit pattern
[28] R←Lφ((pR),56)+(1=1),R A | extend mantissa inc implicit bit
[29] R←S,E,R ⋄ S←E+0 A | put together, 64 bits/real
[30] R←,'RB'CHTYPE R A | convert boolean to real
[31] +(1εU)+L2 ⋄ R[U/1pU]+0 A | underflow adjustments
[32] L2:→(1εM)+L3 ⋄ R[M/1pM]+L/10 A maximum nr adjustments
[33] L3:→(1εS)+EXIT ⋄ R[S/1pS]+-S/R A apply sign
[34]
[35] EXIT:
v

```

In this version, the conversion process is done by bit manipulation, an intellectually appealing approach. Boolean solutions to problems are well-understood by APLers, often providing new insights, and performing very highly.

Firstly, the bit representation of the Real numbers is obtained using `CHTYPE`, and the sign, exponent, and mantissa matrices are extracted.

The IBM360 mantissa has up to three leading 0-bits, so these are counted, L say. The mantissa matrix is then rotated by $(L+1)$ bytes, to also include the first 1-bit, and the last 4 bytes are then dropped. This leaves a 52-bit mantissa, the IEEE form with the first bit being an implicit 1.

The exponent is complicated by the bias, so it is enumerated, bias-adjusted and multiplied by 4. Adding the IEEE bias of 1023 almost gets us there. It is also necessary to compensate for the mantissa rotation by deducting L , since the rotation is equivalent to multiplying the mantissa by 2^L . The revised exponents are then converted back to binary equivalents.

Finally, the sign, exponent and mantissa matrices are joined together, giving the logical IEEE representation. Convert to byte form, rotate, and there we have the transport form, ready for instant conversion in Dyalog APL.

The unpacking process is almost the reverse process, except that checks must be made both for numbers exceeding the maximum absolute value and for underflow. This is done by checking the IBM 360/370 exponent values before conversion to binary, which must be in the range 0 to 127. Exponents outside this range are arbitrarily set to 0, and then the recomposed Real numbers are adjusted later.

This algorithm works quite well. It is fast. It will handle 50,000 Real numbers with 4 Megabytes of free workspace, but it runs out of steam before reaching 100,000 numbers, predictably giving `WS FULL` on line [10] when packing, and on line [27] when unpacking.

The third, and final, variant, which I have left fully-commented, is:

```

v R←A PKREAL3 B;⎕CT;E;I;M;S;Z
[1]  ⍝ Pack/Unpack Real numbers...Mainframe APL+PLUS version
[2]  ⍝ Variant 3: Avoids 1 and 1, alternate code using CHTYPE
[3]  ⍝ A ↔ LS  0:pack, 1:unpack
[4]  ⍝ B ↔ VIV A=0 Pick element to read/replace in global
[5]  ⍝ B ↔ TV  A=1 Text string to be re-constituted
[6]  ⍝ R ↔ A   A=1 Re-composed Real object

```

```

[7]
[8]  A Character form of Real vector B converted from IBM360 form
[9]  A to IEEE storage form
[10] A IBM360 form IEEE form
[11] A  $(-1 \times S) \times (2 \times m) \times 16 \times (2 \times e) - 64$   $(-1 \times S) \times (2 \times m) \times 2 \times (2 \times e) - 1023$ 
[12] A s:sign bit 63 (1) 63 (1)
[13] A m:mantissa 62-56 (7) 62-52 (11)
[14] A e:exponent 55-0 (56) 51-0 (52+1 implicit bit)
[15] A Order As described Intel storage, i.e.
[16] A 8 storage bytes reversed
[17]  $\square CT+0 \circ \rightarrow A+L1$ 
[18] A Pack object from IBM360 form to Intel IEEE form
[19]  $R \leftarrow (Z+R \neq 0) / R+, B \text{ A}$  compress zeros
[20]  $S \leftarrow R < 0 \circ R+|R \text{ A}$  extract sign, make +ve
[21]  $R \leftarrow |0.5 + R \times 2 \times 52 - E + |2 \times R \text{ A}$  | normalised mantissa
[22]  $E \leftarrow E + 1023 \text{ A}$  | add bias
[23]  $R \leftarrow (2 \times 2 \times 26) \uparrow R \text{ A}$  | mantissa as 2 integers
[24]  $R \leftarrow R \uparrow R \text{ A}$  | force Integer
[25]  $R \leftarrow ((pE), 52) p0 \text{ 64 'BI'CHTYPE}((2 \times pE), 1) pR \text{ A}$  | mantissa bits
[26]  $E \leftarrow 0 \text{ 21 + 'BI'CHTYPE}((pE), 1) pE \text{ A}$  | exponent bits
[27]  $R \leftarrow S, E, R \circ S \leftarrow E + 0 \text{ A}$  put together with sign bit
[28]  $R \leftarrow \phi \text{ 'CB'CHTYPE } R \text{ A}$  character equivs, 8 per Real
[29]  $R \leftarrow Z \wedge R \circ R[(-Z) / \uparrow pZ;] \leftarrow \square AV[1] \circ R+, R \text{ A}$  expand to restore zeros
[30]  $\rightarrow EXIT$ 
[31]
[32] A Unpack object from Intel IEEE form to IBM360 form
[33]  $L1: R \leftarrow \phi((\uparrow \uparrow / \rightarrow pR) \div 8), 8) pR+, B \text{ A}$  group by Real nr
[34]  $R \leftarrow \text{'BC'CHTYPE } R \text{ A}$  convert to bits
[35]  $I \leftarrow v / 0 \text{ 1 + } R \text{ A}$  note zeros
[36]  $S \leftarrow R[1;] \circ E + 0 \text{ 1 + } 0 \text{ } \sim 52 + R \circ R + 0 \text{ 12 + } R \text{ A}$  | sign, exponent, mantissa
[37]  $R \leftarrow ((2 \times pS), 26) pR \text{ A}$  | split bits
[38]  $R \leftarrow ((\rightarrow pR), \sim 32) \uparrow R \text{ A}$  | adjust bits as 2 integers
[39]  $R \leftarrow \phi((pS), 2) p \text{ 'IB'CHTYPE } R \text{ A}$  | convert to Integers
[40]  $R \leftarrow (2 \times 26) \downarrow R \text{ A}$  | convert to mantissa values
[41]  $R \leftarrow 1 + R \times 2 \text{ } \sim 52 \text{ A}$  | decimal mantissa
[42]  $E \leftarrow \sim 1023 +, \text{'IB'CHTYPE}((pS), \sim 32) \uparrow E \text{ A}$  | exponent to Integer, less bias
[43]  $E \leftarrow E \times \sim M + E \div 251 \text{ A}$  | adjust if maximum nr exceeded
[44]  $R \leftarrow I \times R \times 2 \times E \circ E + I + 0 \text{ A}$  | non-zero  $\times$  mantissa  $\times 2 \times$  exponent
[45]  $\rightarrow (1 \times M) \div L2 \circ R[M / \uparrow pM] \div \uparrow / 10 \text{ A}$  maximum nr adjustments
[46]  $L2: \rightarrow (1 \times S) \div EXIT \circ R[S / \uparrow pS] \div \rightarrow S / R \text{ A}$  apply sign
[47]
[48] EXIT:

```

v

This, in fact, is a speeded-up version of the first variant. Use of *Encode* and *Decode* is reduced by making use of *CHTYPE*, and also build-up of temporary storage is avoided by continuous re-assignment. For example, in line [24] $R \leftarrow R \uparrow R$, if the middle assignment were to be omitted, there would be the temporary storage of the intermediate result of $\uparrow R$ whilst the transpose took place. Care is taken to make lines restartable.

Lines [23-25] obtain the same functional result as line [10] $R \leftarrow \frac{1}{2} (52 \rho 2) \tau R$ of the first variant. The technique is to encode a mantissa number (integral, but Real) into two Integers, temporarily returned as Real numbers and doubling the space taken until the Floor is applied, but somewhat better than 52 Real numbers that result in the first variant. What we actually need is the lowest 52 bits of the 53 bits that fully define the number. The implicit bit is not required. So the encoding base is $2 \rho 2 * 26$. Using the *CHTYPE* function, the conversion of each Integer to binary form is extremely rapid with no workspace overload. Drop off the first 6 bytes of each representation, join the 26 bits for consecutive pairs of Integers, and the result is the mantissa that is required.

A similar technique is used for the exponent, but here just a single Integer is sufficient, of which the last 11 bits are what is required.

The same techniques are used in reverse for the unpacking process.

This variant will handle 100,000 Real numbers, and is always faster than the other methods.

Table 3 below summarises timing tests. These were actually done under Dyalog APL v 8.1 where the prototypes were developed, simulating *CHTYPE*, so an adjustment has been made for the extra overhead by comparison with the time $\square DR$ takes for a similar conversion. The PC was a 266MHz Pentium II with 64 MB RAM. The available workspace allowed for each evaluation was 4 Megabytes. Where *WS FULL* is indicated, the function line of the variant where it occurred is noted.

The actual time for the full 100,000 number test under mainframe APL*PLUS is about 2 seconds, which makes the PC results look impressive. In fact, the PCs in the target working environment are rated at lower clock speeds.

Test/Variant	5000	10,000	50,000	100,000
Pack - v1	0.660	WS FULL[10]	WS FULL[10]	WS FULL[10]
- v2	0.220	0.440	2.250	WS FULL[12]
- v3	0.170	0.330	1.700	3.200
UnPack - v1	0.380	0.710	3.850	7.960
- v2	0.270	0.550	2.800	WS FULL[27]
- v3	0.170	0.330	1.700	3.300

Table 2: Timing tests showing Variants v. Number of Reals

It is clear that variant 3 not only has greater capacity but is significantly faster. Although, variant 2 could be made faster using similar techniques, investigation of time tests on individual lines suggests that the binary algorithm is not as fast *per se*.

Nested Arrays

The functions *PACK* and *UNPACK* below illustrate a technique for putting together the transfer string, and re-constructing the original structure when dealing with a nested array.

The *PACK* function takes the object to be converted. In fact, this is not a good idea, but I shall come back to that. As you will notice, this function is recursive.

```

v R←PACK X;J
[1]  aV Pack array X
[2]  →LC LB LI LR LH LP['CBIRHN'\DTYPE X]
[3]  LC:R+0 PKCHAR X ◊ →EXIT
[4]  LB:R+0 PKBOOL X ◊ →EXIT
[5]  LI:R+0 PKINT X ◊ →EXIT
[6]  LR:R+0 PKREAL X ◊ →EXIT
[7]  LH:R+0 PKHET X ◊ →EXIT
[8]  LP:R+0 PKCTRL 0 7 0, (pJ), J+pX
      a data length, data type, pack, rank, shape
[9]  LP1:→LP3 LP2[1+0eJ]
[10] LP2:X+1+, X a to get correct prototype
[11] LP3:R+R, >, /PACK'', X
[12] EXIT:
v

```

Firstly, the function determines the data type of the object it is dealing with. If it is a simple object, it calls the appropriate conversion function and returns the packed result. I am assuming here that the conversion function places a header in the form discussed earlier before the data part. This code is not included in the test versions for the *PKREAL* functions discussed earlier. A 0 left argument indicates *pack*, and 1 indicates *unpack*.

How the *DTYPE* function is written is platform dependent, or of course it could be written in standard APL. I leave that as an exercise.

If the array is nested, the header for the tuple is determined. There is no data part for a pointer tuple; it simply identifies the structure. The tuples that define its elements are then converted to string form by a recursive call for each element of the structure. The individual results are catenated to the header and returned. Note that if the pointer structure is empty, we still need to build its prototype which is achieved by a recursive call on the 1+ of the structure.

The *UNPACK* function makes use of a global *BUFFER* which will initially contain the transfer string to be re-constructed into an APL array. This is a string of consecutive tuples, each made up of a header and, usually, a data part, as discussed. Again, this is a recursively called function.

```

▽ R←UNPACK DUMMY;C;X
[1]  ⍵ Unpack transfer string from global BUFFER.
[2]  R←''
[3]  →(0⊆pX+PKNEXT) EXIT
[4]  C←1 PKCTRL X ⍵ hdr length,data type,pack method,rank,shape
[5]  →LC LB LI LI LI LR LH LP[1+2>C]
[6]  LC:R←1 PKGHAR X ⍵ →EXIT
[7]  LB:R←1 PKBOOL X ⍵ →EXIT
[8]  LI:R←1 PKINT X ⍵ →EXIT
[9]  LR:R←1 PKREAL X ⍵ →EXIT
[10] LH:R←1 PKHET X ⍵ →EXIT
[11] LP:R←(4+C)⊃UNPACK''(1[,×/4+C)⊃0
[12] EXIT:
▽

```

Its first action, through function *PKNEXT*, is to get the next tuple from the buffer whose length is in the first 4 bytes of the header for each tuple, and then to drop those bytes from the buffer.

Function 1 *PKCTRL X* translates the header information as commented.

The data type then identifies how to unpack the string. If it is a simple data type, the appropriate function is called with the full tuple including header. Each

function returns the re-constructed and re-shaped array which in turn is returned by *UNPACK*.

If the data type is nested, function *UNPACK* is called recursively for each element at this level of the nested structure determined from the shape, but at least once to generate the correct prototype. The dummy argument to *UNPACK* is there simply to obtain the required number of calls. The result is reshaped and returned. On completion, the original array should be returned, and the buffer should be empty.

So what is wrong with the *PACK* function ? The problem is that it passes down at each recursive call a part of the structure that is also taking up storage space at each calling level. If the object is large, a lot of workspace can disappear very quickly.

The technique I actually use is to assign the object to be packed to a global named, say *OBJECT*, and pass the Pick index as the argument to function *PACK*, not the object itself. For the first call, this Pick index is (10), since (10)=*OBJECT* is itself. Thereafter, at each recursive call for a nested object, its Pick index is extended so that it selects the item at the next level. A further refinement is to ravel the item from *OBJECT* that is currently being processed before the recursive call (its shape has already been recorded in the header), so that the Pick index is always a simple vector - a nested Pick index takes considerably more space. Pick assignment in Dyalog APL makes it straightforward to ravel the element, but the process is slightly more complicated otherwise, e.g. for Pick index *I*:

$(I>\underline{OBJECT})+ , I>\underline{OBJECT}$

Finally, when an element of the structure has been dealt with, it is set to 0:

$(I>\underline{OBJECT})\leftarrow 0$

This is done as soon as the object is read for processing in the individual conversion functions, or after the recursive call for a pointer structure. Thus, the space in the object is freed up and replaced by the generated transfer string. On completion, the global object has the scalar value 0.

Compression Methods

There are many ways to attempt to compress data. The ubiquitous PKZIP product does it extremely well and quickly. In APL, the interpretive overhead rules out many algorithms from practical consideration. I have found only three that are

worth implementing from the viewpoint of the trade-off between processing cost and transfer volume.

Zero compression

Numeric data often contains a lot of zeros. A well-known method of reducing such data uses a Boolean expansion vector, where 0 marks a zero data item, and the compressed numeric data. It is simple to test for zeros, indeed the Real data conversion already requires it, and it is simple to expand, all at very little extra processing overhead. The criterion as to whether it is worth doing may be expressed as:

$$((pbool) \div 8) < bytes \times + / - bool$$

where *bool* is the boolean vector and *bytes* is the element size of the numeric data (1, 2, 4 or 8 bytes). In other words, is the extra space required for the boolean vector less than the space saved. The header protocol that I use allows each of the 8-bits of the Compression Method byte to identify up to 8 different packing methods. This is one of them.

Vector of Text Vectors (VTV)

This is such a commonly-used structure for holding lists of items that I find it worthwhile making it a special case. The APL internal structure is not efficient for large lists, though it can still sometimes improve on a Text Matrix with many trailing blanks. This case is handled by converting the VTV to a linelist[†] using any of the $\square TC$ characters that do not occur in the enlisted text as delimiters (if they all do, it is treated as a nested array). Both APLs provide external functions to make the conversion to and from a linelist very rapid. As a linelist, it can then be treated as a simple Character vector, but with its packing method noted.

† A linelist is a character vector representing a list whose items are separated by an arbitrary character that does not occur in the list items. Its advantage is that unnecessary blanks may be omitted.

Heterogeneous Arrays

A heterogeneous array is something of an oddball. It contains a mix of scalar Character and Numeric items, at least one of each type. Although notionally a simple array, it is usually stored as a nested structure with the top-level tuple pointing to, sometimes many, scalar tuples. This is horribly expensive in storage terms, requiring at least 12 bytes per item. Uniquely, to my knowledge, Apl+Win

has a special data type for heterogeneous arrays that does away with the pointer tuple.

The approach I have adopted for transfer is to split the data part of a heterogeneous array into three vectors, being:

- a boolean vector that identifies Numeric items by 1 and Character items by 0
- a vector of just the Numeric items
- a vector of just the Character items

The converted form of a heterogeneous array has a header identifying heterogeneous type and shape whose total tuple length includes the converted forms for each of the three vectors. Since the numeric part is handled by one of the normal conversion functions, zero compression will be employed if worthwhile.

More in hope...

I am always a little disappointed when I have to find an alternative to a pure APL approach to a problem. Faster machines have had a big impact on making an APL solution, previously untenable, more than just acceptable, even when performance is an issue. The implementers, and with all due credit, are aware of many speed issues, and indeed improvements and new facilities are constantly being added to APL. But there remain overlooked corners.

This exercise has firmly placed the focus on *Encode*. My work often involves use of this primitive in the efficient handling of large-scale, multi-dimensional data. Its converse, *Decode*, and also *Index Of* and *Membership*, are heavily overworked in this context. The latter two functions have long benefited from clever algorithms that improve performance. I make the same plea for *Encode* when results should be Integer or Boolean. The workspace overload in the latter case is particularly excessive, a case that is straightforward algorithmically when dealing with data that is exactly integer (including Real representations). After all, it is represented in binary form. I also have my doubts about the processing for *Decode*, which can give unexpected *WS FULLs*.

By the bye, when is a Boolean not a Boolean? Try *bool_matrix,0* in Dyalog APL. The result has 1-byte Integer type (each item takes 1 byte). The reason is because scalar 0 is actually stored as a 1-byte Integer since this is a common use, and there are small performance gains to be made in general. But if *bool_matrix* is very large, the 1-byte Integer result may have a substantial workspace impact. There are a number of ways to avoid this, and some that might

be expected to that do not. Thus, *bool_matrix,0=1* does not work, but *bool_matrix,0=0* does. Life has it little surprises. Workspace compaction, explicitly caused by `%WA` reduces all arrays to their most compact form, as does `%SIZE'array'` on that specific array. For this reason, in my *DTYPE* function, I apply `%SIZE` on the argument before using `%DR` to obtain its data type.

So maybe my anticipation will be fulfilled?

Index to Advertisers

Causeway Graphical Systems Ltd	6
Dyadic Systems Ltd	2
Russian Fund Appeal	42
Strand Software	48
Vector Back Numbers	8

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (hardcopy with diskette as appropriate) to the Vector Working Group via:

Vector Administration, c/o Gill Smith
Brook House
Gilling East
YORK, YO6 4JJ
Tel: +44 (0) 1439-788385
Email: apl385@compuserve.com

Authors wishing to use Word for Windows should contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template. These may also be downloaded from the Vector web site at www.vector.org.uk

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO6 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

British APL Association: Membership Form

Membership is open to anyone interested in APL. The membership year normally runs from 1st May to 30th April, but new members may join from 1st August, November or February if preferred. The British APL Association is a special interest group of the British Computer Society, Reg. Charity No. 292,786

Name: _____
 Address: _____

 Postcode / Country: _____
 Telephone Number: _____
 Email Address: _____

Category (please tick box) to run from: 1st May ☐ August ☐ Nov ☐ Feb ☐
 UK private membership £12 ☐
 Overseas private membership £14 ☐
 Airmail supplement (not needed for Europe) £4 ☐
 UK Corporate membership £100 ☐
 Corporate membership overseas £135 ☐
 Sustaining membership £430 ☐
 Non-voting UK member (student/OAP/unemployed only) £6 ☐

PAYMENT — in Sterling or by Visa/Mastercard/JCB only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard, Visa or JCB number.

I authorise you to debit my Visa/Mastercard/JCB account

Number: Expiry date: |

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

(please tick the required option above)

Data Protection Act:
 The information supplied may be stored on computer and processed in accordance with the registration of the British Computer Society.

Signature: _____ Send the completed form to:

British APL Association, c/o Rowena Small, 8 Cardigan Road, LONDON E3 5HU, UK

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1998/99 Committee

Chairman	Anthony Camacho 0117-973 0036 100612.1057@compuserve.com	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Secretary	Ajay Askoolum 01737-771643 106173.3347@compuserve.com	42 Hanworth Road, Redhill, Surrey RH1 5HT
Treasurer	Nicholas Small 0181-980 7870 treas.apl@bcs.org.uk	8 Cardigan Road, LONDON E3 5HU
Journal Editor	Stefano Lanzavecchia stf@adaytum.dk	c/o Vector Administration, Brook House, Gilling East YORK YO62 4JJ
Projects and Publicity	Dr Alan Mayer 01792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Webmaster	Ray Cannon 01252-874697 100430.740@compuserve.com	21 Woodbridge Road, Blackwater, Camberley, Surrey GU17 0BS
Activities	Jon Sandles 01904-612882 jon_sandles@csi.com	138 Burton Stone Lane, YORK YO30 6DF
Education	Dr Ian Clark 01388-605544 100021.3073@compuserve.com	Unit R21, Auckland New Business Centre, St. Helen Auckland, Bishop Auckland, Co. Durham DL14 9TX
Administration	Rowena Small 0181-980 7870 treas.apl@bcs.org.uk	8 Cardigan Road, LONDON E3 5HU

Journal Working Group

Editor:	Stefano Lanzavecchia	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Jonathan Barman (01488-648575), Richard and Adam Weber (0121-3546550), Anthony & Sylvia Camacho, Ray Cannon (01252-874697), Marc Griffiths (01653-691745), Bob Hoekstra (01483-771028), Jon Sandles (01904-612882)	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" — an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd
The Maltings, Castlegate,
MALTON, North Yorks YO17 7DP
Tel: 01653-666760
Fax: 01653-697719
Email: causeway@csi.com
Web: www.causeway.co.uk

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants. RG24 0AL
Tel: 01256-811125
Fax: 01256-811130
Email: sales@dyadic.com
Web: www.dyadic.com

Insight Systems ApS
Nordre Strandvej 119C
DK-3150 Hellebæk
Denmark
Tel: +45 49 76 20 20
Fax: +45 49 76 20 30
Email: info@insight.dk
Web: www.insight.dk

Soliton Associates Ltd
Groot Blankenberg 53
1082 AC Amsterdam
Netherlands
Tel: +31 20 646 4475
Fax: +31 20 644 1206
Email: sales@soliton.com

Compass Ltd
Compass House
60 Priestley Road
GUILDFORD, Surrey GU2 5YU
Tel: 01483-514500

HMW Trading Systems Ltd
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel: 0171-353 8900
Fax: 0171-353 3325
Email: 100020.2632@compuserve.com

APL2000
Rapid Application Development
6610 Rockledge Drive
Suite 502
Bethesda, MD 20817
USA
Email: sales@apl2000.com
Web: www.apl2000.com

Dutch APL Association
Postbus 1341
3430BH Nieuwegein
Netherlands
Tel: +31 347 342 337